

Chaos Engineering (Pawlikowski) — Complete Study Guide - Claude

Start here — navigation and study workflow

Open one chapter at a time. Each chapter begins with a study route; its original section numbers remain available for cross-reference. Experiment fields follow purpose → setup → execution → interpretation → limits. Use the Master Cheat Sheet for revision; expand each review answer after attempting the question.

Supplementary explanation — Editorial verification: How to read this refactor

The linked Notion page is the canonical source for this structural refactor. Original technical prose, quotations, commands and reported outcomes are retained. Clearly labelled editorial verification notes take precedence over the specific original claims they correct. Book-derived material and existing supplementary explanations remain distinct. No experiment was executed for this edit; version-sensitive examples remain historical examples, not current compatibility guarantees.

Shell example caveat — High confidence. Some original command listings place comments after a continuation backslash. Bash continues a line only when the unquoted backslash is immediately followed by the newline. Retained annotated listings with trailing text after `\` need syntax repair before use. Source: Bash escape character.

- Navigate to a section

Source: *Chaos Engineering: Site Reliability through Controlled Disruption* — Mikolaj Pawlikowski (Manning, 2021). 426 pages, 13 chapters, 4 appendices. Forewords by Casey Rosenthal and Nora Jones.

Scope: eight selected core chapters (source chapters 1–3, 5–6, and 10–12), plus all appendices, the Master Cheat Sheet, Completeness Audit and Hands-on Labs. Theory and practice carry equal weight.

How to use this guide

This guide is a primary study and reference resource, not a summary. Use it three ways.

- **Study:** read a chapter's *Theory* sections, then its *Experiment Cards*, then *Theory ↔ Practice*, then *Key Takeaways*.
- **Revise:** the **Master Cheat Sheet** at the end is self-contained. It holds definitions, methodology, every experiment, every command, checklists, mental models, a glossary, and 40 review questions with answers.
- **Work at a keyboard:** use the cheat sheet's **Tool and command reference** and **Failure-injection matrix**.

Conventions

- **Theory** sections answer: what it is, why it matters, how it works, the mental model, its relationships to other concepts, failure modes, trade-offs, misconceptions, and when to use it.
- **Practice** sections answer: scenario, architecture, hypothesis, steady state, observability signals, injected failure, tools, commands, procedure, expected vs. observed behaviour, why the result happened, the operational lesson, and production implications.
- **Experiment Cards** hold the book's experiments in one fixed structure. Every card states *why* the result happened.
- **Callouts** mark the ideas that carry the most weight: load-bearing claims, traps, transferable technique, and definitions.
- **Diagrams** are described in words: which components exist, how they interact, and **which failure paths the diagram reveals**.
- Content that is not from the book is labelled "**Supplementary explanation**." Direct phrases from the author are quoted.

The author corrects his own book. Appendix C states that the four-step model is really **five steps**. A reviewer asked why there was no *analysis* step. The author admits he kept it implicit "primarily for the promotional reasons: fewer steps sound easier and are admittedly catchier." Read every experiment as **observability → steady state → hypothesis → run → analysis**. Every Experiment Card carries *Interpretation* and *Lesson learned* fields for this reason.

- Chapter directory — topics at a glance

Table of contents

1. **Chapter 1 — Into the World of Chaos Engineering** · definition, motivations, risk and SLI/SLO/SLA, emergent properties, the four-step model, what it is not, the FizzBuzzAAS case study
2. **Chapter 2 — First Cup of Chaos and Blast Radius** · the lab VM, Linux forensics (exit codes, signals, the OOM Killer), the first experiment, blast radius, the systemd restart-limit bug

3. **Chapter 3 — Observability** · the USE method, the resource map, `uptime/dmesg`, block I/O, networking, RAM, CPU, OS-level tracing, application profiling, Prometheus and Grafana
 4. **Chapter 4 — Poking Docker** · virtualization vs. containers, the seven kernel features, chroot and union filesystems, namespaces, cgroups, Docker networking, capabilities and seccomp, a DIY container in three parts, Pumba
 5. **Chapter 5 — Who You Gonna Call? Syscall-Busters!** · syscalls and libc, `strace` and its 100× cost, BPF/BCC, `-e inject`, seccomp the easy and the hard way
 6. **Chapter 6 — Chaos in Kubernetes** · what Kubernetes is and why, Minikube, pods/deployments/services, RBAC, labels, killing pods, Toxiproxy and the added-degraded-replica technique
 7. **Chapter 7 — Automating Kubernetes Experiments** · PowerfulSeal policies, match→filter→act, the clone + toxiproxy mutation, continuous SLO verification, MTTF at scale, regions and availability zones, VM-level experiments
 8. **Chapter 8 — Under the Hood of Kubernetes** · etcd and Raft, kube-apiserver, controller-manager, scheduler, kubelet and the pause container, CRI and the runtime zoo, pod/service/ingress networking — with experiment ideas for each
 9. **Appendices A–D** · installation, the consolidated pop-quiz answers, what the author left out and why, and the recipes
 10. **Master Cheat Sheet** · definitions · methodology · theory summary · experiment playbook · tool and command reference · experiment-design checklist · observability checklist · failure-injection matrix · Docker mental model · Kubernetes mental model · injection-layer comparison · SLI/SLO/SLA reference · common failure patterns · production safety and blast-radius checklist · glossary · rapid exam review
 11. **Completeness Audit**
 12. **Hands-on Labs**
-

Chapter 1 — Into the World of Chaos Engineering

Study route

Concepts: §§1.1–1.4 → worked case and Experiment Card 1.1: §1.5 → Theory ↔ Practice → Key Takeaways.

Supplementary explanation — Editorial verification: Availability arithmetic and the firewall example

Correction — High confidence. At 99.999% availability, daily downtime is $86,400 \times 0.00001 = \mathbf{0.864 \text{ seconds (864 ms)}}$, not the original 840 ms. This also applies to cheat-sheet §12. The Google SRE availability table gives a rounded daily figure.

Command qualification — High confidence. The original `iptables -A ${CACHE_SERVER_IP} -j DROP` and matching `-D` example omit a chain and destination selector. In iptables syntax, `-A/-D` take a chain; destination matching uses `-d`. A rule for locally generated egress would target the appropriate OUTPUT chain and destination, with rule ordering checked. Retained examples are not valid literal IP-targeting commands as written.

1.1 Theory — What chaos engineering is

Definition (the book's, from principlesofchaos.org): chaos engineering is "the discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production."

Pawlikowski's compression: **a software testing method that finds evidence of problems before users experience them.**

The governing analogy: crash tests. Engineers test a car's subsystems in isolation — airbags, wipers, crumple zones. That does not prove a passenger survives a real crash. So the industry drives a production car into a concrete block at a controlled speed, films it with high-speed cameras, and measures the dummies. Chaos engineering applies the same move to software: a deliberate, observed, controlled disruption of the **whole** system, in lifelike conditions.

Three parts of that analogy carry the whole book:

- **Controlled speed** → blast radius. You choose the severity.
- **High-speed cameras** → observability. Without measurement, the crash teaches nothing.
- **Production car, not a prototype** → realism. You test the system you ship.

Why the whole system matters. Your code can be correct and your system can still fail. The book's list of causes: floods and earthquakes that destroy datacenters, power cuts, hardware failures, networking problems, resource starvation, race conditions, unexpected traffic peaks, unaccounted-for interactions between components, and human error. More complexity creates more opportunities to fail.

The book uses real events, not hypotheticals: two lunar crash landings in 2019 (Chandrayaan-2, Beresheet), and major outages at Google Cloud, Cloudflare, Facebook/WhatsApp and Apple inside roughly one month in summer 2019. The lesson is dependency. A

perfect system still inherits the failure modes of the systems it depends on.

Misconception #1, stated in Chapter 1 and repeated all book: chaos engineering is *not* "randomly breaking things in production." Production experimentation is a unique capability of the discipline, but it is a subset, not the definition. Chaos engineering interfaces with SRE, performance analysis and other forms of testing.

1.2 Theory — Three motivations

1.2.1 Risk, cost, and SLI / SLO / SLA

Memorize this chain. It is the business justification for everything that follows.

1. A system exists to serve a business model. "Running well" derives from business objectives, not engineering taste.
2. Each engineering risk has a **cost per unit of time**: direct lost business plus intangibles such as damage to public image. The book's number — Forbes estimated Amazon lost **\$66,240 per minute** of downtime in 2013.
3. **SLIs** (service-level *indicators*) quantify risk. An SLI puts a number on an event: "percentage of time users can access the website"; "ratio of requests served by the cat-photo service within a time window."
4. Two parties who agree on a range of an SLI create an **SLO** (service-level *objective*) — a tangible target for the engineering team.
5. An SLA (service-level *agreement*) is an SLO enforced by contract, with a penalty if you miss it.

	Consumer photo site ("Bookface")	Financial trading API
Main risk	People can't access the website (downtime)	Slow responses; speed is critical
SLI	Ratio of success responses to errors from our servers	99th percentile response time
SLO	> 99.95% on average monthly	99th percentile < 25 ms, 99.999% of the time
Engineering consequence	Cheap-ish; tolerate rare single-photo failures	"Mission impossible" — only ~5 min/year where the slowest 1% averages over 25 ms. Expensive to build.

Number of nines — memorize this table.

Availability	Downtime per year	Downtime per day
90% (one nine)	36.53 days	2.4 hours
99% (two nines)	3.65 days	14.40 minutes
99.95% ("three and a half nines")	4.38 hours	43.20 seconds
99.999% (five nines)	5.26 minutes	840 milliseconds

The book flags that "three and a half nines" is popular but not technically correct. In error-budget terms, 99.9 → 99.95 is a factor of 2, while 99.9 → 99.99 is a factor of 10. (The text says "a factor of 5" for the step it describes. The point holds: the scale is multiplicative, not linear.)

Where chaos engineering plugs in. To satisfy an SLO you engineer for sinister scenarios. The only way to know how the system behaves in those scenarios is to create them. Work *backward*: business goal → SLO → a condition you can test continuously. This is the seed of Chapter 11's continuous SLO testing.

1.2.2 Testing a system as a whole

The testing ladder:

- **Unit tests** — single functions or small modules, in isolation.
- **Integration / end-to-end tests** — assembled components that mimic a real system; verify correct behaviour.
- **Benchmarking** — performance, from micro-benchmarks to simulated client loads.
- **Chaos engineering** — "the next logical step." Like end-to-end testing, but you *rig the conditions* to introduce the failure you expect, then measure that you still get the correct answer inside the expected time frame.

Easy to miss: the book states that even a **single-process system** can be tested with chaos engineering techniques. Part 2 proves it. Chapters 6–9 inject failure into syscalls, a JVM, an application's Redis client and a browser. Chaos engineering is not synonymous with distributed systems.

1.2.3 Finding emergent properties

Mental model. Emergent properties are behaviours of the whole that no part possesses. The book's analogies: one heart cell does not pump blood, but the right configuration of cells does; single neurons do not think, but their interconnected collection does.

The canonical software example — referenced throughout the book:

1. Many services use a DNS server to find each other.
2. Each service retries DNS errors **up to 10 times**.
3. External users are also told to retry on failure.
4. The DNS server fails and restarts.
5. On restart it meets traffic **amplified by the layers of retries**, far more than its provisioned capacity.
6. It fails again, enters a restart loop, and the whole system goes down.

No component has the property "creates infinite downtime." The system does. This is the archetype of what chaos engineering finds best: "simple, predictable failures cascading into large problems."

Supplementary explanation (not the book's term): the industry names for this shape are **retry storm** and **metastable failure** — a system that cannot recover after the original trigger is gone, because recovery itself generates the load that keeps it down. The book describes the mechanism exactly but does not use these labels.

Chaos engineering and randomness. The book allows borrowing from **fuzzing** — feeding pseudorandom payloads to find errors your written tests miss. Randomness is a legitimate tool. But you must control experiments to *understand results*. Random destruction without control produces incidents, not knowledge.

1.3 Theory — The four-step experiment model

This model is the spine of the book. Every later experiment is an instance of it.

1. **Observability** → 2. **Steady state** → 3. **Hypothesis** → 4. **Run the experiment**

Step 1 — Ensure observability. You must see the metric you care about *reliably*. The book warns that OS and hardware metrics carry their own bugs and caveats: "If the process you're using to measure CPU load ends up using more CPU than your application, that's probably a problem." Chapter 6 turns that warning into a measured number for **strace**.

Step 2 — Define a steady state. Use observed data to define *what normal is*, so you can detect abnormality. The book's examples: "CPU load on a 15-minute average below 20% for application servers during the working week"; "500 to 700 requests per second per instance running with four cores on our reference hardware"; "99% of our users can access our API in under 200 ms."

The book's caveat about steady state on a modern Linux server: many variables sit outside your control. Is your process getting enough CPU, or is the CPU stolen? Did a cron job start mid-experiment? Did the kernel scheduler prefer a higher-priority process? Does a hypervisor give your CPU to another tenant? Mitigation: repeat experiments many times so hidden variables surface.

Step 3 — Form a hypothesis. Make an educated, *testable* guess about system behaviour under a well-defined problem. The book's categories:

- External events (earthquakes, floods, fires, power cuts)
- Hardware failures (disks, CPUs, switches, cables, power supplies)
- Resource starvation (CPU, RAM, swap, disk, network)
- Software bugs (infinite loops, crashes, hacks)
- Unsupervised bottlenecks
- Unpredicted emergent properties
- Virtual machine layer (JVM, V8, others)
- Hardware bugs
- Human error (wrong button, wrong config, wrong cable)

The book's own phrasing: "If we take 30% of our servers down, the API continues to serve the 99th percentile of requests in under 200 ms." "If one of our database servers goes down, we continue meeting our SLO."

Note the shape: **condition → measurable outcome bounded by a number**. A hypothesis without a number is not a hypothesis.

Step 4 — Run the experiment and prove or refute. The scoring rule is the cultural heart of the discipline:

- Right? More confidence in the system.
- Wrong? You found a problem before your clients did, and you can fix it before anyone gets hurt.

Craftsmanship rule, enforced for the rest of the book: "The simpler your experiment, usually the better. You earn no bonus points for elaborate designs, unless that's the best way of proving the hypothesis."

The datacenter power-supply illustration. A datacenter has two independent power sources. In theory you are covered. In practice the automatic switchover may not work, the site may have outgrown one supply, or nobody paid the electrician for quarterly checks. Run this for each power source, one at a time:

1. Check that The Website is up.
2. Open the electrical panel and turn the power source off.
3. Check that The Website is still up.
4. Turn the power source back on.

Crude, obvious, and still a complete chaos experiment: system + characteristic + designed disruption + observation. The book then asks the uncomfortable question — *what if it fails?* Then you created your own outage. Hence: "a big part of your job will be about minimizing the risks coming from your experiments and choosing the right environment to execute them."

The Mendel framing. Gregor Mendel formed an intuition about heredity and designed experiments on yellow and green peas. His results did not match his expectations, and that mismatch produced the breakthrough. The book returns to pea genetics as the template of good experimental method.

1.4 Theory — What chaos engineering is NOT

Read this list before any conversation with management.

- **Not a silver bullet.** It does not fix your system automatically, and it may not apply to your use case.
- **Not random destruction.** Chaos Monkey made randomness famous, but "adding failure is the easy part; the hard part is to know where to inject it and why."
- **Not a tool.** It is not Chaos Monkey, Chaos Toolkit, PowerfulSeal, or any GitHub project. Tools implement experiment types. The real difficulty is "learning how to look critically at systems and predict where the fragile points might be."
- **Not a replacement for unit or integration tests.** It complements them. Engineers test airbags in isolation *and* again inside the car during the crash test.
- **Not a source of ready-made answers.** Value depends on your system, your understanding of it, and your observability.
- **Not only about production.** Production is unique to the discipline but not its main focus. Much value comes from other environments.
- **Not chaos theory.** No mathematical lineage, despite the name.

1.5 Practice — A taste: the FizzBuzzAAS outage

The first case study is a narrative, not a lab. It is still a complete four-step experiment, and it sets the pattern for everything after.

Setting. Fictional island country Glanden; capital Donlon; startup **FizzBuzzAAS Ltd.**, selling FizzBuzz as a Service on a flat monthly subscription. Competitor: the real FizzBuzzEnterpriseEdition repo, which returns in Chapter 7 as the JVM target. Betty works in sales. Alice and Bob are the engineering team.

Architecture (Figure 1.3). External request → **load balancer** → one of **two identical API server instances** → **cache**. The API server serves a precomputed response if it is fresh enough. Otherwise it computes a new one and stores it in the cache.

Read the diagram for failure paths. That is the skill the book trains:

- The load balancer is a single entry point — an availability chokepoint.
- The two API instances are identical, so any bug in one exists in both. Redundancy protects against *machine* failure, not *logic* failure.

- The cache is a dependency on the request path. What the code does when the cache is slow or unreachable is the whole story.

The incident. The biggest client cannot access the API. The symptoms confuse the team: servers reachable, servers reporting healthy, expected processes running and responding. Alice finds it in the logs — "I don't see any errors, but all of these requests seem to stop at the cache lookup."

Root cause, in two layers:

- *Code layer:* the code handled the cache being **down** (connection refused, no host). It had **no time-outs for no response at all**. A hung connection is not a refused connection, so the requests blocked forever.
- *Operational layer:* a **badly rolled-out firewall policy** stopped the API servers reaching the cache. Someone forgot to whitelist it. Human error.

The distinction that makes this chapter matter: *failing fast* and *hanging* are different failure modes. Code that handles one may be defenceless against the other. A dropped packet (silence) behaves differently from a rejected connection (immediate error). This distinction recurs in Chapter 4 (slow connections), Chapter 8 (Redis latency vs. failed requests) and Chapter 9 (browser latency vs. failure).

Postmortem and fix. Alice asks how to be immune next time. Bob jokes about "setting some of our servers on fire once in a while." Alice takes the joke seriously: if we can simulate a broken firewall rule, we can put it in our integration tests.

The book's first chaos command, written by Bob on the whiteboard:

```
# Inject: silently drop all traffic to the cache server (simulates the bad firewall rule)
iptables -A ${CACHE_SERVER_IP} -j DROP

# Revert: delete that rule again
iptables -D ${CACHE_SERVER_IP} -j DROP
```

What they do: **-A** **appends** a rule; **-D** **deletes** the same rule; the **-j DROP** target makes the kernel discard matching packets **silently** — no RST, no ICMP rejection. That silence reproduces a hang instead of a connection-refused error. **-j REJECT** would produce the failure mode their code already handled, which is exactly the point.

They wired both commands into the **setup and teardown of their integration tests**, confirmed the old version broke and the new version survived, and updated their LinkedIn titles to SRE.

Experiment Card 1.1 — Cut the cache off (the FizzBuzzAAS experiment)

Purpose and concepts

Goal

Prove the API keeps serving when the cache is unreachable in the *hanging* mode, not just the *refused* mode.

Relevant theory

The four-step model (§1.3); emergent behaviour of a dependency on the request path; graceful degradation; fail-fast vs. hang.

Setup, hypothesis and boundaries

System / setup

Load balancer → 2 identical API server instances → cache. Injection on the API server host.

Hypothesis

"If we drop connectivity to the cache, we continue getting a successful response."

Steady state

The API responds successfully.

Observability signal

Success or error of an API call.

Blast radius

One host's egress to one dependency, inside an integration-test environment, reverted in teardown. In the story it was first fixed by a hot-fix in production — the book presents that as the *bad* version of events.

Execution and tools

Failure injected

All packets to the cache server IP dropped silently (`iptables ... -j DROP`).

Tools / commands

`iptables -A ${CACHE_SERVER_IP} -j DROP` (inject) / `iptables -D ${CACHE_SERVER_IP} -j DROP` (revert), run as test setup and teardown.

Procedure

1. Confirm API returns success.
2. Add the DROP rule.
3. Call the API and observe.
4. Delete the DROP rule.
5. Repeat against the fixed build.

Results, interpretation and lessons

Expected behaviour (pre-fix)

The team assumed graceful degradation, because cache-down was "handled."

Observed result

Pre-fix: requests hang at the cache lookup and never complete, with no errors in the logs. Post-fix (time-outs added): requests complete successfully without the cache.

Why it happened

Error handling covered *connection refused / no host*, not *no response*. With packets dropped, the socket waits; without a time-out, the request thread waits with it.

Lesson learned

Every network call on a request path needs an explicit time-out. Handling "down" is not handling "slow" or "silent." Absence of errors in logs is not evidence of health.

Limits and follow-up

Production considerations

A DROP rule in production affects every request routed through that host. Bound the blast radius, and always script the teardown so a crashed test cannot leave the rule behind.

Theory ↔ Practice connections for Chapter 1

- **Four-step model (§1.3) ↔ FizzBuzzAAS (§1.5):** observability = "can we call the API successfully"; steady state = "the API responds successfully"; hypothesis = "if we drop connectivity to the cache, we continue getting a successful response"; experiment = confirm the old version breaks and the new one works. The book maps all four explicitly.
- **Emergent properties (§1.2.3) ↔ retry storms:** the DNS-restart example is the theory. Chapter 4's WordPress network experiments and Chapter 10's Kubernetes network experiments show amplification and cascade in a lab.
- **SLI/SLO/SLA (§1.2.1) ↔ Chapter 11:** continuous SLO verification with PowerfulSeal turns the paper SLO into an executing test.
- **"Even a single-process system" (§1.2.2) ↔ Chapters 6–9:** syscall, JVM, application and browser fault injection.
- **Blast radius warning (§1.3, power-supply example) ↔ §2.5:** where blast radius gets its formal treatment.

Key Takeaways — Chapter 1

1. Chaos engineering means experimenting on a system to build confidence that it withstands turbulent conditions. It finds evidence of problems before users do.
2. The crash-test analogy carries three requirements: controlled severity, reliable measurement, and a realistic whole system.
3. Three motivations: quantify risk and drive SLIs/SLOs/SLAs; test the system as a whole; discover emergent properties.
4. SLI = the number. SLO = the agreed target. SLA = the contract with a penalty. Chaos experiments keep an SLO honest.
5. The four steps — observability, steady state, hypothesis, run — are the atomic unit. Everything later is an instance of it.
6. Being wrong is a win. A refuted hypothesis is a bug found before a customer found it.
7. Chaos engineering is not randomness, not a tool, not a replacement for other tests, not production-only, and not chaos theory.
8. Simplicity beats cleverness in experiment design.
9. Handling "dependency is down" is not handling "dependency is silent." Put time-outs on every call on the request path.
10. Experiments carry their own risk. Choosing the environment and bounding the damage is part of the engineering.

Chapter 2 — First Cup of Chaos and Blast Radius

Study route

Orient to the lab: §§2.1–2.2 → mechanisms: §2.3 → first experiment: §2.4 → blast radius: §2.5 → repeated-failure experiment and interpretation: §2.6 → connections and takeaways.

Supplementary explanation — Editorial verification: Signal exit status and OOM evidence

Correction — High confidence. Bash reports a signal-terminated command as 128 + signal number: **143 corresponds to SIGTERM (15); 137 corresponds to SIGKILL (9)**. The Linux 5.4 OOM killer sends SIGKILL, so the claim that its normal kill explains 143 is incorrect. A status alone does not establish who sent a signal or prove an OOM event; correlate kernel logs. This correction also applies to the chapter takeaways and review question 6. Sources: Bash exit status, Linux 5.4 OOM-killer implementation.

2.1 Practice — The lab environment

The book ships a **VirtualBox VM image** so every command runs with matching tool versions.

1. Download the VM image from <https://github.com/seeker89/chaos-engineering-book> → **Releases** → latest release. Follow the release notes: download several files, verify them, decompress them.
2. Install VirtualBox.
3. **File > Import Appliance**, pick the image, follow the wizard.
4. Change these settings: General > Advanced > **Shared Clipboard = Bidirectional**; System > Motherboard > **4096 MB** base memory; Display > **Video Memory ≥ 64 MB**; Display > Remote Display > **uncheck Enable Server**; Display > Graphics Controller = VirtualBox's recommendation.
5. Start and log in. **Username and password are both chaos.**

Gotcha: paste into an Ubuntu terminal with **Ctrl-Shift-C / Ctrl-Shift-V**, not Ctrl-C / Ctrl-V. VMware works if VirtualBox misbehaves. All source, including the code that builds the VM, is in that repo.

The VM itself teaches a chaos-engineering lesson. You need an environment where you can be **playful** — fill the RAM, kill processes at random, break the network — without the blast radius reaching your laptop. Choosing the environment is part of experiment design.

2.2 Practice — The scenario

The Glanden team returns. Early clients complain the product "sometimes doesn't work." Engineers who test it see nothing wrong. The book flags this as the realistic case: *the problem already exists, existing testing finds nothing, the clock is ticking*.

You get the two things you usually get in real life: **the architecture** and **the logs**.

Architecture (Figure 2.1) — FizzBuzz as a Service, now concrete:

- An **NGINX load balancer**.
- **Two identical copies of an API server**, written in Python.
- Flow: (1) client makes an HTTP request → (2) LB picks an instance that is up → (3) if that instance becomes unavailable, the request fails → (4) LB retransmits to the other instance → (5) instance B responds successfully → (6) LB returns the response.

The internal failure in step 3 should be invisible to the user.

Failure paths in this diagram: the LB is the single entry point; the retry from step 3 to step 4 is the *only* thing that hides instance death; and if both instances are unavailable at once, nothing is left to retry onto.

The logs:

```
[14658.582809] ERROR: FizzBuzz API instance exiting, exit code 143
[14658.582809] Restarting
[14658.582813] FizzBuzz API version 0.0.7 is up and running.
```

Two clues: instances **restart**, and there is an **exit code**. To use them you need Linux forensics.

2.3 Theory + Practice — Linux forensics 101

Definition (the book's): a **black box** is a system that is opaque to us — we see inputs and outputs but not the inner workings. A **white box** is the opposite. Chaos engineering often operates on black boxes. This chapter finds and fixes a real weakness **without ever reading the application source**.

2.3.1 Exit codes

Mechanism. A process that terminates returns a number — the **exit code** — that tells the caller whether it succeeded. Read the last one:

```
echo $?
```

Conventions:

- Many, not all, UNIX commands return **0** on success and **1** on failure. Some use a distinct code per error. Bash has a compact convention (www.tldp.org/LDP/abs/html/exitcodes.html).
- Codes in the 128–192 range decode as 128 + n, where n is the kill-signal number.** This rule is the core forensic tool of the chapter.

Worked example from the VM:

```
~/src/examples/killer-whiles/mystery000    # prints: Floating point exception (core dumped)
echo $?                                     # prints: 136
```

136 = 128 + 8 = **SIGFPE**, sent when a program attempts an erroneous arithmetic operation.

List all signals with their numbers:

```
kill -L
```

It prints 1) SIGHUP 2) SIGINT 3) SIGQUIT 4) SIGILL 5) SIGTRAP 6) SIGABRT 7) SIGBUS 8) SIGFPE 9) SIGKILL 10) SIGUSR1 11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM ... up to the real-time signals SIGRTMIN–SIGRTMAX.

Limitation the book insists on: a program can return *any* exit code, sometimes by mistake. Nothing stops someone running **kill -8** and producing exactly the code an arithmetic error produces. **Exit codes are a convention, not proof.** Most popular tooling follows the convention. Your forensic conclusion is a hypothesis, not a verdict.

2.3.2 Killing processes

The two-terminal drill, exactly as the book runs it:

```
# Terminal 1 – a stand-in long-running process
sleep 3600

# Terminal 2 – see it, with parent/child relationships (the f flag)
ps f
# PID TTY STAT TIME COMMAND
# 4214 pts/1 Ss 0:00 bash
# 4262 pts/1 R+ 0:00 \_ ps f
# 2430 pts/0 Ss 0:00 bash
# 4261 pts/0 S+ 0:00 \_ sleep 3600

# Terminal 2 – kill it (default signal)
pkill sleep          # Terminal 1 prints: Terminated

# Terminal 1
echo $?              # 143 = 128 + 15 = SIGTERM  <-- the code from the FizzBuzz logs
```

Then with an explicit KILL:

```
sleep 3600          # terminal 1
pkill -9 sleep      # terminal 2
echo $?             # 137 = 128 + 9 = SIGKILL
```

The aha moment: 143 in the FizzBuzz logs = SIGTERM = something sent the default kill signal. Candidates: an administrator, the system, or a daemon that manages the process.

Pop quiz (§2.3.1): which statement is false? *"There are 32 possible exit codes."* — false. `kill -L` already shows 64 signals, and exit codes span 0–255.

2.3.3 The Out-Of-Memory (OOM) Killer

Theory. The OOM Killer is one of the more interesting and controversial memory-management features of the Linux kernel. Under low memory it starts, **scores every process with heuristics** (niceness, how recent the process is, how much memory it uses), and kills the winner to reclaim memory and regain stability. Further reading the book recommends: https://linux-mm.org/OOM_Killer and Goldwyn Rodrigues, "Taming the OOM Killer" (<https://lwn.net/Articles/317814/>).

Practice — reproduce and diagnose it:

```
~/src/examples/killer-whiles/mystery001      # allocates memory until the kernel intervenes

top -n1 -o+%MEM          # -n1: print once and exit; -o+%MEM: sort by memory use
```

Representative output (the VM has ~3.9 GiB):

```
MiB Mem : 3942.4 total, 98.9 free, 3745.5 used, 98.0 buff/cache
MiB Swap:  0.0 total, 0.0 free,  0.0 used.  5.3 avail Mem
PID  USER  PR NI  VIRT  RES  SHR S %CPU %MEM  TIME+ COMMAND
5451  chaos  20  0 3017292 2.9g   0 S  0.0 74.7  0:07.95 mystery001
```

`mystery001` holds 2.9 GB — roughly three-quarters of the VM. Free memory is near 100 MB and there is no swap. The shell then prints:

```
Killed
```

The forensic step — read the kernel log:

```
dmesg | grep -i mystery001
```

```
[14658.582932] Out of memory: Kill process 5451 (mystery001) score 758 or sacrifice child
[14658.582939] Killed process 5451 (mystery001) total-vm:3058268kB, anon-rss:3055776kB, file-
rss:4kB, shmem-rss:0kB
[14658.644154] oom_reaper: reaped process 5451 (mystery001), now anon-rss:0kB, file-rss:0kB,
shmem-rss:0kB
```

Further in `dmesg` you see the **task table** the killer evaluated, including the `oom_score_adj` column — the per-process bias applied to the score.

Tunables (from www.kernel.org/doc/Documentation/sysctl/vm.txt):

Flag	Meaning	Default in the book's VM
<code>oom_kill_allocating_task</code>	0 = scan the whole task list and pick a victim by heuristics (usually the rogue memory hog). Non-zero = kill the task that triggered the OOM, avoiding the expensive scan. <code>panic_on_oom</code> takes precedence over it.	<code>0</code>
<code>oom_dump_tasks</code>	Dump extra information when killing a process, for easier debugging.	<code>1</code>

```
cat /proc/sys/vm/oom_kill_allocating_task
cat /proc/sys/vm/oom_dump_tasks
```

Three ways a process dies:

Cause	How you detect it	Example code
The program did something illegal	Exit code 128+n for the faulting signal; message on the terminal	136 = SIGFPE
Something killed it explicitly	Exit code 128+n for the sent signal	143 = SIGTERM, 137 = SIGKILL
The OOM Killer chose it	Exit code looks like a kill; the truth is in <code>dmesg</code> (<code>oom_reaper</code> , "Out of memory: Kill process")	shows as <code>Killed</code>

The dead end is deliberate. Exit code 143 cannot separate "an admin killed it" from "the OOM Killer killed it." The book says so, then makes the key move: *chaos engineering still lets you make progress, because you form hypotheses about the system as a whole rather than about one process's cause of death.*

2.4 Practice — The first chaos experiment

The reframing (Figures 2.2 → 2.3). Figure 2.2 shows the internal architecture. Figure 2.3 draws a box around the whole thing and shows only: *client makes an HTTP request to the system → client gets an HTTP response from the system*. Everything inside is the black box.

Why this matters, in the book's words: fixing whatever kills the API servers "solves" the problem only until the next bug, outage or human error reintroduces it. **"In our system, or any bigger distributed system, components dying is a norm, not an exception."** The property worth testing is that *clients see no errors*, not that *processes never die*.

The system under test in the VM:

```
# Two API server instances, as systemd units (installed, disabled by default)
sudo systemctl status faas001_a
```

```

sudo systemctl status faas001_b
sudo systemctl start faas001_a
sudo systemctl start faas001_b      # only /api/v1/ is implemented; everything else 404s

# The load balancer config
cat ~/src/examples/killer-whiles/nginx.loadbalancer.conf | grep -v "#"

```

```

upstream backend {
    server 127.0.0.1:8001 max_fails=1 fail_timeout=1s;
    server 127.0.0.1:8002 max_fails=1 fail_timeout=1s;
}
server {
    listen 8003;
    location / {
        proxy_pass http://backend;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

These two NGINX parameters drive the whole experiment:

- `max_fails=1` — a single error response takes that instance out of the pool.
- `fail_timeout=1s` — the ejected server stays out for **1 second**, then returns gracefully.

```

sudo systemctl start nginx
curl 127.0.0.1:8003/api/v1/
# { "FizzBuzz": true }

```

The API servers are literally `python3 -m http.server 8001 --directory ../static` under systemd. That is why killing them is easy, and why systemd's restart policy becomes the star of the chapter.

Experiment Card 2.1 — Kill both instances once each

Purpose and concepts

Goal

Verify the user-visible property: killing API server instances one at a time produces no client-visible errors.

Relevant theory

Four-step model (Ch. 1); whole-system (black-box) thinking (Fig. 2.3); "components dying is the norm."

Setup, hypothesis and boundaries

System / setup

NGINX on :8003 round-robin to `127.0.0.1:8001` (faas001_a) and `:8002` (faas001_b), both systemd units with `Restart=always`.

Hypothesis

"If we kill both instances, one at a time, the users won't receive any error responses from the load balancer."

Steady state

`ab` reports **Failed requests: 0**.

Observability

Apache Bench (`ab`) generates load and counts failures. Single metric: **Failed requests**.

Blast radius

Everything on the VM matching `grep 8001 / grep 8002` — deliberately too wide; see §2.5.

Execution and tools

Failure injected

`kill` (SIGTERM) to instance A, wait 2 s, `kill` to instance B.

Tools / commands

```
ab -t 30 -c 10 -l http://127.0.0.1:8003/api/v1/ ; ~/src/examples/killer-whiles/cereal_killer.sh
```

Procedure

1. Start both services + nginx. 2. Establish steady state with `ab`. 3. In window 1 run `bash ~/src/examples/killer-whiles/run_ab.sh`. 4. In window 2 run `bash ~/src/examples/killer-whiles/cereal_killer.sh`. 5. Read `Failed requests`.

Results, interpretation and lessons

Expected

0 failed requests.

Observed

`Complete requests: 50000, Failed requests: 0`. Both instances killed and restarted (PIDs change; systemd reports `active`). Hypothesis **confirmed**.

Interpretation

NGINX's retry to the other backend plus systemd's restart covers a *single, spaced-out* instance death.

Lesson

A passing experiment proves resilience *to the exact conditions you chose*, nothing more. The author chose the 2-second `sleep` precisely to make the experiment pass.

Limits and follow-up

Production

`Failed requests` is the right kind of metric: user-visible, not implementation-visible.

The `ab` flags:

```
ab -t 30 -c 10 -l http://127.0.0.1:8003/api/v1/
# -t 30    run for up to 30 seconds
# -c 10    concurrency of 10
# -l      ignore content-length differences between responses
# (stops at 50,000 requests, whichever comes first)
```

The injection script, `cereal_killer.sh` (abridged in the book):

```
echo "Killing instance A (port 8001)"
ps auxf | grep 8001 | awk '{system("sudo kill " $2)}' # find PID in ps output, kill it
echo "Wait some time in-between killings"
sleep 2 # let NGINX re-add instance A
echo "Killing instance B (port 8002)"
ps auxf | grep 8002 | awk '{system("sudo kill " $2)}'
```

Read the awk: `$2` is the PID column of `ps auxf`, and `system()` shells out to `sudo kill <pid>` for **every matching line**.

2.5 Theory — Blast radius

Definition: like an explosive, a software component can go wrong and break other things it connects to. The **blast radius** is *the maximum number of things that can be affected by something going wrong* — here, by our experiment. Managing it means **limiting the number of things our experiments can affect**.

The chapter criticises its own script:

```
grep sudo ~/src/examples/killer-whiles/cereal_killer.sh
# ps auxf | grep 8001 | awk '{system("sudo kill " $2)}'
# ps auxf | grep 8002 | awk '{system("sudo kill " $2)}'
```

Any process whose **ps** line merely *contains* the string **8001** dies — including a process that happens to **have PID 8001**. "Innocent and without trial."

Figure 2.5 — three nested blast radii for the same intent:

Selector	Blast radius
grep 8001	Anything mentioning 8001 anywhere in its ps line, including unrelated PIDs. Widest.
`grep python`	grep 8001`
grep "python3 -m http.server 8001"	Only the exact target command line. Narrowest.

Other fixes for this case: fetch PIDs from systemd, or use **systemctl restart** directly.

Two categories of blast-radius control:

- **Implementational** — make the execution safer: narrow the selector, target by PID from a trusted source, add guards. "As with any code, you are bound to make mistakes."
- **Strategic** — plan experiments so a failure cannot be catastrophic:
 - Roll out on a **small subset of traffic** first, expand later.
 - Run in a **QA environment** before production.
 - **Automate early**, so findings are reproducible.
 - **Be careful with randomness**. It finds race conditions but makes results hard to reproduce.

2.6 Practice — Digging deeper: the real bug

The first experiment passed, but clients still see errors. The author admits he chose the 2-second sleep to make it pass — "to show you how a seemingly successful experiment might prove insufficient."

New hypothesis, with concrete numbers: "If we kill instance A six times in a row, spaced out by 1.25 seconds, and then do the same to instance B, we continue seeing no errors."

The script, **killer_while.sh**. Note the *narrower* selector — an implementational blast-radius fix applied in place:

```
i="0"
while [ $i -le 5 ]
do
    echo "Killing faas001_a ${i}th time"
    ps auxf | grep killer-whiles | grep python | grep 8001 | awk '{system("sudo kill " $2)}'
    sleep 1.25
    i=$((i+1))
done
systemctl status faas001_a --no-pager
```

(**--no-pager** stops systemd piping output into **less**, which would block a script.)

Result:

```
Active: failed (Result: start-limit-hit) since ...
systemd[1]: faas001_a.service: Service RestartSec=100ms expired, scheduling restart.
systemd[1]: faas001_a.service: Scheduled restart job, restart counter is at 6.
systemd[1]: faas001_a.service: Start request repeated too quickly.
systemd[1]: faas001_a.service: Failed with result 'start-limit-hit'.
```

Both instances end up dead. Errors reach the client.

Root cause. The unit file looks sufficient:

```
[Unit]
Description=FizzBuzz as a Service API prototype – instance A

[Service]
ExecStart=python3 -m http.server 8001 --directory /home/chaos/src/examples/killer-whiles/static
Restart=always
```

Restart=always does *not* mean always. From systemd's documentation:

```
DefaultStartLimitIntervalSec= defaults to 10s
DefaultStartLimitBurst=       defaults to 5
```

So systemd allows **only five restarts inside a 10-second moving window**. Exceed that and systemd gives up on the unit permanently (**start-limit-hit**). Six kills at 1.25 s apart is 6 restarts in ~7.5 s — one over the limit. That is why the author picked those weirdly specific numbers.

The fix:

```
cat >> ~/src/examples/killer-whiles/faas001_a.service <<EOF
[Unit]
StartLimitIntervalSec=0
EOF

sudo systemctl daemon-reload
sudo systemctl start faas001_a
sudo systemctl start faas001_b
```

StartLimitIntervalSec=0 disables the rate-limiting window, so restarts are unlimited.

Verification. Re-run **killer_while.sh**. Instance A now reports **active (running)** after six kills. Instance B still reports **failed (start-limit-hit)** because it was not patched. That asymmetry is the *control group*: one run shows fixed and unfixed behaviour side by side. Patch B the same way and the errors disappear.

The double-edged lesson: fixing **StartLimitIntervalSec** makes the symptom disappear, which means "the API itself might keep crashing, and our friends from Glanden might never fix it, because their clients are no longer complaining." Resilience can hide the defect. Keep the crash *visible in metrics* after it stops being visible to users.

Experiment Card 2.2 — Repeated kills (six in a row, 1.25 s apart)

Purpose and concepts**Goal**

Test whether the system survives *repeated, rapid* instance death, not just isolated death.

Relevant theory

Steady state and hypothesis with concrete numbers; blast radius (narrowed selector); emergent behaviour from NGINX ejection + systemd restart limits.

Setup, hypothesis and boundaries

System / setup

Same as Card 2.1.

Hypothesis

"If we kill instance A six times in a row, spaced by 1.25 s, then do the same to B, we continue seeing no errors."

Steady state

Failed requests: 0.

Observability

ab failed-request count; systemctl status; systemd journal lines.

Blast radius

Narrowed to grep killer-whiles | grep python | grep <port>.

Execution and tools

Failure injected

6 × SIGTERM per instance at 1.25 s intervals.

Tools / commands

killer_while.sh; systemctl status --no-pager; systemctl daemon-reload.

Results, interpretation and lessons

Observed result

Hypothesis **refuted**. Both units enter failed (start-limit-hit); clients see errors.

Why

systemd's DefaultStartLimitIntervalSec=10s / DefaultStartLimitBurst=5: more than 5 starts in 10 s stops the unit permanently. Restart=always is bounded by that rate limit.

Lesson

Read the *defaults* of your supervisor, not just the directive you wrote. A restart policy has an implicit give-up threshold, and a crash loop is exactly the condition that trips it.

Fix

StartLimitIntervalSec=0 in the [Unit] section, then daemon-reload.

Limits and follow-up

Production considerations

Unlimited restarts mask a crashing application. Pair the fix with alerting on restart counts and keep the crash on a dashboard. The Kubernetes equivalent trap is CrashLoopBackOff exponential back-off (Ch. 10–12).

Theory ↔ Practice connections for Chapter 2

- **Four-step model (Ch. 1) ↔ Cards 2.1 and 2.2:** the same template, filled in twice with different numbers. Only the timing parameter separated a pass from a fail. That is the strongest argument for stating hypotheses with numbers.

- **Blast radius (§2.5) ↔ every later chapter:** Pumba's container-selection filters (Ch. 5), `strace -p <pid>` targeting one process (Ch. 6), seccomp per container (Ch. 5), PowerfulSeal's pod selectors and `--dry-run` (Ch. 11), and cloud experiments scoped to one availability zone (Ch. 11).
- **Linux forensics (§2.3) ↔ Ch. 3 and Ch. 5:** OOM scoring returns when cgroups limit container memory (Ch. 5, Experiment 4) and when you read RAM metrics with the USE method (Ch. 3).
- **Black-box reasoning (§2.3, Fig. 2.3) ↔ the book's method:** the chapter found and fixed the weakness without reading the API server's source.

Key Takeaways — Chapter 2

1. A process dies three ways that matter: it faulted, something signalled it, or the OOM Killer chose it. Exit code `128 + n` decodes the signal; `dmesg` reveals the OOM Killer.
 2. `echo $?, kill -L, ps f, pkill [-N], top -n1 -o+%MEM` and `dmesg | grep` are the minimum forensic kit.
 3. Exit codes are conventions, not evidence. 143 could be an admin, a supervisor, or the OOM Killer.
 4. When the cause of death is ambiguous, stop asking "why did this process die" and start asking "does the system still serve users when it does."
 5. **Blast radius = the maximum number of things your experiment can affect.** Control it implementationally (precise targeting) and strategically (small subsets, QA first, automate, care with randomness).
 6. A passing experiment proves resilience only to the conditions you chose. Vary the parameters — especially timing and repetition — before believing it.
 7. `Restart=always` in systemd is bounded by `DefaultStartLimitBurst=5` per `DefaultStartLimitIntervalSec=10s`. Set `StartLimitIntervalSec=0` for genuinely unlimited restarts.
 8. Useful chaos experiments can be a handful of bash commands. Tooling sophistication is not the point.
 9. Making a system resilient can hide the defect that made resilience necessary. Keep the underlying failure observable.
-

Chapter 3 — Observability

Study route

Start with USE: §§3.1–3.2 → resource-by-resource mechanisms and commands: §3.3 → application profiling: §3.4 → time-series monitoring: §3.5 → connections and takeaways.

Supplementary explanation — Editorial verification: Load averages and CPU controls

Correction — High confidence. Exponential averaging describes time weighting; it does not make the displayed load-average scale logarithmic. Linux load averages count runnable and uninterruptible tasks, so CPU count and I/O context matter. Sources: `proc_loadavg`, Brendan Gregg's explanation.

Correction — High confidence. CPU shares are relative weights under contention, not an absolute allocation or a reserved core. CPU quota/period controls impose a bandwidth ceiling; CPU placement is a different control. This also qualifies Chapter 5 and review answer 15. Sources: Linux CFS group scheduling, Docker CPU resource controls.

The book's framing: "Observability is the cornerstone of chaos engineering — it makes the difference between doing science and guessing."

3.1 Theory — Why "my app is slow" is the hard case

"My app doesn't work" is binary and usually easy to localise. **"My app is slow" is subtler.** The software passed its tests, several people signed off, and now it degrades for no obvious reason. Enough slowness makes the system "as good as down." The book cites the pattern of products that get positive media attention, fall over under the traffic spike, and earn negative coverage for unreliability.

Where this sits relative to chaos engineering. A thin line separates chaos engineering, SRE and systems performance engineering. Ideally a chaos engineer only does prevention. In practice you debug first, then design an experiment so the issue cannot recur. Either way you need **fast, reliable insight into performance metrics**, because during an incident everyone panics and you must think quickly.

The narrative: Alice, head of engineering at FaaS, sits in a New York cab in the rain and gets the call from the Big Client — "The app is slow."

3.2 Theory — The USE method

USE = Utilization, Saturation, Errors (Brendan Gregg, www.brendangregg.com/usemethod.html). **For each resource, check errors, then utilization, then saturation.**

The book's definitions:

- A **resource** is any physical component of a physical server — CPU, disk, networking devices, RAM. **Software resources** also count: threads, PIDs, inode IDs.
- **Utilization** = average time or proportion of the resource used. For a CPU, the percentage of time spent doing work. For a disk, both the percentage full *and* the throughput matter.
- **Saturation** = the amount of work the resource **can't service at any given moment**, often queued.

Crucial nuance: **high saturation is not automatically bad**. In a batch-processing system you *want* to run as close to 100% of available processing power as possible.

The flowchart (Figure 3.1):

```

Start → Identify resources (CPU, RAM, block I/O, networking, filesystem, software resources...)
      → Pick resource
          → Errors?          yes → Investigate
          → High saturation? yes → Investigate
          → High utilization? yes → Investigate
          → More resources left? yes → pick next; no → Done
Investigate → Identified the problem? yes → Fix it; no → back to the loop
  
```

Three caveats stated with the flowchart:

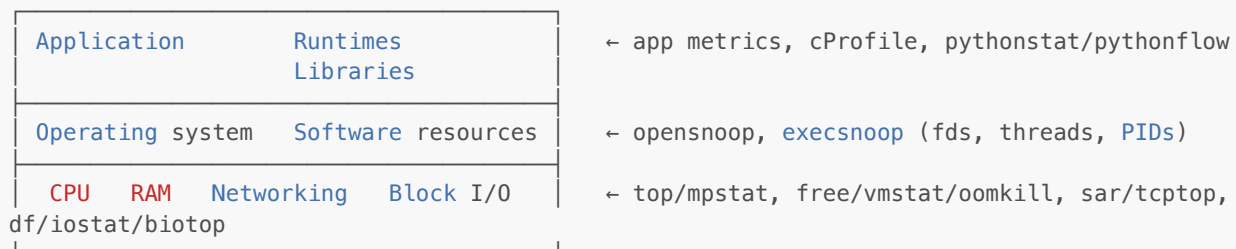
- Errors "can point to your issue **or distract you from the real one.**"
- At any step you often find a problem but not *the* problem. Add it to the to-do list and continue.
- Finding nothing still has value: "at least you've reduced the number of unknown unknowns."

Known unknowns, unknown unknowns, and "dark debt." Known unknowns are things you know you don't know — is there bacon in the fridge? Bacon is at least on your radar. **Unknown unknowns** are not on your radar at all. Every sufficiently complex system has some, and by the time you realise you needed to know, it is usually too late. After an incident you invent the monitoring that would have caught it; that is an unknown *becoming* a known unknown. Unknown unknowns are also called **dark debt**.

Pop quiz answer: USE = "a method of debugging a performance issue, based around measuring utilization, saturation, and errors."

3.3 Theory — The resource map (Figure 3.2)

The mental model the chapter navigates:



Four physical components sit at the bottom. The OS layer sits above and *provides* software resources such as file descriptors and threads. The application layer sits on top, with its libraries and runtimes.

The simulation. Start Alice's app in the VM:

```
~/src/examples/busy-neighbours/mystery002
```

It prints the time to compute 3000 digits of pi, in a loop:

```
Calculating pi's 3000 digits...
3.141592653589793238462643383279502884197169399375105820974944592307\
real  0m4.183s
user  0m4.124s
sys   0m0.022s
```

After the first few iterations the calculations take **much longer and vary more**. That is the "my app is slow" signal. The script uses two cores and heats your machine, so stop it between tools.

3.3.1 System overview — `uptime` and `dmesg`

`uptime` is usually the first command you run. It gives uptime (did the host restart recently?) and **load averages**.

```
uptime
# 05:27:47 up 18 min,  1 user,  load average: 2.45, 1.00, 0.43
```

- The three numbers are moving-window sum averages of processes competing for CPU time over **1, 5 and 15 minutes**.
- They are **exponentially scaled**. Twice the number does not mean twice the load.
- Here 2.45 / 1.00 / 0.43 means load is **rising**.

Don't worry about the absolute values — read the direction. Sharply decreasing numbers can mean you are too late and the resource hog has already gone. Increasing numbers are a good proxy for rising load.

Read load averages programmatically:

```
cat /proc/loadavg
# 0.12 0.91 0.56 1/416 5313
```

Fields: the three averages; `runnable/total` kernel-schedulable entities (processes and threads); the **PID of the most recently started program**. See `man proc`, search `loadavg`.

`dmesg` reads the kernel's message buffer — kernel and driver logs.

```
dmesg | less          # then type /Kill and Enter to search for OOM kills
dmesg --human         # relative, human-readable timestamps
```

Look for errors and anomalies. The OOM Killer lines from Chapter 2 appear here. Practical filter: kernel messages are verbose, so **most of the time you can ignore anything that does not mention `error`**.

3.3.2 Block I/O — `df`, `iostat`, `biotop`

Storage has **two independent kinds of utilization**, and you must check both: **capacity** (how full it is — when full, nothing can be written) and **throughput** (how much it can write per unit of time).

`df -h` shows filesystem capacity utilization. Here `-h` means *human readable*, not help.

```
df -h
# Filesystem  Size  Used Avail Use% Mounted on
```

```
# /dev/sda1    40G   13G   27G   33% /
```

iostat -x shows extended block-device statistics: throughput utilization and saturation.

```
iostat -x
```

Column	Meaning	USE category
r/s, w/s	Reads / writes per second (raw counts)	Utilization (partial)
rkB/s, wkB/s	Read / write kilobytes per second — total throughput	Utilization
aqu-sz	Average queue length of requests issued to the device	Saturation
%util	Percentage of time the device spent doing work	Utilization

r/s plus **rkB/s** also give you the **average size** of a read or write.

Two interpretation traps for %util:

1. A **logical device** such as a RAID can show high utilization while the underlying disks are underused.
2. **High saturation does not automatically mean an application bottleneck.** Many techniques let an application do useful work while it waits on I/O.

In the book's run, **sda** shows ~744 MB/s writes at 46% **%util** — busy, but inside spec. Nothing suspicious.

biotop is "block I/O top", part of **BCC**. It shows *which processes* drive disk load.

```
sudo biotop-bpfcc          # -C prevents clearing the screen each refresh
# PID  COMM                D MAJ MIN DISK  I/O  Kbytes  AVGms
# 5137  kworker/u4:3            W  8   0   sda   677  611272  3.37
# 246   jbd2/sda1-8          W  8   0   sda    2    204    0.20
```

sudo is required because BPF needs administrator privileges. On Ubuntu the BCC tools carry the **-bpfcc** suffix. They are written in Python, so you can read any of them:

```
less $(which biotop-bpfcc)
```

BPF / eBPF and BCC. The **Berkeley Packet Filter** is a kernel feature that lets a programmer **execute code inside the kernel** with guaranteed safety and performance. The **BCC** project (github.com/iovisor/bcc) adds wrappers and abstraction layers, and ships example utilities — **biotop**, **tcptop**, **oomkill**, **opensnoop**, **execsnoop** and more — that are useful on their own and written as starting points for your own programs. The **e** in eBPF means "extended"; "BPF" now usually means eBPF, and "classic BPF" means the older one. Recommended reading: Brendan Gregg, *BPF Performance Tools*. Gregg also maintains the Linux-tooling diagrams at www.brendangregg.com/linuxperf.html; the book suggests pinning them to your cubicle wall.

3.3.3 Networking — **sar** and **tcptop**

sar collects, reports and saves system metrics. Enable it first on the VM:

```
# edit /etc/default/sysstat : ENABLED="false" -> ENABLED="true"
sudo service sysstat restart
```

Interval and count. **sar**, and many BCC tools, take two optional positional parameters: **[interval]** **[count]** — how often to print, and how many times before exiting. The default is usually 1 second and infinite count. The book uses **1 1** to print one set of stats and exit.

Interface utilization:

```
sar -n DEV 1 1
# IFACE rxpck/s txpck/s rxkB/s txkB/s rxcmp/s txcmp/s rxmcast/s %ifutil
# eth0 1823.00 592.00 1616.29 34.69 0.00 0.00 0.00 1.32
```

%ifutil is the utilization. From `man sar`: `rxpck/s` and `txpck/s` = packets received and transmitted per second; `rxkB/s` and `txkB/s` = kilobytes received and transmitted per second; `rxcmp/s` and `txcmp/s` = compressed packets received and transmitted; `rxmcast/s` = multicast packets received.

To generate traffic the book downloads an Ubuntu ISO from a deliberately slow mirror:

```
wget http://mirrors.us.kernel.org/ubuntu-releases/19.10/ubuntu-19.10-desktop-amd64.iso
```

Interface errors:

```
sar -n EDEV 1 1
```

Field	Meaning
<code>rxerr/s</code>	Bad packets received per second
<code>txerr/s</code>	Errors per second while transmitting
<code>coll/s</code>	Collisions per second while transmitting
<code>rxdrop/s</code>	Received packets dropped per second because of a lack of space in Linux buffers
<code>txdrop/s</code>	Transmitted packets dropped per second, same reason
<code>txcarr/s</code>	Carrier errors per second while transmitting
<code>rxfram/s</code>	Frame-alignment errors per second on received packets
<code>rxfifo/s</code> / <code>txfifo/s</code>	FIFO overrun errors per second, received / transmitted

TCP statistics and TCP errors:

```
sar -n TCP,ETCP 1 1
```

Field	Meaning (SNMP counter)
<code>active/s</code>	Direct transitions CLOSED → SYN-SENT per second [<code>tcpActiveOpens</code>]
<code>passive/s</code>	Direct transitions LISTEN → SYN-RCVD per second [<code>tcpPassiveOpens</code>]
<code>iseg/s</code>	Total segments received per second, including in error [<code>tcpInSegs</code>]
<code>oseg/s</code>	Total segments sent per second, excluding retransmit-only [<code>tcpOutSegs</code>]
<code>atmptf/s</code>	Transitions to CLOSED from SYN-SENT or SYN-RCVD, plus SYN-RCVD → LISTEN [<code>tcpAttemptFails</code>]
<code>estres/s</code>	Transitions to CLOSED from ESTABLISHED or CLOSE-WAIT [<code>tcpEstabResets</code>]
<code>retrans/s</code>	Segments retransmitted per second [<code>tcpRetransSegs</code>]
<code>isegerr/s</code>	Segments received in error, for example bad TCP checksums [<code>tcpInErrs</code>]
<code>orsts/s</code>	TCP segments sent per second containing the RST flag [<code>tcpOutRsts</code>]

For network chaos experiments, `retrans/s`, `atmptf/s`, `estres/s` and `orsts/s` turn "the app feels slow" into "the network layer is failing." Watch these four in Chapter 4's WordPress experiments and Chapter 10's Kubernetes network experiments.

tcptop is the BCC tool that shows the top processes using TCP, sorted by bandwidth. The default is 20.

```
sudo tcptop-bpfcc 1 1
# PID  COMM  LADDR          RADDR          RX_KB  TX_KB
# 8142  wget   10.0.2.15:60080 149.20.37.36:80 2203   0
```

RX_KB = received, **TX_KB** = transmitted. This finds *who* eats the bandwidth, not just *that* bandwidth is eaten.

3.3.4 RAM — **free**, **top**, **vmstat**, **oomkill**

free -h is the **df** of RAM.

```
free -h
#          total  used  free  shared  buff/cache  available
# Mem:    3.8Gi  1.1Gi 121Mi 107Mi   2.7Gi       2.4Gi
# Swap:   750Mi  3.0Mi 747Mi
```

The classic confusion, and the answer. 3.8 GB total, 1.1 GB used, but only 121 MB "free"? The Linux kernel uses spare memory for **disk caches** and returns it the moment anyone asks. That memory is **not free but it is available**. There is a website for this reaction: www.linuxatemyram.com. The book's analogy: your younger brother borrows your car while you are not using it, except Linux always returns it unscathed and instantly.

How to tell you have actually run out of RAM: the **available** column near zero, plus the **OOM Killer going wild** in **dmesg**. Older **free** versions lacked **available** and showed a **-/+ buffers/cache** row instead: used minus buffers/cache, free plus buffers/cache.

top gives a memory and CPU overview. Most people under-use it. Press **?** to reveal the interactive commands:

Key	Effect
e / E	Toggle memory units (KB → MB → GB...) in the task list / in the summary
m / t	Toggle memory / CPU summary into progress bars
0	Hide zeros (remove clutter)
f	Field-management dialog: choose, reorder and pick the sort column
< / >	Move the sort column left/right; x bolds the sorted column so you can see which it is
L	Locate (search) a process name
V	Forest view — parent/child relationships, like ps f
1	Split CPU stats per core
k / r	Kill / renice a task from inside top
d or s	Set update interval
W	Write the config file — saves your interactive settings for next time
q	Quit

The field dialog also exposes columns worth knowing for chaos work: **OOMa** (OOM adjustment) and **OOMs** (current OOM score); **nsIPC/nsMNT/nsNET/nsPID/nsUSER/nsUTS** (**namespace inodes**, important in Chapter 5); **CGROUPS / CGNAME** (**control groups**, also Chapter 5); **LXC** (container name); **nMaj/nMin** (major/minor page faults); **SWAP**; **nTH** (thread count); **P** (last used CPU).

On a macOS host the built-in **top** is disappointing. Use **htop** or **glances** from Homebrew or MacPorts.

vmstat covers far more than virtual memory:

```
vmstat
# procs  ---memory--- --swap-- --io--  -system-  ----cpu----
```

```
# r b swpd free buff cache si so bi bo in cs us sy id wa st
# 5 0 0 1242808 47304 1643184 0 0 1866 53616 564 928 17 13 69 1 0
```

Key columns: **r** = **runnable processes** (running or waiting to run), an indicator of **saturation**; **b** = processes in **uninterruptible sleep**; **swpd** = used swap; **in** = interrupts; **cs** = context switches. Everything fits on one row, which makes **vmstat** practical as a repeating monitor.

Other **vmstat** modes:

```
vmstat -s # readable system usage stats, incl. "forks" = total processes run since boot
vmstat -f # just the fork count
vmstat -d # per-disk utilization/saturation stats
vmstat -D # one-off disk summary
```

oomkill is the BCC tool that traces kernel calls to **oom_kill_process** and prints every OOM kill as it happens. The book calls it "the equivalent of plugging directly into the Matrix", compared with grepping **dmesg** afterwards.

```
# terminal 1
sudo oomkill-bpfcc
# terminal 2
top -d 0.5 # press m a couple of times for memory progress bars
# terminal 3 - eat all the RAM
perl -e 'while (1) { $a .= "A" x 1024; }'
```

```
06:49:11 Triggered by PID 3968 ("perl"), OOM kill of PID 3968 ("perl"), 1009258 pages, loadavg:
0.00 0.23 1.22 3/424 3987
```

Chapter 2's **mystery001** produces the same trace more slowly, so you can watch memory creep up in **top**.

3.3.5 CPU — **top** and **mpstat**

```
cat /proc/cpuinfo # model name, cpu MHz, stepping, per-processor detail
```

The **%Cpu(s)** row in **top**. Memorise all eight numbers:

Field	Meaning
us (user time)	% of time the CPU spent in user space
sy (system time)	% of time in kernel space
ni (nice time)	% of time on low-priority processes
id (idle)	% of time doing literally nothing (it can't stop)
wa (I/O wait)	% of time waiting on I/O
hi (hardware interrupts)	% of time servicing hardware interrupts
si (software interrupts)	% of time servicing software interrupts
st (steal time)	% of time a hypervisor stole the CPU for someone else — only in virtualized environments

Niceness. A numeric value that shows how willing a process is to give CPU cycles to higher-priority neighbours. Range **-20 to 19**. Higher = nicer = lower priority. See **man nice** and **man renice**. This is the **ni** column per process.

In the sample: ~72% user, 25% system, ~3% software interrupts, **0% idle**. The CPU is fully consumed.

mpstat -P ALL 1 gives the same statistics **split per CPU**:

```
mpstat -P ALL 1
# CPU  %usr %nice %sys %iowait %irq %soft %steal %guest %gnice %idle
# all 60.10 0.00 33.33 0.00 0.00 6.57 0.00 0.00 0.00 0.00
# 0 41.41 0.00 45.45 0.00 0.00 13.13 0.00 0.00 0.00 0.00
# 1 78.79 0.00 21.21 0.00 0.00 0.00 0.00 0.00 0.00 0.00
```

Why the split matters: restart **mystery002** and for the first 20 seconds **bc** takes as much CPU as it wants, but it is **single-threaded and scheduled on only one CPU**. After 20 seconds **stress** spawns workers for both CPUs and both go busy. **top** does the same split with the **1** key.

Solving the mystery. The "mysterious" script:

```
cat ~/src/examples/busy-neighbours/mystery002
```

```
#!/bin/bash
echo "Press [CTRL+C] to stop.."
export dir=$(dirname "$(readlink -f "$0")")
(bash $dir/benign.sh)&                                # "completely benign" background daemon
while :
do
    echo "Calculating pi's 3000 digits..."
    time echo "scale=3000; 4*a(1)" | bc -l | head -n1 # 4*arctan(1) = pi, to 3000 decimals
done
```

```
cat ~/src/examples/busy-neighbours/benign.sh
```

```
#!/bin/bash
sleep 20                                              # sneaky delay: the app looks fine at
first
while :
do
    stress --cpu 2 -m 1 -d 1 --timeout 30 2>&1 > /dev/null
    sleep 5
done
```

stress --cpu 2 -m 1 -d 1 spawns 2 CPU-spinning workers, 1 memory worker and 1 disk worker for 30 seconds at a time. **That is the busy neighbour**. In **top**, the **stress** processes together out-compete **bc** for CPU.

The 20-second sleep is the most instructive line in the script. It explains why the app "was fine" when first observed and slow later. It is also why a steady state measured once, briefly, at the start is untrustworthy. Measure the steady state long enough to include the system's periodic behaviour.

Experiment Card 3.1 — The busy-neighbour experiment

Purpose and concepts

Goal

Explain why the pi-calculating application became slow, and confirm the cause is CPU contention.

Relevant theory

USE method; resource contention; steady state measured over time; the four-step model.

Setup, hypothesis and boundaries

System / setup

Two-core VM. Foreground workload: `bc` computing 3000 digits of pi in a loop. Background: `stress --cpu 2 -m 1 -d 1` cycling every 35 s after a 20 s delay.

Observability (step 1)

Wall-clock time per pi iteration (from `time`). Supporting: `top`, `mpstat -P ALL 1`, `uptime`.

Steady state (step 2)

"Around 5 seconds per iteration" (the first runs show ~4.18 s).

Hypothesis (step 3)

"When other processes are running, the speed should remain the same."

Blast radius

The whole VM, deliberately. Every process on the box contends.

Execution and tools

Failure injected

Resource starvation — a competing CPU/memory/disk load on the same host.

Tools / commands

```
~/src/examples/busy-neighbours/mystery002; top; mpstat -P ALL 1; cat /proc/cpuinfo; stress.
```

Results, interpretation and lessons

Observed result (step 4)

Hypothesis refuted. Iterations take much longer and vary more. `top` shows four `stress` processes taking 52.9 / 23.5 / 23.5 / 17.6 %CPU while `bc` gets only 17.6 %CPU. `%Cpu(s)` shows 0% idle.

Why it happened

`bc` is single-threaded and competes on equal terms with `stress` workers for two cores. The kernel's default scheduler shares CPU fairly among runnable processes; "fair" here means the application loses.

Lesson learned

Slowness is often *not* a bug in your code. It is a neighbour. Find it with per-process tooling (`top`, `mpstat`, `biotop`, `tcptop`) before you touch the application.

Fix applied

See below — `cgroups`, not niceness.

Limits and follow-up

Production considerations

On shared hosts, and on every container platform, this is the default condition rather than an exception. The fix belongs in the platform layer (cgroup limits, Kubernetes requests and limits), not in the application.

Why `nice` is the wrong fix and `cgroups` are the right one. Niceness sets a *relative* priority. Its drawback, in the book's words: "it's hard to control precisely how much CPU they would get." **Control groups (`cgroups`)** are a kernel feature that specifies **exact amounts** of resources — CPU, memory, I/O — that the kernel allocates to a group of processes.

```
sudo cgcreate -g cpu:/formulaone
sudo cgcreate -g cpu:/formulatwo
```

The fixed script, `mystery002-cgroups.sh`:

```
sudo cgcreate -g cpu:/formulaone
sudo cgcreate -g cpu:/formulatwo

export dir=$(dirname "$(readlink -f "$0")")
(sudo cgexec -g cpu:/formulatwo bash $dir/benign.sh)&      # noisy neighbour in its own box

while :
do
    echo "Calculating pi's 3000 digits..."
    sudo cgexec -g cpu:/formulaone bash -c 'time echo "scale=3000; 4*a(1)" | bc -l | head -n1'
done
```

`cgcreate -g cpu:/<name>` creates a CPU-controlled cgroup. `cgexec -g cpu:/<name> <cmd>` runs a command inside it. **By default each control group gets 1024 shares, effectively one core.** Result in `top`: `bc` gets ~80% of a CPU while all four `stress` processes share the other at ~26.7% each.

The book's image: cgroups are "Tupperware... (oh my, was I just about to say containers?)". That aside is the bridge to Chapter 5 — containers **are** cgroups plus namespaces plus chroot.

3.3.6 OS layer — `opensnoop` and `execsnoop`

`opensnoop` shows every file opened by every process, in near real time:

```
sudo opensnoop-bpfcc
# in another terminal: top -n1
# 12396 top 6 0 /proc/sys/kernel/osrelease
# 12396 top 6 0 /proc/meminfo
# 12396 top 7 0 /sys/devices/system/cpu/online
# 12396 top 8 0 /proc/12386/stat
# 12396 top 7 0 /proc/loadavg
```

This shows *where a tool gets its data* — here, `/proc`. The chaos-engineering use: "you will often want to know what a particular application **you didn't write** is actually doing, in order to know how to design or implement your experiments."

`execsnoop` shows every process started on the machine. It traces the `exec` family:

```
sudo execsnoop-bpfcc
# PCOMM  PID    PPID  RET  ARGS
# ls      12419  2073   0    /usr/bin/ls --color=auto
```

Run against `mystery002`, it exposes the whole hidden process tree: `readlink`, `dirname`, `bash` `benign.sh`, `bc`, `head`, `sleep 20`, and finally `stress --cpu 2 -m 1 -d 1 --timeout 30`. **This alone would have solved the mystery.**

The book deliberately omits `strace`, `dtrace` and `perf` here — "I've opted to give you a taste of what BPF has to offer, because I believe that it will slowly replace the older technologies for this use case." `strace` gets full treatment in Chapter 6, including *why* its overhead disqualifies it for some uses.

3.4 Practice — Application layer: profiling Python

Every application differs, so you handle high-level application metrics case by case: bank transaction latencies, number of concurrent players, hashes per second. But **runtimes and libraries are shared across applications**, so they are easier to inspect

generically.

3.4.1 cProfile

Python ships two profilers, **cProfile** and **profile**. Use **cProfile** — lower overhead, recommended for most use cases.

```
>>> import cProfile, re
>>> cProfile.run('re.compile("foo|bar")')
# 243 function calls (236 primitive calls) in 0.000 seconds
# ncalls tottime pcall cumtime pcall filename:lineno(function)
```

Columns that matter: **ncalls** (total calls; **a/b** means total/primitive, that is, non-recursive), **tottime** (time spent *in* that function, excluding subcalls), **cumtime** (cumulative — that call **and all its subcalls**).

Whole modules or scripts:

```
python -m cProfile [-o output_file] [-s sort_order] (-m module | myscript.py)

python3.7 -m cProfile -m http.server 8001      # then Ctrl-C to print stats
curl localhost:8001                            # generate some work first
```

The instructive line in the output:

```
36  17.682  0.491  17.682  0.491 {method 'poll' of 'select.poll' objects}
```

The program spent almost all its time **waiting to accept new requests**. That is correct behaviour for an idle server, and a reminder that "where time is spent" and "where the problem is" are different questions. The book points at **py-spy** for more ergonomic profiling.

3.4.2 BCC and Python — **pythonstat**, **pythonflow**

These need a Python binary compiled with **--with-dtrace**, which enables **USDT (User Statically Defined Tracing) probes** — points the software's authors defined in the code for DTrace to attach to. BPF/BCC can use those same probes. Many popular applications can be built with them: **MySQL**, **Python**, **Java**, **PostgreSQL**, **Node.js**. The VM ships a prebuilt **~/Python-3.7.0/python**.

```
# terminal 1 - some workload
~/Python-3.7.0/python -m freegames.life      # Conway's Game of Life

# terminal 2
sudo pythonstat-bpfcc
# PID  CMDLINE          METHOD/s  GC/s  OBJNEW/s  CLOAD/s  EXC/s  THR/s
# 7139 /home/chaos/Python-3  480906   3       0       0       0       0
```

pythonstat reports, per second: **method invocations**, **garbage collections**, **new objects**, **classes loaded**, **exceptions**, **new threads**.

```
sudo pythonflow-bpfcc $(pidof python)
```

pythonflow traces the **beginning and end of each function execution**. It prints an indented call tree with CPU, PID, TID and timestamp — for example every **importlib._bootstrap** step triggered by **import this**.

The book's generalisation: "each language ecosystem has its own equivalent tools and methods. Each stack will let you profile and trace applications." Chapter 7 does exactly this for the JVM.

3.5 Practice — Automation with time series: Prometheus + Grafana

The drawback of every tool above: you must sit down and run each command. Automation fixes that.

Commercial options named: Datadog, New Relic, Sysdig. The book uses open source:

- **Prometheus** — monitoring system and **time-series database**; gathers, stores, queries and alerts.
- **Grafana** — analytics and visualization over many data sources, including Prometheus.
- **Node Exporter** — a Prometheus subproject that **exposes a large set of system metrics**.

Step 1 — run Node Exporter. Docker is used here purely "as a program launcher":

```
docker run -d \
  --net="host" \
  --pid="host" \
  -v "":"/host:ro,rslave" \
  quay.io/prometheus/node-exporter \
  --path.rootfs=/host
```

Why those flags matter, and they preview Chapter 5: `--net=host` and `--pid=host` put the container in the **host's network and PID namespaces** so it sees real host metrics. `-v "":"/host:ro,rslave"` mounts the host filesystem read-only, so `--path.rootfs=/host` reads `/proc` and `/sys` from the host instead of the container.

```
curl http://localhost:9100/metrics
# promhttp_metric_handler_requests_total{code="200"} 0
# promhttp_metric_handler_requests_total{code="500"} 0
# promhttp_metric_handler_requests_total{code="503"} 0
```

The Prometheus data model, in one sentence: one line per metric, and the same metric name with different **label** values (`code="200"`, `"500"`, `"503"`) is **three separate time series**, each with some value at any point in time.

Step 2 — scrape config (`/home/chaos/prom.yml`):

```
global:
  scrape_interval: 5s           # 5s so metrics appear quickly
scrape_configs:
- job_name: 'node'
  static_configs:
  - targets: ['localhost:9100'] # the Node Exporter
```

Step 3 — run Prometheus:

```
docker run \
  -p 9090:9090 \
  --net="host" \
  -v /home/chaos/prom.yml:/etc/prometheus/prometheus.yml \
  prom/prometheus
```

Open `http://127.0.0.1:9090/`, query `node_cpu_seconds_total`, and look at the **mode label**: `idle`, `user`, `system`, `steal`, `nice`. These are the categories from `top`'s `%Cpu(s)` row, now as time series you can plot, aggregate and alert on.

How Prometheus works: it **scrapes**. It calls an HTTP endpoint, interprets the time-series data, and stores each value at the timestamp of the scrape.

Further pointers: PromQL documentation, and the Grafana dashboard library at grafana.com/grafana/dashboards. The book shows dashboard 11074.

3.6 Further reading (the book's own list)

- Brendan Gregg, *Systems Performance: Enterprise and the Cloud*
- Brendan Gregg, *BPF Performance Tools*
- Robert Love, *Linux Kernel Development*

Theory ↔ Practice connections for Chapter 3

- **USE method (§3.2) ↔ the tool tour (§3.3):** every tool answers "how do I read utilization, saturation or errors for *this* resource." Keep the mapping, not the tool names.
 - **Observability step of the four-step model (Ch. 1) ↔ Experiment Card 3.1:** the pi-iteration time *is* the observability metric. The chapter re-draws Figure 1.2 as Figure 3.7 with those values filled in.
 - **Steady state (Ch. 1) ↔ the 20-second sleep in benign.sh:** a steady state sampled too briefly is a lie. This is the concrete argument for measuring over a window.
 - **cgroups (§3.3.5) ↔ Chapter 5:** the mechanism that fixes the busy neighbour here is one of the three pillars of Docker containers.
 - **Prometheus (§3.5) ↔ Chapter 11:** continuous SLO verification needs continuous metrics. PowerfulSeal's SLO experiments assume this stack.
 - **BPF/BCC (§3.3.2, §3.3.6, §3.4.2) ↔ Chapter 6:** BPF returns as the low-overhead alternative to *strace* for syscall observation.
 - **free's available column ↔ Chapter 5, Experiment 4:** RAM accounting inside a cgroup-limited container behaves differently, and the OOM Killer arrives sooner.
-

Key Takeaways — Chapter 3

1. **USE = Utilization, Saturation, Errors, applied per resource.** Check errors first; they may be the cause or a distraction. High saturation is not automatically a problem.
 2. The resource map has four layers: physical (CPU, RAM, network, block I/O) → OS (software resources) → runtimes and libraries → application.
 3. Load averages show **direction**, not magnitude. Read the trend; ignore the absolute value.
 4. Storage has two utilizations: **capacity (df)** and **throughput (iostat)**. *aqu-sz* is its saturation. *%util* on logical devices lies.
 5. *free*'s *available* column, not *free*, tells you whether you are out of RAM. Linux caching is not memory loss.
 6. *top*'s *%Cpu(s)* row decomposes CPU time into *us/sy/ni/id/wa/hi/si/st.st* (steal) shows that a hypervisor is taking your cycles.
 7. *mpstat -P ALL 1*, or *1* in *top*, exposes per-core imbalance that aggregate numbers hide. This matters most for single-threaded workloads.
 8. BCC/eBPF tools (*biotop*, *tcptop*, *oomkill*, *opensnoop*, *execsnoop*, *pythonstat*, *pythonflow*) answer "**which process**" with negligible overhead. Aggregate metrics cannot answer that question.
 9. *nice* sets relative priority. **cgroups set absolute allocations.** Prefer cgroups when you need a guarantee.
 10. A resource-contention problem often has nothing to do with your code. Look for the neighbour before you refactor.
 11. Manual tools do not scale to continuous practice. Push USE metrics into a time-series database (Node Exporter → Prometheus → Grafana) so a steady state becomes a query instead of a memory.
-

Chapter 4 — Poking Docker

Study route

Orient to the scenario: §5.1 → virtualization and kernel mechanisms: §§5.2–5.9 → applied diagnosis and Pumba: §§5.10–5.11 → operational pitfalls: §5.12 → connections and takeaways.

Supplementary explanation — Editorial verification: CPU weights and the memory experiment

Study cross-reference. Apply the CPU-shares correction in Chapter 3 when reading the cgroup examples here.

Unresolved explanation — Unknown confidence in the original causal claim. The reported VIRT/RES values alone do not establish why the memory-stress command completed. The original claim that *stress* never touches its allocation has not been verified against the demonstrated binary and environment. Docker memory and swap configuration must also be considered; see

Docker memory resource controls. Preserve the reported observation, but do not memorize that proposed cause as established. This also applies to review answer 24.

The author calls this "one of my favorite chapters of the book." It goes from a vague idea of what Docker is, to reimplementing a container from scratch, then breaks both the application *and* Docker itself.

5.1 Practice — The scenario: Meower USA

Meower expanded to the US. The new team rejected WordPress/PHP and rebuilt on **Ghost** (ghost.org), a Node.js blogging engine, running on **Docker**. The symptom matches Chapter 4: customers occasionally complain about slowness while engineering sees nothing wrong.

Architecture (Figure 5.1):

- A **third-party load balancer**, outsourced, round-robin.
- Multiple **Ghost** instances, each in a Docker container.
- One **MySQL** container as the datastore. Ghost also supports SQLite3.

Definition (the book's working one): a **container** is "a construct designed to limit the resources that a particular program can access."

Same shape as Chapter 4, with one component simpler (the load balancer is someone else's problem) and **one new source of complexity: Docker**. To debug it you must know what Docker actually is.

5.2 Theory — Emulation, simulation, virtualization

Term	Definition (the book's, from Wikipedia)	Keyword	Example
Emulator	"Hardware or software that enables one computer system (the host) to behave like another computer system (the guest)"	reproduce the internals	PlayStation/Game Boy/DOS emulators; x86 Linux emulated in JavaScript in a browser (bellard.org/jslinux)
Simulation	"An approximate imitation of the operation of a process or system that represents its operation over time"	imitate the behaviour, to study and analyse	Flight simulator; physics simulation
Virtualization	"The act of creating a virtual (rather than actual) version of something, including virtual computer hardware platforms, storage devices, and computer network resources"	the umbrella term	Both of the above are means of achieving it

Why emulation is useful: testing software for a platform you do not own (rare, fragile or expensive); making products backward compatible by reusing firmware; running software from platforms no longer produced.

Two kinds of hardware (platform) virtualization matter here (Figure 5.2):

- **Full virtualization (virtual machines)** — a complete simulation of the underlying hardware. **Each VM has its own kernel.** Also called *strong isolation*.
- **OS-level virtualization (containers)** — the OS isolates system resources from the software's point of view, but **all containers share the same kernel.** Also called *lightweight isolation*.

VMs vs. containers — the comparison table

	Virtual machine	Container
Pros	Fully isolated — more secure than containers. Can run a different OS than the host. Better resource utilization (unused VM resources can go to another VM).	Lower overhead, better performance — the kernel is shared. Quicker startup. Portable packaging of software with its dependencies.
Cons	Higher overhead (operating systems stacked on operating systems). Longer startup (the OS must boot).	Bigger blast radius for security issues — shared kernel. Cannot run a different OS or even kernel

	Virtual machine	Container
	One app per VM typically wastes resources.	version. Often not all of the OS is virtualized → weird edge cases.
Typical use	Partition large physical machines into smaller chunks, with APIs to create/resize/delete. The managing software is a hypervisor .	Package and release software in a truly portable manner — "containers as a means of packaging software with extra benefits."
Providers named	KVM, Microsoft Hyper-V, QEMU, VirtualBox, VMware vSphere, Xen Project	Docker, LXC/LXD, Microsoft Windows containers

VMs and containers are not exclusive. Running containers inside VMs is common; Chapter 10 does it, and so does this chapter.

Security answer to memorise: VMs typically offer better security than containers, because containers share a kernel.

"VM, container, and everything in between" — the hybrid projects the book names:

- **Firecracker** (used by Amazon) — microVMs with fast startup *and* strong isolation.
- **Kata Containers** — hardware-virtualized Linux containers (VT-x, ARM HYP mode, IBM Power/Z).
- **UniK** — builds applications into **unikernels** for microVMs that boot quickly with low overhead on traditional hypervisors.
- **gVisor** (Google) — a **user-space kernel** implementing only a subset of the Linux system interface, to raise container security.

Also worth knowing: **hardware-assisted virtualization** is now expected — hardware designed for virtualization, so software runs at approximately host speed.

5.3 Theory — The road to Linux containers (Table 5.1)

Year	Isolation of	Event
1979	Filesystem	UNIX v7 includes the chroot system call — change the root directory of a process and its children. Often considered the first step toward containers.
2000	Files, processes, users, networking	FreeBSD 4.0 introduces the jail system call — mini-systems that prevent processes interacting with processes outside their jail.
2001	Filesystems, networking, memory	Linux VServer — a jail-like mechanism via kernel patches. Some syscalls and parts of /proc , /sys left unvirtualized.
2002	Namespaces	Linux kernel 2.4.19 introduces namespaces — control which set of resources is visible to each process. Initially mounts only; PID, network, cgroup, time added later.
2004	Sandbox	Solaris Containers / Zones — isolated environments for processes.
2006	CPU, memory, disk I/O, network	Google launches "process containers" to limit, account for and isolate resource usage of process groups. Renamed control groups (cgroups) , merged into kernel 2.6.24 in 2007 .
2008	Containers	LXC — the first container manager for Linux, built on cgroups + namespaces.
2013	Containers	Google shares Imctfy ("Let Me Contain That For You"); parts end up in libcontainer .
2013	Containers	First release of Docker , built on LXC; later libcontainer replaces LXC (cgroups + namespaces + capabilities). Containers explode in popularity.

The seven kernel features Docker uses (Figure 5.3)

Feature	What it does
chroot	Changes the root of the filesystem for a particular process
Namespaces	Isolate what a container can see — PIDs, mounts, networking, and more
cgroups	Control and limit access to resources such as CPU and RAM
Capabilities	Grant subsets of superuser privileges , for example killing other users' processes
Networking	Manages container networking through various tools

Feature	What it does
Filesystems	Uses Unionfs to create container filesystems efficiently (copy-on-write)
Security	seccomp , SELinux, AppArmor to further limit what a container can do

The mental model to carry for the rest of the book: *namespaces control what a process can SEE; cgroups control what a process can USE; chroot controls what filesystem it has; capabilities control what privileged actions it may take; seccomp controls which syscalls it may make.*

What Docker itself adds

Docker does not implement containers. The kernel does. Docker adds **convenience**:

- **Container runtime** — makes the syscalls that create, modify and delete containers, creates filesystems, implements networking.
- **dockerd** — daemon providing an API for the runtime.
- **docker** — the CLI client of that API.
- **Dockerfile** — format describing how to build a container.
- **Container image format** — archive of all files and metadata needed to start a container.
- **Docker Registry** — hosting for images. **Docker Hub** is the free public one.
- **A protocol** for exporting, importing (pull) and sharing (push) images.

The lifecycle (Figure 5.4): **docker build** reads a Dockerfile → produces an image → **docker push** uploads it to a registry → **docker pull** downloads it elsewhere → **docker run** starts a container from it.

Pop quiz answer: "Docker is built on top of existing Linux technologies to provide an accessible way of using containers, rendering them much more popular." Docker did **not** invent Linux containers.

5.4 Practice — Under the hood: chroot and filesystems

```
docker run \
  --name firstcontainer \      # name it
  -ti \                       # keep stdin open + allocate a pseudo-TTY (note: SINGLE hyphen)
  --rm \                      # delete the container when you exit
  alpine:3.11 \               # image:tag – Alpine, a minimal distro popular in containers (5.61
  /bin/sh                     # what to execute inside
```

The first demonstration — same path, different content:

```
# inside the container (terminal 1)
head -n1 /etc/issue          # Welcome to Alpine Linux 3.11
# on the host (terminal 2)
head -n1 /etc/issue          # Ubuntu 20.04.1 LTS \n \l
```

The container's filesystem is **chroot'ed**. **/** inside the container is a different location on the host.

Figure 5.5 explained. The host has **/bin/ls**, **/fake-root-dir/bin/ls**, **/fake-root-dir/my-app**, **/log/a-log-file**. A process chroot'ed to **/fake-root-dir** sees only **/bin/ls**, **/my-app**, **/some-other-dir**. The contents of **/fake-root-dir** become the root. Note that the host holds **two copies of the ls binary**.

Union filesystems, layers, overlay2. In a **union filesystem**, two or more host folders are presented transparently as a single merged folder — a **union mount**. Those folders, arranged in order, are **layers**. Upper layers can **hide** lower layers' files by providing another file at the same path.

When you specify a base image, Docker downloads all its layers, unions them, and starts the container **with a fresh layer on top**. Read-only layers are reused efficiently: one file on disk, read by all containers that use that layer. If a process modifies a file from a

lower layer, the file is **copied in its entirety into the current layer — copy-on-write (COW)**. **overlay2** is the modern driver that implements this.

Finding where the container actually lives:

```
docker inspect firstcontainer
```

The **GraphDriver** section:

Key	Meaning
LowerDir	The top layer of the image the container is based on (read-only)
UpperDir	The read-write layer of the container
MergedDir	The merged (union) view of the two
WorkDir	overlay2's internal working directory
Name	overlay2 — the storage driver

The inode proof — how thin the isolation really is:

```
# host
export CONTAINER_ROOT=$(docker inspect -f '{{.GraphDriver.Data.MergedDir}}' firstcontainer)
sudo ls -li $CONTAINER_ROOT/etc/issue      # 800436
/var/lib/docker/overlay2/dc2.../merged/etc/issue
# container
ls -li /etc/issue                          # 800436 /etc/issue
```

Same inode. The same actual file, shown at two different paths.

"This is telling of the container's experience in general — **the isolation is really thin**." That sentence is the thesis of the chapter, and the reason chaos experiments on containers find so much.

5.4.2 DIY container, part 1 — chroot + filesystem

new-filesystem.sh, annotated:

```
#!/bin/bash
export NEW_FILESYSTEM_ROOT=${1:-~/new_filesystem}
export TOOLS="bash ls pwd mkdir ps touch rm cat vim mount"      # binaries to copy in

echo "Step 1. Create a new folder for our new root"
mkdir $NEW_FILESYSTEM_ROOT

echo "Step 2. Copy some (very) minimal binaries"
for tool in $TOOLS; do
    cp -v --parents `which $tool` $NEW_FILESYSTEM_ROOT;      # --parents keeps relative
done

echo "Step 3. Copy over their libs"
echo -n > ~/.deps
for tool in $TOOLS; do
    ldd `which $tool` | egrep -o '(/usr)?/lib.*\.[0-9][0-9]?' >> ~/.deps    # ldd = shared
done
cp -v --parents `cat ~/.deps | sort | uniq | xargs` $NEW_FILESYSTEM_ROOT

echo "Step 4. Home, sweet home"
NEW_HOME=$NEW_FILESYSTEM_ROOT/home/chaos
```

```
mkdir -p $NEW_HOME && echo $NEW_HOME created!
cat <<EOF > $NEW_HOME/.bashrc
echo "Welcome to the kind-of-container!"
EOF
```

The one command that may be unfamiliar: **ldd** prints shared-object dependencies of a binary. Without copying those **.so** files, nothing inside the chroot can start.

```
bash ~/src/examples/poking-docker/new-filesystem.sh not-quite-docker
sudo chroot not-quite-docker          # sudo is required by chroot
```

Inside you can make files and folders, but **ps fails** because there is no **/proc**. Optionally:

```
mkdir not-quite-docker/proc
sudo mount -t proc /proc/ not-quite-docker/proc
```

Note what mounting **/proc** means for isolation: you just gave the "container" a view of every process on the host. **Isolation is a series of deliberate choices, not a single switch.**

Experiment Card 5.1 — Can one container prevent another from writing to disk?

Purpose and concepts

Goal

Test whether container filesystems, being chroot'ed locations on the host's filesystem, allow one container to starve another of disk space.

Relevant theory

chroot + union filesystems (§5.4.1); "the isolation is really thin"; cgroups do *not* cover storage by default.

Setup, hypothesis and boundaries

System / setup

Two containers built from **ubuntu:focal-20200423**: **control** (writes a 50 MB file, deletes it, repeats every 2 s) and **failure** (allocates 50 MB files forever).

Hypothesis

"If another **failure** container writes to disk until it can't, the **control** container won't be able to write to disk anymore."

Steady state

The **control** container prints "OK wrote the file" every two seconds.

Observability

The **control** container's own success/failure message; **df -h** on the host.

Blast radius

The entire host filesystem — that is precisely what is being tested.

Execution and tools

Failure injected

Disk exhaustion by a second container.

Results, interpretation and lessons

Observed result

Hypothesis confirmed. `failure` writes ~475 files then hits `fallocate: fallocate failed: No space left on device`. Within the same seconds, `control` also fails. `df -h` shows `/dev/sda1 32G 32G 0 100% /`. Stopping `failure` (with `--rm`) frees its storage and `control` resumes.

Why it happened

Containers' filesystems are layers on the **same host filesystem**. Nothing in the default Docker configuration limits per-container storage.

Lesson learned

Running programs in containers does not automatically prevent one process from stealing disk space from another.

Fix / caveat

Docker exposes `--storage-opt size=X`, but with the `overlay2` driver it requires an `xfs` filesystem mounted with the `pquota` option for Docker's data root (`/var/lib/docker`), which a default Ubuntu install does not provide. "Therefore, allowing Docker containers to be limited in storage requires extra effort, which means that there is a good chance that **many systems will not limit it at all.**"

Limits and follow-up

Production considerations

Storage-driver setup "requires careful consideration and will be important to the overall health of your systems." See §5.12.2 for the second-order failures: an app that crashes on `ENOSPC` that Docker then cannot restart *because* the disk is full.

The two scripts and Dockerfiles:

```
# control/run.sh
FILESIZE=$((50*1024*1024))
FILENAME=testfile
while :
do
    fallocate -l $FILESIZE $FILENAME \
        && echo "OK wrote the file" `ls -alhi $FILENAME` \
        || echo "Couldn't write the file"
    sleep 2
    rm $FILENAME || echo "Couldn't delete the file"
done
```

```
# failure/consume.sh - same, but never deletes, and keeps counting up
new_name=$FILENAME.$count
fallocate -l $FILESIZE $new_name \
    && echo "OK wrote the file" `ls -alhi $new_name` \
    || (echo "Couldn't write " $new_name "Sleeping"; sleep 5)
(( count++ ))
```

```
FROM ubuntu:focal-20200423      # base image, pinned by tag
COPY run.sh /run.sh            # copy from the build context
ENTRYPOINT ["/run.sh"]         # what runs when the container starts
```

```
cd ~/src/examples/poking-docker/experiment1/control/
docker build -t experiment1-control .      # -t tags it; "." is the build context
docker images                             # list images
docker run --rm -ti experiment1-control
```

Read the build output this way: each Dockerfile line **produces a new intermediate container/layer**, and the last one gets your tag. `fallocate -l <bytes> <name>` allocates space for a file without writing its contents — fast, and enough to consume the filesystem.

5.4.4–5.5 Theory + Practice — Namespaces

Namespaces control which subset of resources is visible to certain processes. Think of them as filters on what a process can see. If a resource is not visible to your namespace, the kernel makes it look like it does not exist (Figure 5.7: resource A visible only to namespace 1, B shared, C only to namespace 2).

The namespace types (as of writing):

Namespace	Isolates
Mounts (mnt)	Which mounts are accessible in the namespace
Process ID (pid)	An independent set of PIDs
Network (net)	Virtualizes the network stack; interfaces, physical or virtual, are attached to net namespaces
Interprocess Communication (ipc)	System V IPC objects and POSIX message queues
UTS (uts)	Host and domain names
User ID (user)	User identification and privilege isolation
Control group (cgroup)	Hides the real identity of the cgroup the processes belong to
Time (time)	Different times in different namespaces — introduced in kernel 5.6, March 2020 ; the book's VM (4.18) does not have it

Linux starts with a single namespace of each type and creates new ones on the fly.

Inspecting namespaces:

```
lsns                                # as your user
sudo lsns                          # as root – many more, incl. PID 1 /sbin/init
lsns --json                        # machine-readable
lsns --type pid                    # one type
sudo lsns --task <PID>             # namespaces of one process  <-- the most useful form

ls -l /proc/$$/ns                  # $$ = current shell's PID; one symlink per namespace type
file /proc/$$/ns/pid               # "broken symbolic link to pid:[4026531836]" – special format
readlink /proc/$$/ns/pid           # pid:[4026531836]

ps ao pid,pidns,command            # print the PID namespace alongside each process
```

The links use the format `<namespace type>:[<namespace number>]`. Under the hood `lsns` reads `/proc/<pid>/ns`.

What Docker creates. Start `docker run --name probe -ti --rm ubuntu:focal-20200423`, get its PID with `docker inspect (.State.Pid)`, then run `sudo lsns --task <PID>`:

```

      NS TYPE  NPROCS  PID USER COMMAND
4026531835 cgroup    210    1 root /sbin/init  <-- SHARED with the host
4026531837 user      210    1 root /sbin/init  <-- SHARED with the host
4026532355 mnt         1  3603 root /bin/bash   <-- new
4026532356 uts         1  3603 root /bin/bash   <-- new
4026532357 ipc         1  3603 root /bin/bash   <-- new
4026532358 pid         1  3603 root /bin/bash   <-- new
4026532360 net         1  3603 root /bin/bash   <-- new
```

Docker creates a new namespace of each type EXCEPT `cgroup` and `user`. By default the container is therefore *not* isolated in the user namespace: `root` inside the container maps to `root` on the host. This is the root of most "container escape" concerns, and one command reveals it.

Experiment Card 5.2 — Killing a process in a different PID namespace

Purpose and concepts

Goal

Confirm that PID namespaces prevent a container from signalling host processes — and confirm you understand *how* they prevent it.

Relevant theory

PID namespaces as visibility filters (§5.4.4).

Setup, hypothesis and boundaries

System / setup

Host runs `pid-printer.sh`, a loop printing `Hi, I'm PID $$` every 2 s. A container runs `ubuntu:focal-20200423` interactively.

Hypothesis

"If we issue a `kill` command from inside the container, for a process outside the container, it should fail."

Steady state

The target process is running and printing.

Observability

Whether the process keeps printing; the error message from `kill`.

Execution and tools

Failure injected

`kill -9 <host PID>` issued from inside the container.

Results, interpretation and lessons

Observed result

Hypothesis confirmed — `bash: kill: (9000) - No such process`. The target keeps running.

Why — the important part

The failure mode is **not "permission denied"**; it is **"no such process"**. `ps a` inside the container lists only PID 1 (`/bin/bash`) and the `ps` itself. The host's PID does not exist in that namespace.

Lesson

Namespaces isolate by **making things invisible**, not by refusing access. Error messages tell you which mechanism stopped you: a permission error means capabilities or DAC; a "not found" means namespaces.

Limits and follow-up

Production considerations

This isolation holds *from inside*. From the host, nothing stops you entering the namespace (below), so host access is container access.

Entering a container's namespace — `nsenter`:

```
# attach-pid-namespace.sh
CONTAINER_PID=$(docker inspect -f '{{.State.Pid}}' experiment2)
sudo nsenter \
```

```
--pid \                # enter the PID namespace...
--target $CONTAINER_PID \  # ...of this process
/bin/bash /home/chaos/src/examples/poking-docker/experiment2/pid-printer.sh
```

The same script now reports **I'm PID 15**, and **ps a inside the container** shows it. Two ways to target a namespace: **--target <pid>**, or the namespace file directly, used later for networking: **sudo nsenter --net=/proc/\$CONTAINER_PID/ns/net**.

5.5.1 DIY container, part 2 — namespaces via **unshare**

```
# syntax: unshare [options] [program [arguments]]
sudo unshare --fork --pid --mount-proc /bin/bash
ps          # your bash reports PID 1
```

```
# container-ish.sh
bash $CURRENT_DIRECTORY/new-filesystem.sh $FILESYSTEM_NAME      # step 1: the chroot tree
cd $FILESYSTEM_NAME
sudo unshare \
  --fork \                # forking is REQUIRED for a PID namespace change to take
effect
  --pid \                 # new PID namespace
  chroot . \              # change the filesystem root
  /bin/bash -c "mkdir -p /proc && /bin/mount -t proc proc /proc && exec /bin/bash"
```

Verify: **ps aux** inside reports PID 1 for bash, and **sudo lsns -t pid** on the host shows the new pid namespace with NPROCS 1.

5.5.2 Theory + Practice — cgroups

Control groups organise processes into hierarchical groups, then limit and monitor their usage of resource types (CPU, RAM, I/O). Figure 5.9: without a limit the process uses any available cycles; with a 50% limit it is **throttled** whenever it tries to exceed 50%.

The kernel exposes a pseudo-filesystem, **cgroupfs**, usually mounted at **/sys/fs/cgroup**.

v1 vs v2. v1 "evolved over the years in a mostly uncoordinated, organic fashion". **v2 was introduced to reorganize, simplify and remove inconsistencies.** At the time of writing most of the ecosystem still used v1 or defaulted to it; runc's v2 work is tracked at github.com/opencontainers/runc/issues/2315. The book sticks to v1. In the listing, **unified** is where cgroups v2 is mounted.

```
ls -al /sys/fs/cgroup/
# blkio cpu -> cpu,cpuacct cpuacct cpu,cpuacct cpuset devices freezer hugetlb
# memory net_cls -> net_cls,net_prio net_cls,net_prio net_prio perf_event pids
# rdma systemd unified
```

Each resource type has a **controller**. Note that **cpu** is a symlink to **cpu,cpuacct**: one controller both **limits** and **accounts for** CPU.

Creating a cgroup = creating a folder under **/sys/fs/cgroup/<resource>/**. Docker creates a parent cgroup **docker/** with one nested folder **per container ID**.

The CPU knobs

```
ls -l /sys/fs/cgroup/cpu/docker
```

File	Meaning
cpu.cfs_period_us	The period in microseconds

File	Meaning
<code>cpu.cfs_quota_us</code>	Microseconds of CPU time the group may consume within that period . <code>-1</code> (default) = no limit. A hard limit.
<code>cpu.shares</code>	Arbitrary relative weight . Same value = same share. Default 1024 . Enforced only when there isn't enough CPU for everyone ; otherwise no effect. A soft limit.
<code>cpu.stat</code>	<code>nr_periods, nr_throttled, throttled_time</code>
<code>cgroup.procs / tasks</code>	The PIDs in this cgroup
<code>cpuacct.*</code>	Accounting: usage totals, per-CPU, user/sys split

Example: 50% of a CPU = `cpu.cfs_period_us=1000`, `cpu.cfs_quota_us=500`.

Docker's defaults for a fresh container: `cpu.cfs_period_us=100000`, `cpu.cfs_quota_us=-1`, `cpu.shares=1024`. That is **no hard limit, default weight**.

```
export CONTAINER_ID=$(docker inspect -f '{{ .Id }}' <name>)
ps -p $(cat /sys/fs/cgroup/cpu/docker/$CONTAINER_ID/cgroup.procs) # what's inside the cgroup
```

The memory knobs

```
ls -l /sys/fs/cgroup/memory/docker/$CONTAINER_ID/
```

File	Meaning
<code>memory.limit_in_bytes</code>	Hard RAM limit. Default <code>9223372036854771712</code> — max 64-bit int minus a page size, effectively infinity. Writable.
<code>memory.usage_in_bytes</code>	Current RAM utilization (read-only)
<code>memory.max_usage_in_bytes</code>	High-water mark
<code>memory.failcnt</code>	How many times the limit was hit
<code>memory.oom_control</code>	OOM behaviour for this cgroup
<code>memory.soft_limit_in_bytes, memory.swappiness, memory.kmem.*, memory.stat</code>	Soft limit, swap tendency, kernel-memory accounting, statistics

```
# impose a 20 MB limit by writing to the file
echo 20971520 | sudo tee /sys/fs/cgroup/memory/docker/$CONTAINER_ID/memory.limit_in_bytes
```

"When practicing chaos engineering on systems leveraging Docker, **you must understand and be able to observe the limits that your applications are running with.**" `cgcreate` and `cgexec` (Chapter 3) are convenient; reading cgroupfs directly is what lets you verify.

Experiment Card 5.3 — Using all the CPU you can find (`--cpus`)

Purpose and concepts

Goal

Verify that Docker's `--cpus` hard limit actually caps CPU consumption.

Relevant theory

cgroup CPU quota/period vs. shares; hard vs. soft limits.

Setup, hypothesis and boundaries

System / setup

Image `stressful = ubuntu:focal-20200423` • `apt-get install -y stress`. Two-CPU VM.

Hypothesis

"If we run `stress` in CPU mode, in a container started with `--cpus=0.5`, it will use no more than 0.5 processor on average."

Steady state

CPU utilization close to 0 (`%idle` $\approx$ 99.75).

Observability

`mpstat -u -P ALL 2`; cross-check via `/sys/fs/cgroup/cpu/docker/$CONTAINER_ID/cpu.stat`.

Blast radius

One container's cgroup — this is the *point* of the experiment.

Execution and tools

Failure injected

`stress --cpu 1 --timeout 30` inside a `--cpus=0.5` container.

Results, interpretation and lessons

Observed result

Hypothesis confirmed. One CPU at ~49%, the other ~0%, total ~24.5% of two CPUs = half a CPU. `cpu.stat` shows `nr_periods 311`, `nr_throttled 304`, `throttled_time 15096182921` and rising.

Why

`--cpus=0.5` sets `cpu.cfs_period_us=100000` and `cpu.cfs_quota_us=50000`. The kernel throttles the group once the quota is spent in each period. `cpu.shares` stays 1024.

Lesson

`--cpus=N` $\equiv$ period 100000 / quota $N \times 100000$. **`throttled_time` and `nr_throttled` prove throttling is happening.** Check them first when a containerised app is mysteriously slow.

Limits and follow-up

Production considerations

With a higher `--cpu` worker count the load spreads across both CPUs, but the *average* stays capped. A CPU-limited container does not report "out of CPU" anywhere except `cpu.stat`.

```
docker run --cpus=0.5 -ti --rm --name experiment3 stressful
# in another window:
mpstat -u -P ALL 2
# inside:
stress --cpu 1 --timeout 30
# verify:
CONTAINER_ID=$(docker inspect -f '{{ .Id }}' experiment3)
cat /sys/fs/cgroup/cpu/docker/$CONTAINER_ID/cpu.stat
```

Docker's two CPU controls mirror the cgroup ones: `--cpus` = hard limit; `--cpu-shares` = relative weight.

Experiment Card 5.4 — Using too much RAM (`--memory`)

Purpose and concepts

Goal

Find out what actually happens when a container reaches its memory limit.

Relevant theory

cgroup memory limits; the OOM Killer (Ch. 2); virtual vs. resident memory.

Setup, hypothesis and boundaries

System / setup

Same `stressful` image, started with `--memory=128m`. `--memory` accepts `b`, `k`, `m`, `g`.

Hypothesis

"If we run `stress` in RAM mode, trying to consume 512 MB, in a container started with `--memory=128m`, it will use no more than 128 MB of RAM."

Steady state

No OOM-kill logs in `dmesg` (`dmesg | egrep "Kill|oom"` — note existing timestamps first).

Observability

`top` (VIRT vs RES); `dmesg | grep Kill; watch -n 1 sudo cat /sys/fs/cgroup/memory/docker/$CONTAINER_ID/memory.usage_in_bytes`.

Execution and tools

Failure injected

`stress --vm 1 --vm-bytes 512M --timeout 30`; then `--vm 3`; then a **fork bomb**.

The fork bomb

`boom () { boom | boom & }; boom` — recursive self-calls exhausting resources. Now `memory.usage_in_bytes` oscillates slightly above 128 MB; `top` shows **~89% system CPU time**; inside the container `bash: fork: Cannot allocate memory`; `dmesg | grep Kill` shows `Memory cgroup out of memory: Kill process 1929 (bash) score 2 or sacrifice child`.

Results, interpretation and lessons

Observed result

Hypothesis fails — in an instructive way. `stress` shows **VIRT 528152 KiB but RES 127400 KiB**, just under the limit, and **completes successfully: successful run completed in 30s**. With `--vm 3`, all three workers show 512 MB virtual but their resident total is ~115 MB, still under the limit.

Why

"**Just allocating the memory doesn't trigger the OOM Killer**" — `stress` does not touch all the memory it allocates, so the kernel never has to back it with physical pages. Virtual allocation is free; residency is what counts.

Three lessons the book draws

1. Allocating memory does not trigger the OOM Killer; you can allocate far more than the cgroup allows.
2. Under a fork bomb the **total memory used was slightly higher than the container's limit** — useful for capacity planning.
3. **Running the OOM Killer against a fork bomb costs a non-negligible amount of CPU**; consider `--oom-kill-disable` for a container if you have done your capacity maths.

Limits and follow-up

Production considerations

A memory limit is enforced against **resident** memory, and enforcement arrives as a kill, not as an error your app can handle. Budget headroom above the limit and expect a CPU spike during OOM reaping.

5.7.1 DIY container, part 3 — cgroups

```
# container-ish-2.sh
CPU_LIMIT=${1:-50000}
RAM_LIMIT=${2:-5242880}

echo "Step A: generate a unique ID (uuid)"
UUID=$(date | sha256sum | cut -f1 -d" ")

echo "Step B: create cpu and memory cgroups"
sudo mkdir /sys/fs/cgroup/{cpu,memory}/${UUID}
echo $RAM_LIMIT | sudo tee /sys/fs/cgroup/memory/${UUID}/memory.limit_in_bytes
echo 100000 | sudo tee /sys/fs/cgroup/cpu/${UUID}/cpu.cfs_period_us
echo $CPU_LIMIT | sudo tee /sys/fs/cgroup/cpu/${UUID}/cpu.cfs_quota_us

echo "Step C: prepare the folder structure to be our chroot"
bash $CURRENT_DIRECTORY/new-filesystem.sh $UUID > /dev/null && cd $UUID

echo "Step D: put the current process (PID $$) into the cgroups"
echo $$ | sudo tee /sys/fs/cgroup/{cpu,memory}/${UUID}/tasks # children inherit automatically

echo "Step E: start our namespaced chroot container-ish: $UUID"
sudo unshare --fork --pid chroot . \
/bin/bash -c "mkdir -p /proc && /bin/mount -t proc proc /proc && exec /bin/bash"
```

Verify from the host:

```
ps -ao pid,command -f # find the inner bash PID
ps -p <PID> -o pid,cgroup -ww # -ww: don't truncate; shows memory:/<UUID> and
cpu,cpuacct:/<UUID>
```

What the DIY container now contains (Figure 5.11): filesystem access (chroot), PID namespace separation, CPU and RAM limits. **Still missing: networking, capabilities, seccomp** — "its networking access is still exactly the same as for any other process running on the host, and we haven't covered any security features at all."

5.8 Theory + Practice — Docker networking

```
docker network ls
# bridge (driver bridge) | host (driver host) | none (driver null)
```

Mode	Behaviour
none	No networking is set up. Useful to isolate a container from the network entirely. A runtime option — you can <i>build</i> an image by downloading packages and then <i>run</i> it with no network. Driver: null .
host	The container uses the host's networking setup directly, with no isolation . Ports used inside are the same as outside. Driver: host .
bridge	The interesting one. A bridge is an interface connecting multiple networks and forwarding traffic between them — think of it as a network switch.

How bridge mode works (Figure 5.12):

1. Docker creates a bridge interface called **docker0** and connects it to the host's logical interface.

2. For each container, Docker creates a **net namespace**, so interfaces are accessible only to processes in that namespace.
3. Inside that namespace Docker creates a **virtual interface connected to the `docker0` bridge** and a **local loopback device**.

A packet from inside a container goes out through its virtual interface → the bridge → then either to another container's virtual interface, or to the host's logical interface and onto the physical network.

```
ip addr
# 1: lo ...
# 2: eth0 ... inet 10.0.2.15/24
# 3: docker0 ... inet 172.17.0.1/16
```

Creating a custom network:

```
docker network create \
  --driver bridge \           # bridge driver for host connectivity
  --attachable \             # allow containers to attach manually
  --subnet 10.123.123.0/24 \  # the network's subnet
  --ip-range 10.123.123.0/25 \ # allocate container IPs only from this sub-range
  chaos
```

A new host interface appears: `br-<network id>` with `inet 10.123.123.1/24`. A container started with `--network chaos` gets `eth0@if6` with `10.123.123.2/24`. Confirm with `hostname -I`.

Tying it back to namespaces:

```
sudo lsns -t net                # list net namespaces
CONTAINER_PID=$(docker inspect -f '{{ .State.Pid }}' explorer)
sudo readlink /proc/$CONTAINER_PID/ns/net  # net: [4026532245]
sudo nsenter --net=/proc/$CONTAINER_PID/ns/net  # enter it directly by namespace FILE
ip addr                                # identical output to inside the
container
```

This is the mechanism Pumba uses (§5.11), and the one behind Kubernetes' pause container (Chapter 12): a second process can join an existing container's net namespace and manipulate its interfaces, including running `tc` against them.

5.8.1 Capabilities and seccomp

Capabilities split superuser privileges, which skip all checks, into **smaller, granular units**. The book's examples: `CAP_KILL` bypasses permission checks for sending signals to processes; `CAP_SYS_TIME` allows changing the system clock.

```
docker run --name cap_explorer -ti --rm ubuntu:focal-20200423
# in another window:
CONTAINER_PID=$(docker inspect -f '{{ .State.Pid }}' cap_explorer)
getpcaps $CONTAINER_PID
```

Docker's default capability set:

```
cap_chown, cap_dac_override, cap_fowner, cap_fsetid, cap_kill, cap_setgid,
cap_setuid, cap_setpcap, cap_net_bind_service, cap_net_raw, cap_sys_chroot,
cap_mknod, cap_audit_write, cap_setfcap
```

The demonstration — chroot inside a chroot:

```
NEW_FS_FOLDER=new_fs
mkdir $NEW_FS_FOLDER
cp -v --parents `which bash` $NEW_FS_FOLDER
ldd `which bash` | egrep -o '(/usr)?/lib.*\.[0-9][0-9]?' | xargs -I {} cp -v --parents {}
$NEW_FS_FOLDER
chroot $NEW_FS_FOLDER `which bash`      # works - cap_sys_chroot is granted by default
```

Restart with `--cap-drop ALL`:

```
docker run --name cap_explorer -ti --rm --cap-drop ALL ubuntu:focal-20200423
getpcaps $CONTAINER_PID      # Capabilities for `4813': = (none)
# retry the chroot:
# chroot: cannot change root directory to 'new_fs': Operation not permitted
```

`--cap-add` and `--cap-drop` accept individual capabilities or the keyword **ALL**. "It's always a good idea to give the container only what it really needs."

seccomp filters **which syscalls a process can make**. Under the hood **seccomp uses BPF**, the same BPF from Chapter 3. Docker uses seccomp to limit the default set of allowed syscalls. Profiles are JSON:

```
{
  "defaultAction": "SCMP_ACT_ERRNO",      // by default, block all calls
  "syscalls": [
    { "names": [ "accept", "accept4", ... "write", "writev" ],
      "action": "SCMP_ACT_ALLOW" } // allow these
  ]
}
```

Use a custom one with `--security-opt seccomp=/my/profile.json`. Key point: **seccomp is a kernel feature — you can use it without Docker**. Chapter 6 does exactly that, both through Docker and through libseccomp directly.

5.9 Docker demystified — the consolidated mental model

	Linux kernel	
chroot	namespaces	cgroups
↓	↓	↓
change the filesystem root	isolate WHAT a process can SEE (PIDs, mounts, net)	limit WHICH RESOURCES and HOW MUCH a process can USE (CPU, RAM, I/O)
networking	capabilities	seccomp
↓	↓	↓
bridge/host/none, veth + net namespaces	granular superuser privileges (CAP_KILL, ...)	filter which SYSCALLS are allowed (implemented with BPF)
filesystems → Unionfs / overlay2: layers + copy-on-write		

"Containers are implemented with a collection of **loosely connected technologies** and... in order to know what to expect from a dish, you need to know the ingredients." Recommended follow-up: `man namespaces`, `man cgroups`, the Docker docs.

5.10–5.11 Practice — Fixing the slow Dockerized app

Bring the stack up with **docker stack deploy**:

```
# meower-stack.yml
version: '3.1'
services:
  ghost:
    image: ghost:3.14.0-alpine
    ports:
      - 8368:2368 # host port : container port
    environment:
      database__client: mysql
      database__connection__host: db # service name resolves to the MySQL container
      database__connection__user: root
      database__connection__password: notveryunsafe
      database__connection__database: ghost
      server__host: "0.0.0.0"
      server__port: "2368"
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: notveryunsafe
```

```
docker swarm init # required for stack commands
docker stack deploy -c ~/src/examples/poking-docker/meower-stack.yml meower
docker stack ls
docker ps
```

A detail worth noticing in the book's own output: **"the ghost container will crash and restart until the mysql container is actually ready, and that one takes longer to start."** That is a real dependency-ordering failure mode, visible in a two-service stack, and exactly what §5.12.1 asks you to experiment with at scale.

Two plausible causes of slowness, given the chapter's knowledge:

1. **CPU starvation** — "I've seen it happen a lot, when someone... else... typed one zero too few or too many." Now trivially checkable: read `cpu.stat` of the container's cgroup and look for throttling.
2. **Fragility to database network slowness** — the Chapter 4 finding, now in a modern stack. "It's a common gotcha to make assumptions based on... test environments and local databases, and then be surprised when networking slows down in the real world."

Pumba — the Docker chaos tool

Pumba (github.com/alexei-led/pumba) can kill containers, emulate network failures using `tc` under the hood, and run stress tests using `stress-ng` from inside a particular container's cgroup.

```
USAGE: pumba [global options] command [command options] containers (name, list of names, RE2 regex)
COMMANDS:
  kill      kill specified containers
  netem     emulate the properties of wide area networks
  pause     pause all processes
  stop      stop containers
  rm        remove containers
```

The gotcha and its workaround — a genuinely useful trick. `pumba netem` executes `tc` from inside the target container, so `tc` must exist there, which is unlikely outside a test image. The workaround: Docker can start a container that **shares another container's networking configuration**. Pumba exposes this as `--tc-image`: it starts a throwaway container that *has* `tc`, for example `gaiadocker/iproute2`, in the target's net namespace, and the `tc` command affects both.

```
pumba netem \
  --duration 60s \ # how long the impairment stays
```

```

--tc-image gaiadocker/iproute2 \      # image that has tc, run in the target's net namespace
delay \                               # the netem sub-command
--time 100 \                          # 100 ms
--jitter 0 \                          # no random jitter – keeps the analysis clean
--correlation 0 \                    # no correlation between events
"re2:meower_db"                      # target by RE2 regular expression

```

Proof of the mechanism afterwards:

```

docker ps --all
# gaiadocker/iproute2  "tc qdisc del dev et..." Exited (0)
# gaiadocker/iproute2  "tc qdisc add dev et..." Exited (0)
docker inspect <id>    # "Entrypoint": ["tc"], "Cmd":
["qdisc", "del", "dev", "eth0", "root", "netem"]

```

Note the **pair**: one container adds the qdisc, another deletes it. The teardown is a first-class part of the injection.

Experiment Card 5.5 — 100 ms latency to MySQL with Pumba

Purpose and concepts

Goal

Measure how Ghost's user-visible response time reacts to added latency on its database connection.

Relevant theory

Latency compounding across round trips (Ch. 4); net namespaces (§5.8); **tc/netem** (Ch. 4).

Setup, hypothesis and boundaries

System / setup

meower Swarm stack: **ghost:3.14.0-alpine** on host port 8080 → **mysql:5.7**, both containers.

Hypothesis

"If you introduce 100 ms latency to network connectivity between Ghost and MySQL, you should see the average website latency go up by 100 ms."

Steady state

ab -t 30 -c 1 -l http://127.0.0.1:8080/ → 26.328 ms mean, 0 failed requests, 1140 complete requests. Concurrency 1 deliberately, since the same CPUs generate and serve the traffic.

Observability

ab: mean time per request and failed requests.

Blast radius

One container, matched by regex; 60 seconds; automatically reverted by Pumba's teardown container.

Execution and tools

Failure injected

pumba netem --duration 60s --tc-image gaiadocker/iproute2 delay --time 100 --jitter 0 --correlation 0 "re2:meower_db"

The control experiment

Rerun with **--time 1**, the tool's minimum: **36.212 ms mean**.

Results, interpretation and lessons

Observed result

Hypothesis refuted. 62 complete requests, 0 failures, **490.128 ms mean** — 26 ms → 490 ms, a factor of more than **18**.

The back-of-napkin validation

The 1 ms run puts an **upper bound on the injector's own overhead**: $36 - 26 = 10$ ms per request. Assuming the worst case, a single packet delayed by 1 ms, the theoretical overhead is ~9 ms. The 100 ms run added $490 - 26 = 464$ ms, so even 9 ms of tooling overhead is ~2% of 490 ms. **The result is therefore plausible, not an artefact.**

Why it happened

Ghost, like WordPress, makes many database round trips per page, and each one absorbs the full added delay. (*Supplementary arithmetic, not stated in the book: $\sim 464 \text{ ms} \div 100 \text{ ms} \approx 4\text{--}5$ round trips per request.*)

Lesson learned

Two: (1) the Chapter 4 finding is not a WordPress quirk — it is a property of chatty ORMs and request paths in general; (2) **always measure the overhead of your fault injector before believing a dramatic result.** The 1 ms control run is the most professional move in the chapter.

Limits and follow-up

Production considerations

With this data you can bound the acceptable database RTT for an SLO, or reduce round trips. Pumba's `—duration` gives automatic rollback; use it rather than relying on remembering to revert.

5.12 Theory — Other parts of the puzzle

5.12.1 Docker daemon restarts

"In its current model, a restart of the Docker daemon means a **restart of all applications running on Docker on that host.**" Imagine a host running a few hundred containers and Docker crashing. A simple restart experiment answers:

- How long until all applications are started again?
- Do some containers depend on others, making start order important?
- How do containers react when resources are consumed starting other containers — the **thundering herd problem**?
- Are you running **infrastructure processes**, such as an overlay network, on Docker? What if that container does not start before the others?
- If Docker crashes at the wrong moment, can it recover from an inconsistent state? Does any state get corrupted?
- **Does your load balancer know when a service is really ready**, rather than just starting?

5.12.2 Storage for image layers

Beyond Experiment 1:

- What if an application does not handle lack of space, crashes, **and Docker cannot restart it because of the lack of space**?
- Will Docker have enough storage to download the layers needed for a new container? **The total decompressed storage needed is difficult to predict.**
- How much storage does Docker itself need to start if it crashed when the disk was full?

"A lot of damage can be caused by a single faulty loop writing too much data to the disk, and **running processes in containers might give a false sense of safety** in this respect."

5.12.3 Advanced networking

Docker is often combined with **overlay networks (Flannel)**, **cloud-aware networking (Calico)** and **service meshes (Istio)**, on top of **iptables** and **IPVS**. Complexity grows quickly. Chapter 12 returns to this for Kubernetes.

5.12.4 Security

Chaos techniques are worth applying to security. The commonly-found, easily-fixed problems:

- Containers running with **---privileged**, often without a good reason
- **Running as root inside the container**, the default almost everywhere
- **Unused capabilities** given to containers
- Using **random Docker images from the internet**, often without looking inside
- Running **ancient image versions** with known flaws
- Running an **ancient Docker** with known flaws

Reference given: "Understanding Docker Container Escapes" (Trail of Bits).

Theory ↔ Practice connections for Chapter 5

- **chroot + union filesystems (§5.4.1) ↔ Experiment 1 (§5.4.3)**: the theory says containers share one host filesystem; the experiment shows one container filling it for everyone. The inode check bridges the two.
 - **Namespaces (§5.4.4) ↔ Experiment 2 (§5.5)**: the kill fails with *no such process*, which is the theory's "filter on visibility" made observable.
 - **cgroups (§5.5.2) ↔ Experiments 3 and 4**: CPU limits behave exactly as the quota/period theory predicts. **Memory limits do not behave as naïvely expected**, which makes Experiment 4 the more valuable one.
 - **cgroups in Chapter 3 (§3.3.5 busy neighbour) ↔ §5.5.2**: the same mechanism, now seen as a container building block rather than a scheduling fix.
 - **Net namespaces (§5.8) ↔ Pumba's ---tc-image (§5.11.1)**: Pumba's trick only makes sense once you know a second container can join an existing net namespace. Chapter 12's pause container is the same idea inside Kubernetes.
 - **tc/netem raw (Ch. 4) ↔ Pumba (§5.11)**: the book showed the painful form first so you understand the wrapper instead of trusting it blindly.
 - **Latency compounding (Ch. 4, WordPress) ↔ Experiment 5 (Ghost)**: two different stacks, same failure shape. The repetition is the evidence that it is a general pattern.
 - **OOM Killer (Ch. 2) ↔ Experiment 4**: the same killer, now scoped to a cgroup ("Memory cgroup out of memory"), with a measurable CPU cost.
 - **seccomp mentioned here (§5.8.1) ↔ Chapter 6 (§6.5)**: full treatment, both via Docker and via libseccomp.
-

Key Takeaways — Chapter 5

1. **VMs virtualize hardware (own kernel, strong isolation); containers virtualize the OS (shared kernel, lightweight isolation)**. VMs are more secure. Containers are cheaper and faster, and are best understood as *packaging with extra benefits*.
2. Docker did not invent containers. It packages seven kernel features — chroot, namespaces, cgroups, capabilities, networking, union filesystems, seccomp — behind a good UX.
3. **Namespaces = what you can SEE. cgroups = what you can USE**. Memorise this; everything else follows.
4. Docker creates new **mnt**, **uts**, **ipc**, **pid** and **net** namespaces per container, but **shares cgroup and user** with the host by default.
5. Container isolation is **thin**: the same inode appears inside and outside, and **nsenter** lets any host process join any container's namespace.
6. **Disk space is not isolated by default**. **---storage-opt size=X** needs xfs plus pquota under overlay2, so most installations have no per-container storage limit at all.
7. **---cpus=N** is a **hard** cgroup quota; **---cpu-shares** is a **soft** relative weight enforced only under contention. **cpu.stat's nr_throttled** and **throttled_time** prove throttling.
8. Memory limits apply to **resident**, not virtual, memory. Allocating memory you never touch does not trigger enforcement. When enforcement comes it is the cgroup OOM Killer, and against a fork bomb it costs serious CPU.
9. A DIY container is three ingredients: a prepared filesystem plus **chroot**, **unshare --fork --pid**, and a cgroup folder with your PID written into **tasks**.
10. Docker networking is **none** / **host** / **bridge**. Bridge mode = **docker0** plus a per-container net namespace with a veth pair and a loopback.
11. Capabilities split root into granular units. Docker grants 14 by default, including **cap_sys_chroot**. **---cap-drop ALL** is one flag away. seccomp filters syscalls using BPF.
12. **Pumba** wraps **tc/netem**, stress-ng and container kill/stop/pause/rm. It targets containers by name or RE2 regex, with **---duration** for automatic rollback and **---tc-image** for the net-namespaces trick.
13. **Always run a near-zero control injection to measure your injector's own overhead** before you trust a dramatic result.

14. Apply chaos engineering to Docker itself, not only to the applications on it: daemon restarts, layer storage exhaustion, advanced networking, and security posture.

Chapter 5 — Who You Gonna Call? Syscall-Busters!

The chapter's thesis: **"it's hard to find a piece of software that can't benefit from chaos engineering, even if it's closed source."** Syscalls are the universal injection point, because every program makes them.

6.1 Practice — The scenario: legacy System X

You were promoted. The promotion came with a legacy system whose last knowledgeable maintainer quit.

The book's framing of "legacy," worth quoting in a real argument. Sometimes good reasons exist to keep the same codebase for a long time: requirements have not changed, it runs fine on modern hardware, there is a talent pool. Other times software is archaic for all the wrong reasons — **sunk-cost fallacy, vendor lock-in, bad planning**. "Even modern code can be considered legacy if it's not well maintained." The type this chapter targets: **the kind that works, but no one really knows how**.

What you have (Figure 6.1): no documentation; a binary and its source, found by interviewing senior staff; tribal knowledge that it serves an HTTP interface everyone uses. Request and response formats undocumented. "There be dragons."

```
cd ~/src/examples/who-you-gonna-call/src/
find ~/src/examples/who-you-gonna-call/src/ -name "*.c" -o -name "*.h" | sort | xargs wc -l
# 3128 total
make
./legacy_server
# Listening on port 8080, PID: 1649
```

The source is a *simulated* legacy application in C, deliberately "awfully complicated for what it's doing," generated by `generate_legacy.py` — **which the book asks you not to read until after the chapter**. That constraint is the pedagogy: you will find and characterise a real fragility **without ever reading the source**.

The chapter's method statement: inject failure **on the boundary between the application and the system**, a boundary every program must cross, and see how the application copes with errors from the system. That boundary is the set of syscalls.

6.2 Theory — Syscalls

Syscalls are the APIs of an operating system. For a program running on an OS, syscalls are the way to communicate with the kernel. Even a Hello World program uses a syscall to print its message.

Basic examples:

Syscall	Does
<code>open</code>	Opens a file
<code>read</code>	Reads from a file, or something file-like such as a socket
<code>write</code>	Writes to a file, or something file-like
<code>exec</code>	Replaces the currently running process with another, read from an executable file
<code>kill</code>	Sends a signal to a running process

Kernel space vs. user space (Figure 6.2). In kernel space only kernel code runs — subsystems and most drivers — and it has access to the underlying hardware. Everything else runs in **user space (userland)** with no direct hardware access. The sequence: user runs a program → program executes a syscall → **kernel implementation validates and executes it** → kernel accesses the hardware → results are made available back in user space.

Why not program the hardware directly? Nothing stops you, as embedded systems and unikernels show, but for everything else the well-defined API wins:

- **Portability** — code written against the Linux kernel API runs on any architecture Linux supports.
- **Security** — the kernel verifies syscalls are legal and prevents accidental hardware damage.
- **Not reinventing the wheel** — virtual memory, filesystems and the rest are implemented and thoroughly tested.
- **Rich features** — user management and privileges, drivers, advanced memory management.
- **Speed and reliability** — the kernel implementation, tested daily on millions of machines, beats what you would write.

The cost is **overhead**, easily outweighed for most use cases.

Linux is POSIX-compliant, so much of the API is standardized, and the same or similar syscalls exist in other UNIX-likes such as the BSD family.

6.2.1 Finding out about syscalls — `man` sections

`man` has **sections 1–9**, and different sections can cover items with the same name:

Section	Contains
1	Executable programs or shell commands
2	System calls (functions provided by the kernel)
3	Library calls (functions within program libraries)
4	Special files, usually in <code>/dev</code>
5	File formats and conventions, for example <code>/etc/passwd</code>
6	Games
7	Miscellaneous (macro packages and conventions)
8	System administration commands, usually root only
9	Kernel routines (non-standard)

```
man man           # the section table above
man 2 syscalls    # every syscall, with the kernel version that introduced it
man 2 read        # signature, description, errors, caveats
```

`man 2 read` gives the shape of everything that follows:

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count);
```

The chaos-engineering use of `man 2`: before you inject a failure into a syscall, read its **ERRORS** section. It tells you which failures are *realistic*. Experiment 1 below turns that list into an argument about production risk.

6.2.2 The standard C library and glibc

A standard C library implements the functions whose signatures appear in section 2. Those signatures live in `unistd.h`. `man unistd.h` lists what a standard C library must implement — the POSIX standard.

glibc (GNU C Library) is the most common implementation for Linux. It is three decades old, widely relied on, and criticised as bloated. Alternatives that focus on a smaller footprint: **musl libc** and **diet libc**. See `man 7 libc`.

The critical subtlety: the wrappers are not pass-throughs. glibc's source includes a list of pass-through syscalls whose C code is generated by a script. For version 2.23 **that list has only ~100 of the ~380 syscalls, so almost three-quarters contain auxiliary code.**

The book's example: `exit(3)` in glibc runs any functions preregistered with `atexit(3)` *before* it executes the actual `_exit(2)` syscall. **There is no guaranteed one-to-one mapping between C library functions and the syscalls they implement.** Argument names can also differ between section 2 and section 3 documentation; `man 3 read` shows the C library's signature.

The corrected picture (Figure 6.3): user runs a program → program calls a **libc wrapper** → the wrapper executes the syscall → kernel validates and executes it → kernel touches hardware.

libc matters regardless of your programming language — "that's why using a Linux distribution relying on musl libc (like **Alpine Linux**) might sometimes bite you in the neck when you least expect it."

Pop quiz — what are syscalls? *All of the above:* a way to request actions on physical devices; a way for a process to communicate with the kernel; and **"a universal angle of attack for chaos experiments, because virtually every piece of software relies on syscalls."**

6.3 Practice — Observing a process's syscalls

6.3.1 `strace` and `sleep`

```
sudo strace sleep 1
```

Each line is one syscall: **name, arguments, and the returned value after =**. `sleep 1` makes **12 unique syscalls, 39 total**, and only one of them is the sleep.

The walk-through, condensed into a reference you can reuse on any `strace` output:

Syscall	What it did here
<code>execve("/usr/bin/sleep", ["sleep", "1"], ...)</code>	Replaces the current process with a new one from an executable. Three args: path, argv, environment. This is how a program starts.
<code>brk(NULL)</code>	Reads (arg NULL) or sets the end of the process's data segment
<code>access("/etc/ld.so.preload", R_OK) = -1 ENOENT</code>	Checks user permissions on a file. Returns -1 because it does not exist. <code>man 8 ld.so</code>
<code>openat(AT_FDCWD, "/etc/ld.so.cache", ...) = 3</code>	Opens a file, returns file descriptor 3 . The <code>at</code> suffix marks the variant that handles relative paths
<code>fstat(3, {...})</code>	File status via that descriptor
<code>mmap(..., 3, 0)</code>	Maps the file into the process's virtual memory
<code>close(3)</code>	Closes it; fd 3 is then reused for the next open
<code>openat(... "/lib/x86_64-linux-gnu/libc.so.6" ...), read(3, "\177ELF...", 832) = 832</code>	Loading libc . Note the display quirk: the second parameter is the <i>destination buffer</i> , so showing its contents is confusing. The return value is bytes read
<code>mprotect(addr, len, PROT_NONE) / PROT_READ</code>	Protects a region of mapped memory. <code>PROT_NONE</code> = no access at all. Boilerplate
<code>arch_prctl(ARCH_SET_FS, ...)</code>	Sets architecture-specific process state — ignorable
<code>munmap(...)</code>	Removes an earlier mapping
<code>brk(NULL)</code> then <code>brk(0x...)</code>	Reads, then moves the end of the data segment — effectively allocating memory
<code>openat(... "/usr/lib/locale/locale-archive" ...)</code>	Locale data: open, fstat, mmap, close
<code>clock_nanosleep(CLOCK_REALTIME, 0, {tv_sec=1, tv_nsec=0}, NULL) = 0</code>	The thing you actually asked for
<code>close(1), close(2)</code>	Closes stdout and stderr
<code>exit_group(0)</code>	Terminates with exit code 0

The summary mode:

```
sudo strace -C -S calls sleep 1    # -C: print a summary; -S calls: sort it by call count
```

```
% time  seconds  usecs/call  calls  errors  syscall
0.00 0.000000      0      8          mmap
0.00 0.000000      0      6          pread64
...
0.00 0.000000      0      1          clock_nanosleep
-----
100.00 0.000000          39      2 total
```

The observation that matters: "this simple program spent much longer doing things you didn't explicitly ask it to do, rather than what you asked." The syscall you cared about is 1 of 39. When you inject faults at this layer, **most of what you can break is startup and library loading, not business logic**, which is exactly why the results surprise you.

6.3.2 strace and System X

```
# terminal 1
~/src/examples/who-you-gonna-call/src/legacy_server    # prints "Listening on port 8080, PID: 6757"
# terminal 2 - attach to a RUNNING process with -p
sudo strace -C -p $(pidof legacy_server)
# then refresh http://127.0.0.1:8080/ in the browser
```

```
accept(3, {sa_family=AF_INET, sin_port=htons(53698), sin_addr=inet_addr("127.0.0.1")}, [16]) =
4
read(4, "GET / HTTP/1.1\r\nHost: 127.0.0.1:...", 2048) = 333
write(4, "HTTP/1.0 200 OK\r\nContent-Type: t...", 122) = 122
write(4, "<", 1) = 1
write(4, "!", 1) = 1
...
fsync(4) = -1 EINVAL (Invalid argument)
close(4) = 0
```

The profile of the black box, discovered in one command:

```
% time  seconds  usecs/call  calls  errors  syscall
98.34 0.002903      10     292          write
0.68 0.000020      20      1          close
0.61 0.000018      18      1          accept
0.34 0.000010      10      1          read
0.03 0.000001       1      1      1 fsync
```

Two findings, without reading a line of source:

1. **A bug:** the program calls `fsync` on a socket and gets `EINVAL` every time. `fsync` synchronises a file's in-core state with storage, and a socket is not a file in that sense.
2. **A pathology:** 292 writes, almost all one character at a time, consuming 98.34% of syscall time.

Sampling caveat. Attaching `strace` to a running process samples **only the syscalls made while you were attached**. You will miss the program's initial setup. Starting the process under `strace` catches everything but requires a restart.

6.3.3 strace's problem: overhead

From `man strace(1)`, under **BUGS**: "A traced process runs slowly."

The measurement, borrowed from Brendan Gregg's "strace Wow Much Syscall" post:

```
# baseline: 500,000 one-byte read+write pairs
dd if=/dev/zero of=/dev/null bs=1 count=500k
# 512000 bytes copied, 0.509962 s, 1.0 MB/s

# traced – filtering for a syscall dd never makes
strace -e accept dd if=/dev/zero of=/dev/null bs=1 count=500k
# 512000 bytes copied, 58.4923 s, 8.8 kB/s
```

More than a 100-fold slowdown, while filtering for a syscall the program never calls. The act of attaching costs it.

Why `dd` is the right benchmark: it does little except `read` then `write`. `/dev/zero` is an infinite source and `/dev/null` discards everything, so you measure syscall speed almost purely.

Two consequences for practice: (1) `strace` may be fine in a test environment, but **attaching it to a production process can have serious consequences**; (2) **if you investigate performance with `strace` attached, all your numbers are wrong**. For chaos experiments specifically: measure your steady state *with `strace` attached*, or you are not comparing like with like.

The mechanism: `strace` works through the `ptrace(2)` syscall.

6.3.4 BPF — the low-overhead alternative

BPF was originally designed to filter network packets. **eBPF** extended it into a **generic Linux kernel execution engine that runs programs with guarantees of safety and performance**. "BPF" usually means eBPF today.

Why it fits chaos engineering: efficient programs run on kernel events, with **limits on execution time and memory access** and **built-in efficient aggregation primitives**, so you can trace everything with minimal overhead. The downside is a **steep learning curve** — writing a meaningful program routinely means reading kernel internals.

BCC (BPF Compiler Collection) softens that with wrappers in Python and Lua plus many tools and examples. "Reading through these tools and examples is currently the best way of starting with BPF."

```
sudo apt-get install bpfcc-tools linux-headers-$(uname -r)

sudo syscount-bpfcc                # count syscalls of ALL processes, print top 10
sudo syscount-bpfcc -p $(pidof legacy_server) # only this PID
less $(which syscount-bpfcc)        # read its source – it's Python
```

The overhead comparison — the reason to prefer BPF:

Tool	dd 500k ops	Overhead
Nothing attached	0.509962 s	—
strace -e accept (one unused syscall)	58.4923 s	~100x slower
syscount-bpfcc (tracing <i>everything</i> on the host)	0.541597 s	~6%

The BPF run traced the whole kernel, not one PID, in a close-to-worst case where `dd` does almost nothing but syscalls.

What you give up: `syscount` gives **counts only**, with no per-call arguments or return values. Table 6.1 in the book puts the two outputs side by side. Both report 292 `write`, and 1 each of `accept`, `read`, `close`, `fsync`. `strace` adds time and error columns.

The rule the book states: "**As always, when designing your chaos experiment, pick the right tool for the job.**" Use BPF to *discover and measure*. Use `strace` to *tamper*, accepting its overhead and baselining with it attached.

6.3.5 Other options

- **SystemTap** — dynamically instruments running Linux systems using a DSL that resembles AWK or C. Probes are compiled and inserted into a running kernel. It overlaps with BPF; there is even a BPF backend, **stapbpf**.
- **Ftrace** — another kernel tracing framework, with static and dynamic events, in the kernel codebase since 2008. It requires a kernel built with ftracer support.

6.4 Practice — Blocking syscalls part 1: **strace**

The **-e inject** interface — the fault-injection API of this chapter:

```
-e inject=set[:error=errno][:retval=value][:signal=sig][:when=expr]
```

Argument	Meaning
fault=<syscall>	Inject a fault into that syscall (shorthand; returns -1 on every call)
error=<error name>	Return this specific errno
retval=<value>	Override the actual return value , including turning a real error into a success
signal=<sig>	Send a particular signal to the traced process
when=<n>	Tamper with only the nth call
when=<n>+	Tamper with the nth and all subsequent calls
when=<n>+<step>	Tamper with the nth and every step occurrences after that

Two worked examples:

```
-e inject=write:error=EACCES:when=2+      # fail every write from the 2nd onward with permission
denied
-e inject=fsync:retval=0:when=1           # force the 1st fsync to report success, whatever it
really did
```

when=<n>+<step> is a **blast-radius dial at the syscall level**. **when=1+2** means "every other call" — enough to prove fragility without guaranteeing total failure. This is the syscall-layer equivalent of "roll the experiment out on a small subset of traffic first."

Experiment Card 6.1 — Breaking the **close** syscall

Field	Content
Goal	Find out whether the black-box System X handles an error from close gracefully.
Relevant theory	Syscalls as the universal application/system boundary; man 2 <syscall> ERRORS as the realism check; strace overhead.
System / setup	legacy_server on :8080 (terminal 1), strace attached (terminal 2), ab generating load (terminal 3).
Hypothesis	"If you make calls to close fail for the System X binary, it will handle it gracefully, and transparently to the end user."
Steady state	ab -c1 -t30 http://127.0.0.1:8080/ with strace already attached : 3042 complete requests, 0 failed, 101.39 RPS .
Observability	ab — failures, latency, throughput; strace 's own summary.
Failure injected	sudo strace -p \$(pidof legacy_server) -e close -e inject=close:error=EIO
Blast radius	One PID, one syscall, a test host. strace detaches cleanly with Ctrl-C.

Field	Content
Observed result	Hypothesis refuted, immediately and totally. <code>ab</code> cannot finish: <code>apr_socket_recv: Connection refused (111)</code> , "Total of 1 requests completed." <code>strace</code> shows <code>close(4) = -1 EIO (Input/output error) (INJECTED)</code> then <code>+++ exited with 1 +++</code> . The server printed <code>legacy_server: error closing socket: Input/output error</code> and died on the very first <code>close</code> .
Why it happened	The application treats any <code>close</code> failure as fatal and exits with code 1, a generic error.
Is this realistic? — the analysis step that makes the experiment worth anything	<code>man 2 close</code> lists four ways <code>close</code> can fail: EBADF (not a valid open fd), EINTR (the call was interrupted by a signal), EIO (an I/O error occurred), ENOSPC/EDQUOT (on NFS , storage-space errors are often reported not against the first <code>write</code> but against a later <code>write</code> , <code>fsync</code> or <code>close</code>).
Lesson learned	EINTR is plainly possible — any process can be interrupted by a signal. So a legacy service that has "run fine for years" is one signal away from dying. The NFS case means a full network filesystem surfaces as a <code>close</code> error. The fragility is not hypothetical.
Next step the book suggests	Inject that <i>specific</i> error code (EINTR) to confirm, then find and fix the handling in the source.
Production considerations	You cannot run this injection in production. <code>strace</code> 's 100× overhead makes that irresponsible. Section 6.5 exists to solve exactly that.

Experiment Card 6.2 — Breaking the `write` syscall

Field	Content
Goal	Test the busiest syscall (292 of 296 calls, 98.34% of syscall time) rather than the rarest.
Relevant theory	Retry logic and its throughput cost; partial vs. total failure injection (<code>when=n+step</code>).
System / setup	Same three terminals.
Hypothesis	"If you make every other call to <code>write</code> fail for the System X binary, it will handle it gracefully, and transparently to the end user."
Steady state	Same command, tracing <code>-e write</code> : 1587 complete requests, 0 failed. Lower throughput than Experiment 1's baseline, because strace prints far more lines — 292 writes per request versus 1 <code>close</code> .
Failure injected	<code>sudo strace -p \$(pidof legacy_server) -C -e inject=write:error=EIO:when=1+2</code> — fail the 1st call and every 2nd call after it.
Observed result	Hypothesis confirmed. 570 complete requests, 0 failed requests , but throughput roughly halved .
Why — visible in the trace	The program retries : <code>write(4, "l", 1) = -1 EIO (INJECTED)</code> immediately followed by <code>write(4, "l", 1) = 1</code> . Every failed write is repeated until it succeeds.
Interpretation	"The program implements some kind of algorithm to account for failed <code>write</code> syscalls, which is good news." The visible cost of that resilience is ~50% throughput. In real life it is unlikely every other write would fail — "even in this nightmarish scenario, System X turns out to not be as easy to break as it was with the <code>close</code> syscall."
Lesson learned	Resilience is uneven within one program. The same binary is bulletproof on its hottest path and fatal on a path it takes once per request. Do not generalise from one syscall to the program. Also: measuring <i>availability</i> and measuring <i>throughput</i> give opposite verdicts here. It "all worked out" precisely because the experiment focused on whether System X keeps working rather than how quickly it responds. Choose which one your SLO cares about before you run the experiment.
Production considerations	Retry-on-every-write is a throughput amplifier under a degraded disk or network — the amplification shape from Chapter 1's DNS retry storm, at a smaller scale.

6.5 Practice — Blocking syscalls part 2: seccomp

The problem `strace` leaves behind is **massive overhead**. `seccomp`, from Chapter 5, filters which syscalls a process may make, and it does so in the kernel.

6.5.1 The easy way — a custom Docker seccomp profile

The profile structure: `"defaultAction": "SCMP_ACT_ERRNO"` blocks everything by default, then a long list of `names` is explicitly allowed with `"action": "SCMP_ACT_ALLOW"`. **To block one syscall, take the default profile and remove it from the allow-list.**

```
cd ~/src/examples/who-you-gonna-call/src
curl https://raw.githubusercontent.com/moby/moby/master/profiles/seccomp/default.json \
  | grep -v getpid > profile.json          # delete getpid from the allow list
```

```
FROM ubuntu:focal-20200423
COPY ./legacy_server /legacy_server
ENTRYPOINT [ "/legacy_server" ]
```

```
make
docker build -t legacy .
docker run --rm -ti --name legacy \
  --security-opt seccomp=../profile.json \
  -p 8080:8080 \
  legacy
# Listening on port 8080, PID: -1      <-- getpid blocked, returns -1
```

Experiment Card 6.3 — Blocking `getpid` with `seccomp`

Field	Content
Goal	Block a syscall without <code>strace</code> 's overhead, and quantify the difference.
Relevant theory	<code>seccomp</code> as a kernel-level BPF syscall filter (Ch. 5); overhead as a constraint on where an experiment may run.
System / setup	<code>legacy_server</code> packaged in a container, started with a modified <code>seccomp</code> profile.
Failure injected	<code>getpid</code> removed from the allow-list → returns <code>-1</code> .
Observability	The server's own startup line; <code>ab -c1 -t30 http://127.0.0.1:8080/</code> .
Observed result	The process starts and prints <code>PID: -1</code> . Under load: 36,107 complete requests, 0 failed, 1203.53 RPS, 0.831 ms mean.
The comparison that matters	Against the <code>strace</code> -attached steady state of ~3042 requests (101 RPS), this is more than 10× faster . <code>seccomp</code> 's injection is essentially free where <code>strace</code> 's was ruinous.
Limitations, stated plainly	Less flexible than <code>strace</code> : you cannot fail every other call , you cannot attach to a running process , you choose the error only via the profile's action, and you need Docker to run it , which further limits the use cases.
Lesson learned	Injection mechanisms trade flexibility against overhead . <code>strace</code> = maximum control, unusable in production. <code>seccomp</code> = minimal overhead, coarse control. Choose by where the experiment must run.

6.5.2 The hard way — `libseccomp`

`libseccomp` is a higher-level, platform-independent library for managing `seccomp` that abstracts the low-level syscalls. **Docker itself uses it** to implement its profiles. Best starting points: its tests, and `seccomp_init(3)`, `seccomp_rule_add(3)`, `seccomp_load(3)`.

```
sudo apt-get install libseccomp-dev      # libseccomp-devel on RHEL/CentOS
```

The four functions:

Function	Does
seccomp_init	Initializes the seccomp state, returns a context
seccomp_rule_add	Adds a filtering rule to the context
seccomp_load	Loads the context into the kernel
seccomp_release	Releases the context and frees memory

```
#include <stdio.h>
#include <unistd.h>
#include <seccomp.h>
#include <errno.h>

int main(void)
{
    scmp_filter_ctx ctx;
    int rc;

    // disable everything by default, by returning EPERM (not allowed)
    ctx = seccomp_init(SCMP_ACT_ERRNO(EPERM));

    rc = seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0); // allow write
    rc = seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(exit), 0);  // and exit – else it
                                                                    segfaults on exit

    rc = seccomp_load(ctx);                                     // load the profile into the kernel

    fprintf(stdout, "getpid() == %d\n", getpid()); // write succeeds; getpid does not

    seccomp_release(ctx);
}
```

```
cd ~/src/examples/who-you-gonna-call
cc seccomp.c -lseccomp -o seccomp-example      # -l links the seccomp library
./seccomp-example
# getpid() == -1
```

Read the result carefully. **The output appearing at all proves write was allowed. Finishing without a crash proves exit was allowed. -1 proves getpid was blocked.** Three assertions from one line of output.

Why exit must be allowed: without it the program segfaults on exit. A syscall filter that forgets the teardown path breaks the program for reasons unrelated to the experiment — a blast-radius mistake specific to this technique.

Further reading given: Jake Edge, "A seccomp Overview"; Michael Kerrisk, "Using seccomp to Limit the Kernel Attack Surface"; Paul Moore, "Syscall Filtering and You".

Comparison — the three syscall tools

	strace	BPF / BCC (syscount)	seccomp (Docker profile / libseccomp)
Purpose	Observe and tamper	Observe and count	Block

	strace	BPF / BCC (syscount)	seccomp (Docker profile / libseccomp)
Detail	Per-call: name, arguments, return value, timing, errors	Counts per syscall only	n/a
Overhead	~100× on a syscall-heavy workload	~6% while tracing the whole host	Negligible
Attach to running process	Yes (-p)	Yes (-p)	No — set at process start
Selective by call number	Yes (when=n, n+, n+step)	n/a	No — all-or-nothing per syscall
Choose the error returned	Yes (error=, retval=, signal=)	n/a	Profile action, for example SCMP_ACT_ERRNO
Needs Docker	No	No	Easy way: yes. libseccomp way: no
Underlying mechanism	ptrace(2)	eBPF	seccomp-BPF in the kernel
Production-safe?	No	Yes	Yes

Theory ↔ Practice connections for Chapter 6

- **Syscalls as the app/system boundary (§6.2) ↔ Experiments 1 and 2:** the theory says every program must cross this boundary; the experiments prove a closed-source binary can be characterised and broken there without its source.
- **man 2 <syscall> ERRORS (§6.2.1) ↔ Experiment 1's analysis (§6.4.1):** the injected **EIO** is only interesting because the man page shows **EINTR** and the NFS **ENOSPC** case are realistic. **Reading the ERRORS section converts a stunt into evidence.**
- **strace overhead (§6.3.3) ↔ seccomp (§6.5):** the measured 100× penalty is why the chapter needs a second mechanism at all. Same pattern as Chapter 5's 1 ms control run: *measure your tool before trusting your result.*
- **BPF here ↔ BPF in Chapter 3:** same technology, different question. Chapter 3 asked "which process is using the resource"; Chapter 6 asks "which syscalls is this process making."
- **seccomp here ↔ seccomp in Chapter 5 (§5.8.1):** Chapter 5 introduced it as a container hardening feature; Chapter 6 repurposes the same feature as a fault injector. **Security controls make excellent chaos tools** — a transferable idea.
- **Retry behaviour in Experiment 2 ↔ the DNS retry storm (Ch. 1) and Ghost/WordPress round trips (Ch. 4–5):** retries buy availability and spend throughput, every time.
- **Black-box method (§6.1) ↔ Chapter 2 (§2.3):** the same commitment to not reading the source, one layer lower.

Key Takeaways — Chapter 6

1. Syscalls are the OS's API and the **universal injection point**. Every program uses them, source or no source.
2. **man 2 syscalls** lists them all. **man 2 <name>** gives signature, description and, critically, the **ERRORS** you can realistically inject. **man 3 <name>** gives the C library's view.
3. **libc wrappers are not pass-throughs.** Roughly 3 of 4 glibc wrappers add auxiliary code; **exit(3)** runs **atexit** handlers before **_exit(2)**. Do not assume a 1:1 mapping.
4. **strace <cmd>** traces from the start. **strace -p <pid>** attaches to a running process but **misses everything before you attached**. **-C** prints a summary; **-S calls** sorts it.
5. **strace costs ~100× on syscall-heavy workloads, even filtering for a syscall the program never makes.** Never attach it in production, and never trust performance numbers taken with it attached. Baseline your steady state *with it attached*.
6. **-e inject=<syscall>:error=<errno>[:retval=] [:signal=] [:when=n|n+|n+step]** is a precise fault injector: which syscall, which error, and which calls.
7. **BPF/BCC syscount-bpfcc** gives the same syscall inventory at **~6% overhead**, host-wide or per PID — but counts only, with no arguments or return values.
8. A single **strace** run can reveal both a bug (**fsync** on a socket returning **EINVAL**) and a pathology (292 one-byte writes) in an undocumented binary.
9. **Resilience is uneven inside one program.** System X dies on the first **close** error but retries every failed **write**. Test each syscall on the path; do not generalise.

10. Resilience has a measurable price: retrying every other write halved throughput while keeping failures at zero.
11. **seccomp blocks syscalls at near-zero cost** — via a modified Docker profile (`--security-opt seccomp=profile.json`) or via libseccomp in ~10 lines of C (`seccomp_init` → `seccomp_rule_add` → `seccomp_load` → `seccomp_release`). Less flexible: no "every other call", no attaching to a running process.
12. Chaos engineering pays off even for a **single process on a single host**. Distribution is not a prerequisite.

Chapter 6 — Chaos in Kubernetes

Kubernetes "solves (or at least makes it easier to solve) a lot of problems that arise when running software across a fleet of machines... But, like everything else, it's not perfect, and **it adds its own complexity to the system — complexity that needs to be understood and managed.**"

The plan for Part 3 (three chapters, one arc)

Chapter	Covers
10 — Chaos in Kubernetes	What Kubernetes is and where it came from; setting up a test cluster; testing a real project's resilience manually
11 — Automating Kubernetes experiments	PowerfulSeal; reimplementing chapter 10's experiments declaratively; ongoing SLO verification; cloud-layer experiments
12 — Under the hood of Kubernetes	The anatomy of a cluster and how to break each component

The book is explicit that this is **not** a Kubernetes tutorial. For that it recommends *Kubernetes in Action* by Marko Luksa.

10.1 Practice — The scenario: the ICANT Project

You inherit the "High-Profile Project" after its technical lead left to breed llamas in the Himalayas.

The documentation, verbatim:

ICANT: International, Crypto-fueled, AI-powered, Next-generation market Tracking
Mission: Build a massively scalable, distributed system for tracking cryptocurrency flows with cutting-edge AI for technologically advanced clients all over the world.
Current status: First we approached the "distributed" part. We're running Kubernetes, so we set up **Goldpinger**, which makes connections between all the nodes to simulate the crypto traffic.
To do: The AI stuff, the crypto stuff, and market stuff.

The entire project is an off-the-shelf network diagnostic tool, deployed and left.

What Goldpinger is

Goldpinger (github.com/bloomberg/goldpinger, written by the book's author) **produces a full graph of Kubernetes cluster connectivity by calling all instances of itself, measuring the times, and producing reports based on that data.** Typically you run one instance per node to detect networking issues across nodes.

How it works (Figure 10.4):

1. Each Goldpinger instance **queries Kubernetes for the addresses of all its peers.**
2. It **periodically makes an HTTP call to every peer**, producing statistics on errors and response times.
3. Every instance does the same, producing a **full connectivity graph.**

The UI colours links green (OK) and red (trouble), and offers a **heatmap** for hotspots of slowness and **metrics** for alerting and dashboards. Downloaded more than a million times from Docker Hub.

Goldpinger is worth knowing for its own sake. It is a **purpose-built observability tool for the exact failure mode Kubernetes hides best** — partial, node-to-node network degradation. That is also why it is the perfect subject: the chapter tests whether the thing that detects network problems actually detects network problems.

10.2 Theory — What Kubernetes is (in 7 minutes)

Kubernetes self-describes as "an open source system for automating deployment, scaling, and management of containerized applications."

The book's derivation, the best short explanation of *why* K8s exists:

1. Run software on **1 computer** → log in and start it. A manual deployment.
2. Run it on **10** → write an SSH script, or use **Ansible** or **Chef**.
3. The program **crashes** — maybe not even a bug, perhaps insufficient disk. You need supervision, so your config management configures a **systemd service** to restart it.
4. **Upgrades** → stop, uninstall, install, start; and the new version has different dependencies.
5. Now **200 machines**, because other people want their software run too, so rollouts take a long time.
6. Each machine has limited CPU, RAM and disk, so you keep **a massive spreadsheet** of what runs where. Onboarding a project means allocating resources in the spreadsheet; a machine going down means finding room elsewhere and migrating.
7. **"Wouldn't it be great if a program could do all this for you?"**

A very brief history

Kubernetes — Greek for *helmsman* or *governor* — was released by **Google in 2015** as a reimplement of its internal scheduler **Borg**. Google donated it to the newly formed **Cloud Native Computing Foundation (CNCF)**, creating a neutral home and attracting investment from other companies.

The strategic note the book adds: it worked. In five years it became **the de facto API for scheduling containers**, and by driving adoption Google "managed to pull people away from investing more into solutions specific to AWS." The CNCF also gained auxiliary projects: **Prometheus**, **containerd**, and many more.

What it does for you

Kubernetes works declaratively, rather than imperatively. You describe the software you want to run, and it **continuously tries to converge the current cluster state into the one you requested**. It also lets you read the current state at any time.

"Conceptually, it's an API for herding cats."

A **cluster** is a set of machines running the Kubernetes components, making their resources — CPU, RAM, disk — available. These are **worker nodes**, and a single cluster can have thousands. Nodes can be **heterogeneous**, with different resource configurations, sometimes usefully so.

Deploying something (Figure 10.3): you tell the cluster the container image, the configuration (env vars, secrets), the resources (CPU, RAM, disk), and how to run it (replica count, placement constraints). You do this through an HTTP request to the Kubernetes API, or through **kubectl**. The part that receives the request, **stores it as the desired state**, returns OK, and then works in the background to converge is the **control plane**.

Worked example from the book: deploy **mysoftware:v1.0**, 1 core and 1 GB RAM each, **2 replicas for high availability**, with a constraint that **the two copies must not run on the same worker node**. One worker going down then cannot take both copies with it.

Pop quiz — what's Kubernetes? "A container orchestrator that can manage thousands of VMs and will **continuously try to converge the current state into the desired state**." It is *not* "a solution to all of your problems" and *not* "software that automatically renders the system running on it immune to failure."

10.3 Practice — Setting up a cluster with Minikube

Minikube is an official part of Kubernetes. It deploys **a single node with single instances of all the control-plane components inside a VM**, and handles conveniences such as accessing processes inside the cluster.

Virtualization options by platform: **Linux** — KVM or VirtualBox, or processes directly on the host; **macOS** — HyperKit, VMware Fusion, Parallels, VirtualBox; **Windows** — Hyper-V or VirtualBox. The book uses VirtualBox as the common denominator.

```
minikube start --driver=virtualbox
# Creating virtualbox VM (CPUs=2, Memory=4000MB, Disk=20000MB) ...
# Preparing Kubernetes v1.18.3 on Docker 19.03.12 ...
```

```
# Done! kubectl is now configured to use "minikube"

kubectl get pods -A          # list pods in ALL namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-66bff467f8-62g9p	1/1	Running	0	5m44s
kube-system	etcd-minikube	1/1	Running	0	5m49s
kube-system	kube-apiserver-minikube	1/1	Running	0	5m49s
kube-system	kube-controller-manager-minikube	1/1	Running	0	5m49s
kube-system	kube-proxy-bwzcf	1/1	Running	0	5m44s
kube-system	kube-scheduler-minikube	1/1	Running	0	5m49s
kube-system	storage-provisioner	1/1	Running	0	5m49s

That list is Chapter 12's entire syllabus. `etcd`, `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, `kube-proxy`, `coredns` — every component you will later learn to break is visible in the first command you run. "This command working at all proves that the control plane works."

Use `minikube stop` and `minikube start` to pause and resume. `kubectl --help`, and `kubectl <command> --help` for per-command options. Tested on Minikube 1.12.3 and Kubernetes 1.18.3.

10.4 Theory — Kubernetes terminology you actually need

Resources are the objects representing Kubernetes' abstractions. Three building blocks:

Resource	Definition (the book's)
Pod	"A collection of containers that are grouped together, run on the same host, and share some system resources (for example, an IP address) ." The unit of software you can schedule. Schedulable directly, but usually managed by a higher-level abstraction.
Deployment	"A blueprint for creating pods, along with extra metadata, such as the number of replicas to run." It manages the life cycle of the pods it creates — on an image update it handles a rollout , deleting old pods and creating new ones one by one to avoid an outage, and it offers rollback if the rollout fails.
Service	"Matches an arbitrary set of pods and provides a single IP address that resolves to the matched pods ." That IP is kept up to date with cluster changes — if a pod goes down it is taken out of the pool.

Permissions — RBAC in three objects

Object	Role
ClusterRole	Defines a role and a set of permissions to execute verbs (create, get, delete, list...) on resources
ServiceAccount	Linked to software running on Kubernetes, so it inherits the permissions granted to the account
ClusterRoleBinding	Links a ServiceAccount to a ClusterRole

```
# Listing 10.1 – goldpinger-rbac.yaml
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: goldpinger-clusterrole
rules:
- apiGroups:
  - ""
  resources:
  - pods          # permission on the resource type "pod"...
  verbs:
  - list          # ...limited to the verb "list"
```

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: goldpinger-serviceaccount
  namespace: default
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: goldpinger-clusterrolebinding
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: goldpinger-clusterrole
subjects:
  - kind: ServiceAccount
    name: goldpinger-serviceaccount
    namespace: default

```

The `---` separators let one YAML file describe multiple resources.

This ClusterRole grants exactly one verb on exactly one resource type. That is a **blast-radius control expressed as configuration**, and it is why Goldpinger crashing on startup is itself a permission test: "Goldpinger crashes if it can't list its peers, which means that the permissioning you set up also works as expected."

Labels and matching — the mechanism behind everything

Labels are simple key-value string pairs attached as metadata to any resource. Kubernetes uses them to **match sets of resources**.

- Pod A: `app=goldpinger, stage=dev`. Pod B: `app=goldpinger, stage=prod`.
- Match `app=goldpinger` → both. Match `stage=dev` → only A. Multiple labels → **logical AND**.

Labels are not just for humans. **Deployments find their pods by selector**, and **Goldpinger finds its peers by asking Kubernetes for all pods with `app=goldpinger`** (Figure 10.7).

```

# Listing 10.2 – goldpinger.yml (abridged, annotated)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: goldpinger
  labels:
    app: goldpinger
spec:
  replicas: 3
  selector:
    matchLabels:
      app: goldpinger          # the deployment's selector...
  template:
    metadata:
      labels:
        app: goldpinger       # ...must match the label on the pods it creates
    spec:
      serviceAccount: "goldpinger-serviceaccount"
      containers:
        - name: goldpinger
          image: "docker.io/bloomberg/goldpinger:v3.0.0"
          env:
            - name: REFRESH_INTERVAL
              value: "2"        # ping every 2 seconds
            - name: HOST
              value: "0.0.0.0"
            - name: PORT

```

```

        value: "8080"
      - name: POD_IP          # inject the real pod IP for clearer output
        valueFrom:
          fieldRef:
            fieldPath: status.podIP
    ports:
      - containerPort: 8080
        name: http
---
apiVersion: v1
kind: Service
metadata:
  name: goldpinger
  labels:
    app: goldpinger
spec:
  type: LoadBalancer
  ports:
    - port: 8080
      name: http
  selector:
    app: goldpinger          # the service matches the same pods

```

DaemonSet vs. Deployment. "In a typical installation, you would like to have **one Goldpinger pod per node**... That can easily be achieved by using a **DaemonSet**. It works a lot like a deployment, but instead of specifying the number of replicas, it **assumes one replica per node**." The book uses a Deployment only because a one-node Minikube would otherwise give a single pod, defeating the demonstration.

Deploying and verifying:

```

kubectl apply -f goldpinger-rbac.yml
kubectl apply -f goldpinger.yml
kubectl get pods                # three pods in Running
kubectl describe svc goldpinger # check the Endpoints field
kubectl get pods -o wide        # pod IPs, for comparison with Endpoints
minikube service goldpinger     # opens the UI at a Minikube-provided URL

```

The verification step worth copying: compare the service's **Endpoints** list against `kubectl get pods -o wide` IPs. "**Any mismatch between the IP addresses matched by the service and the IP addresses of the pods would point to misconfigured labels.**" A service with no or wrong endpoints is one of the most common, and most silent, Kubernetes misconfigurations.

Accessing a service is the one thing Kubernetes does **not** standardise. Every cluster supports services, but how you reach them depends on how the cluster was built. Minikube offers **minikube service**; a managed cloud cluster may need an **ingress**.

10.4.2 Experiment 1 — Kill 50% of pods

Why this cheap experiment is worth running. It "can answer a lot of questions about what happens when one of these instances goes down (simulating a machine going down)":

- Do the other instances **detect** that to begin with?
- If so, **how long** before they detect it?
- How does **Goldpinger configuration** affect all of that?
- **If you had an alert set up, would it get triggered?**

Implementation options, and why the book picks one. You could log into the machine and **kill** the process (Ch. 2), or use the Docker tools (Ch. 5) — "all of the techniques you learned in the previous chapters still apply." But Kubernetes offers **directly deleting pods**, which is the most convenient.

The crucial parameter: `REFRESH_INTERVAL=2` means **the maximum time for an instance to notice a peer down is 2 seconds**. The book flags that this is "pretty aggressive, and in a large cluster would result in a lot of traffic and CPU time," chosen here for demonstration.

```
#!/bin/bash
# Listing 10.3 – kube-thanos.sh
kubectl get pods \
  -l app=goldpinger \           # only pods with this label
  -o name \                     # print names only
  | sort --random-sort \        # shuffle
  | head -n 1 \                 # take one
  | xargs kubectl delete        # delete it
```

Experiment Card 10.1 — Kill a random Goldpinger pod

Field	Content
Goal	Verify that Goldpinger detects a peer going down, and that Kubernetes replaces it.
Relevant theory	Declarative convergence (desired vs. current state); deployments managing pod lifecycle; label selectors.
System / setup	Minikube, 3 Goldpinger pods via a Deployment, a Service, RBAC allowing <code>list pods</code> .
Hypothesis	"If you delete one pod, you should see it marked as failed in the Goldpinger UI , and then be replaced by a new, healthy pod ."
Steady state	All three nodes green in the Goldpinger graph.
Observability	The Goldpinger UI graph, plus <code>kubectl get pods --watch</code> in a second terminal to record pods coming and going.
Failure injected	<code>kubectl delete</code> on one randomly selected pod.
Blast radius	One pod of three (33%), on a test cluster; the deployment restores it automatically.
Observed result	Hypothesis confirmed on both counts. The UI briefly shows an unhealthy node and four nodes — the replacement is created before the old one finishes terminating. <code>--watch</code> shows the killed pod go <code>Running</code> → <code>Terminating</code> while the new one goes <code>Pending</code> → <code>ContainerCreating</code> → <code>Running</code> in about 2 seconds. A later refresh shows three healthy pods.
Why the four nodes	"After the pod is deleted, Kubernetes tries to reconverge to the desired state (three replicas) , so it creates a new pod to replace the one you deleted" — while the old one is still draining.
Lesson learned	Convergence is fast and automatic, and the transient state (4 pods, one unhealthy) is part of normal operation , not an anomaly. Anything that alerts on "an unhealthy peer" must tolerate it.

Experiment 1 discussion — four caveats the author volunteers

1. **The UI is served through a service**, which routes to a **pseudorandom instance every call**. You may be routed to the pod you just killed and see an error, and **every refresh shows reality from a different pod's point of view**. On a large cluster, "you need to make sure you consult all available instances, or at least a reasonable subset" — which is what **Goldpinger's Prometheus metrics** are for.
2. **A GUI-based tool is awkward for this**. "If you see what you expect, that's great. But if you don't, it doesn't necessarily mean the event didn't happen; **you might simply have missed it**." Again: use metrics.
3. **Pods sometimes receive traffic before they are actually up**, because the example skips a **readiness probe** — which exists precisely to prevent a pod from receiving traffic until a condition is met.
4. **The data is up to `REFRESH_INTERVAL` seconds stale**, so a killed pod keeps appearing for that long.

Caveats 1 and 2 together are a general rule: **a dashboard is a sampling instrument, not a record**. For any experiment whose result you must be able to defend, use metrics that are recorded continuously, not a UI you refresh by hand.

10.4.3 Party trick: killing pods in style

- **KubeInvaders** (github.com/lucky-sideburn/KubeInvaders) — a Space Invaders clone where **the aliens are pods in a namespace**. Shooting one deletes it.

- **Kube DOOM** (github.com/storax/kubedoom) — the same idea as a first-person shooter. Run a pod on the host, pass it a kubectl config, connect with a desktop-sharing client. The book's tongue-in-cheek justification: "playing the game is often much quicker than copying and pasting the name of a pod."

10.4.4 Experiment 2 — Introduce network slowness

"**Slowness, my nemesis, we meet again.**" The book's recurring point: "when things go wrong, **actual failure is often easier to debug than situations where things mostly work.** And slowness tends to fall into the latter category."

The options, and the reasoning for the choice. This is the most transferable part of the chapter.

You could use **tc** (Ch. 4) or Pumba (Ch. 5) directly on the host. You could use **kubectl cp** and **kubectl exec** to upload and run **tc** inside a pod, without touching the host. You could **add a second container to the Goldpinger pod** that runs the **tc** commands.

"All of these options are viable but share one downside: **they modify the existing software that's running on your cluster, and so by definition carry risks of messing things up.** A convenient alternative is to **add extra software**, tweaked to implement the failure you care about, but otherwise identical to the original, and introduce the extra software in a way that will integrate with the rest of the system."

This is a genuinely different blast-radius strategy from everything earlier in the book: **do not degrade what is running — add a degraded copy and let the system discover it.**

Design details: Goldpinger's default time-out is **300 ms**, so the injected delay is **250 ms** — "enough to be clearly seen, but not enough to cause a time-out." The **heatmap** provides observability for free.

The plan:

1. **Observability:** the Goldpinger UI's graph and **heatmap**.
2. **Steady state:** all existing instances report healthy.
3. **Hypothesis:** *if you add a new instance with a 250 ms delay, the connectivity graph will show all four instances healthy, and the 250 ms delay will be visible in the heatmap.*
4. **Run.**

Theory + Practice — Toxiproxy

Toxiproxy (github.com/shopify/toxiproxy) is a **TCP-level (OSI Layer 4) proxy**. That matters: you need no understanding of HTTP (Layer 7) to add latency, and **the same tool works identically for Redis, MySQL, PostgreSQL and any other TCP protocol.**

Two pieces: **the proxy server**, exposing an API for what to proxy where and what failure to add; and a **CLI client** that changes the configuration live. Clients exist in many languages, and you can also call the API directly.

Each proxy configuration has a **unique name**, a **host and port to listen on**, and a **destination to proxy to**. Failures attached to it are called **toxics**:

Toxic	Effect
latency	Adds arbitrary latency to the connection, either direction
down	Takes the connection down
bandwidth	Throttles the connection to a desired speed
slow close	Delays the TCP socket from closing for an arbitrary time
timeout	Waits an arbitrary time, then closes the connection
slicer	Slices received data into smaller bits before sending it on

Any combination can be attached to one proxy configuration.

The book notes Toxiproxy's dynamic nature makes it excellent for **unit and integration testing** — "your integration test could start by configuring the proxy to add latency when connecting to a database, and then your test could verify that time-outs are triggered accordingly." That is chaos engineering moved left into the test suite.

The setup (Figures 10.13 and 10.14)

A new pod with two containers: Goldpinger and Toxiproxy. Containers in a pod **share an IP and can talk over localhost**. The trick:

- Goldpinger inside the chaos pod listens on **port 9090** (`PORT=9090`) but **calls its peers on 8080** (`CLIENT_PORT_OVERRIDE=8080`), because by default it calls peers on the port it runs on.
- **Toxiproxy listens on 8080**, the port peers expect, and relays to `localhost:9090`.
- So **peers reach the proxy**, get the delay, and are relayed to Goldpinger. **The chaos Goldpinger calls out normally**, unaffected.
- A second port, **8474**, exposes the Toxiproxy **API**, fronted by its own service selected by the label `chaos=absolutely`.

```
# Listing 10.4 – goldpinger-chaos.yml (annotated)
---
apiVersion: v1
kind: Pod
metadata:
  name: goldpinger-chaos
  labels:
    app: goldpinger          # so peers discover it
    chaos: absolutely        # so the API service can select it
spec:
  serviceAccount: "goldpinger-serviceaccount"
  containers:
  - name: goldpinger
    image: docker.io/bloomberg/goldpinger:v3.0.0
    env:
    - name: REFRESH_INTERVAL
      value: "2"
    - name: HOST
      value: "0.0.0.0"
    - name: PORT
      value: "9090"          # listen here...
    - name: CLIENT_PORT_OVERRIDE
      value: "8080"          # ...but call peers on the normal port
    - name: POD_IP
      valueFrom:
        fieldRef:
          fieldPath: status.podIP
  - name: toxiproxy
    image: docker.io/shopify/toxiproxy:2.1.4
    ports:
    - containerPort: 8474
      name: toxiproxy-api
    - containerPort: 8080
      name: goldpinger
---
apiVersion: v1
kind: Service
metadata:
  name: goldpinger-chaos
spec:
  type: LoadBalancer
  ports:
  - port: 8474
    name: toxiproxy-api
  selector:
    chaos: absolutely        # routes only to the chaos pod's API
```

Running it

```
minikube service goldpinger          # UI; check the Heatmap first
kubectl apply -f goldpinger-chaos.yml
# -> the new node appears in the graph but is RED: no traffic is proxied yet

TOXIPROXY_URL=$(minikube service --url goldpinger-chaos) # --url prints only the URL
```

```

toxicproxy-cli -h $TOXIPROXY_URL list # -h is HOST, not help
# "no proxies"

toxicproxy-cli -h $TOXIPROXY_URL create chaos \
  -l 0.0.0.0:8080 \      # listen where peers expect
  -u localhost:9090      # relay to the real Goldpinger
# -> refresh the UI: all four nodes green

toxicproxy-cli -h $TOXIPROXY_URL toxic add \
  --type latency \
  --a latency=250 \      # 250 ms
  --upstream \          # toward the Goldpinger instance
  chaos

toxicproxy-cli -h $TOXIPROXY_URL inspect chaos
# Upstream toxics:
# latency_upstream: type=latency stream=upstream toxicity=1.00 attributes=[ jitter=0
  latency=250 ]

```

Reading the heatmap: columns are **sources (from)**, rows are **destinations (to)**, and colour encodes request time. The legend maps numbers to pods. In steady state all squares are the same shade, everything under 2 ms, since all pods share a host. You can tweak the thresholds and click Refresh.

Experiment Card 10.2 — 250 ms latency via a proxied extra replica

Field	Content
Goal	Verify that Goldpinger — a tool whose <i>purpose</i> is detecting network slowness — actually detects and reports it.
Relevant theory	Blast radius via added capacity rather than degraded capacity ; TCP-level proxying; multi-container pods sharing localhost; timeouts vs. latency thresholds.
System / setup	3 normal Goldpinger pods + 1 chaos pod (Goldpinger on 9090 + Toxicproxy on 8080/8474) + a service selecting chaos=absolutely .
Hypothesis	"If you add a new instance that has a 250 ms delay, the connectivity graph will show all four instances healthy , and the 250 ms delay will be visible in the heatmap ."
Steady state	All three instances healthy; heatmap uniformly under ~2 ms.
Observability	Goldpinger graph (health) and heatmap (latency). Two different signals, deliberately.
Failure injected	Toxicproxy latency toxic, 250 ms, upstream, jitter=0 .
Blast radius	One added pod = 25% of running software affected , existing pods unmodified . Cleanup is kubectl delete -f goldpinger-chaos.yml .
Observed result	Hypothesis confirmed . The graph shows all four healthy (250 ms < the 300 ms timeout), while the heatmap row for goldpinger-chaos turns red — every peer sees slowness reaching it.
Why the two signals disagree	Health is a binary derived from a timeout; latency is a continuous measure. A system can be 100% "healthy" and badly degraded at the same time . That gap is the single most important operational lesson in the chapter.
Lesson learned	Before trusting a monitoring tool, inject the condition it claims to detect and check that it reports it — and check <i>which</i> of its views reports it.
Production considerations	"Does that mean you could do that in production? ... It depends . In this example, if you wanted to verify the robustness of some alerting that relies on metrics from Goldpinger to trigger, this could be a good way to do it. But the extra software could also affect the existing instances in a more profound way, making it riskier."
Known imperfection	The UI service routes pseudorandomly, so sometimes you are talking to the delayed instance yourself . Invisible at 250 ms; a problem if you test a larger delay.

Theory ↔ Practice connections for Chapter 10

- **Declarative convergence (§10.2.2) ↔ Experiment 1:** deleting a pod is not "breaking" the system. It is **giving the control plane something to converge**. The four-pod transient is convergence made visible.
- **Labels and selectors (§10.4.1) ↔ both experiments:** `-l app=goldpinger` targets the kill; `chaos=absolutely` targets the proxy API; `app=goldpinger` on the chaos pod is what makes peers discover it. **Labels are simultaneously the discovery mechanism, the targeting mechanism and the blast-radius control.**
- **tc/netem (Ch. 4) and Pumba (Ch. 5) ↔ Toxiproxy (§10.4.4):** the third latency-injection tool in the book, chosen because it **adds** a component instead of modifying one. Compare the three when picking for your own system.
- **Blast radius (Ch. 2) ↔ "add extra software" (§10.4.4):** a genuinely new technique — scale up, then degrade only the new capacity.
- **Averages hide distributions (Ch. 4) ↔ health vs. heatmap (§10.4.4):** the same lesson in a new form. A binary health check hides everything below its threshold.
- **Prometheus (Ch. 3) ↔ the discussion caveats (§10.4.2):** the UI is a sampling instrument; the metrics are the record. Chapter 11 builds SLO verification on exactly those metrics.
- **The control-plane pod list (§10.3.2) ↔ Chapter 12:** every component you will break later is already on screen.

Key Takeaways — Chapter 10

1. Kubernetes automates what a growing fleet forces you to do by hand: deployment, supervision and restart, rollouts and rollbacks, and resource-aware placement. It is **declarative** — you state desired state and the control plane converges to it.
 2. Kubernetes came from Google's **Borg**, was open-sourced in 2015 and donated to the **CNCF**, and became the de facto container-scheduling API.
 3. **Pod** = co-located containers sharing an IP. **Deployment** = a blueprint plus lifecycle management for pods. **Service** = a stable IP resolving to a matched set of pods.
 4. **RBAC** = ClusterRole (verbs on resources) + ServiceAccount (identity for workloads) + ClusterRoleBinding (the link). Grant exactly the verbs needed.
 5. **Labels drive everything:** deployment-to-pod ownership, service-to-pod routing, and application-level peer discovery. Multiple labels AND together.
 6. `kubectl apply -f`, `kubectl get pods [-o wide|-o name|--watch|-A|-l <label>]`, `kubectl describe svc`, `kubectl delete`, and `minikube service [--url]` are the working set for this chapter.
 7. **Compare a service's Endpoints against the pods' actual IPs.** A mismatch means misconfigured labels — a silent, common failure.
 8. Killing a pod is a one-liner (`kubectl get pods -l ... -o name | sort --random-sort | head -n1 | xargs kubectl delete`), and Kubernetes replaces it in seconds. Expect a transient state with **more pods than replicas**.
 9. **Alternative to degrading running software: deploy a degraded copy.** Same labels so it joins the system, a proxy in front of it so only it is impaired, existing pods untouched, `kubectl delete -f` to clean up.
 10. **Toxiproxy** is a Layer-4 proxy with named configurations and composable **toxics** (latency, down, bandwidth, slow close, timeout, slicer), reconfigurable at runtime through an HTTP API or `toxiproxy-cli`. It works for any TCP protocol.
 11. In a multi-container pod, containers share localhost, which is what lets a proxy take over an application's port (`PORT=9090, CLIENT_PORT_OVERRIDE=8080`).
 12. **250 ms of latency under a 300 ms timeout produced a perfectly "healthy" graph and a red heatmap.** Binary health checks hide degradation up to their threshold. Measure latency separately.
 13. A UI you refresh by hand is a sampling instrument. For defensible results, and for alerting, use the metrics.
-

Chapter 7 — Automating Kubernetes Experiments

"In the previous chapter, you set up experiments manually to build an understanding of how to implement the experiment. But now I want to show you how much more quickly you can go when using the right tools."

11.1 Theory — Automation, and PowerfulSeal

Why automate: "a lot of automation or reducing toil can be seen as a manifestation of being too lazy to do manual labor. Automation also **reduces operator errors and improves speed and accuracy**."

Where to find tools: the **Awesome Chaos Engineering** list (github.com/dastergon/awesome-chaos-engineering). For Kubernetes the book recommends **PowerfulSeal**, written by the author, with **Chaos Toolkit** and **Litmus** as other good options.

What PowerfulSeal is

Feature	Purpose
Interactive mode	Understand how software on your cluster works, and manually break it
Cloud provider integration	Take VMs up and down
Label-based pod killing	Automatically kill pods marked with special labels
Autonomous mode	Sophisticated scenarios described in YAML — the focus of this chapter

Autonomous mode reads a YAML **policy file** containing any number of **scenarios**, each listing the **steps** needed to implement, validate and clean up after an experiment.

```
# Listing 11.1 – the minimal policy
scenarios:
- name: Just check that my service responds
  steps:
  - probeHTTP:                # conduct an HTTP probe
    target:
      service:
        name: my-service      # target this service...
        namespace: myapp
        endpoint: /healthz    # ...on this endpoint; fail the scenario if it doesn't work
```

How it runs (Figure 11.1). Typically either **from your local machine**, the one you already use for **kubectl**, which suits development; or **as a deployment on the cluster**, which suits ongoing, continuous experiments. It needs permission to talk to Kubernetes, through a **ServiceAccount** as Goldpinger did, or a **kubectl config file**. To manipulate VMs it also needs cloud-provider credentials. Then it walks the policy, killing pods and taking VMs down as the scenarios require.

```
python3 --version                # needs Python 3.7+
python3 -m virtualenv env
source env/bin/activate
pip install powerfulseal
powerfulseal --version
powerfulseal --help
```

It is also distributed as the Docker image **powerfulseal/powerfulseal**.

Pop quiz — what does PowerfulSeal do? "Allows you to write a YAML file to describe how to run and validate chaos experiments." It does **not** guess what chaos you need by looking at your clusters.

11.1.3 Experiment 1b — Killing 50% of pods, declaratively

Same plan as Chapter 10's Experiment 1. The implementation "translates one-to-one to a built-in feature."

The podAction model — three steps, and the mental model for all of PowerfulSeal:

1. **Match** some pods, for example by labels.
2. **Filter** the pods — various filters, such as taking a 50% random subset.
3. **Apply an action** on the survivors, such as killing them.

Match → filter → act is blast-radius selection expressed as configuration. Compare with Chapter 2's hand-written **grep | awk | kill**, where the same three stages existed but were implicit and easy to get catastrophically wrong.

```
# Listing 11.2 – experiment1b.yml
config:
  runStrategy:
    runs: 1                # run once, then exit
```

```

scenarios:
- name: Kill 50% of Goldpinger nodes
  steps:
  - podAction:
      matches:
      - labels:
          selector: app=goldpinger
          namespace: default
      filters:
      - randomSample:
          ratio: 0.5          # take 50% of the matched pods
      actions:
      - kill:
          force: true

```

```
powerfulseal autonomous --policy-file experiment1b.yml
```

On Minikube the kubectl config at `~/.kube/config` is picked up automatically, so `--policy-file` is the only flag needed. The log is itself the audit trail:

```

Matched 3 pods for selector app=goldpinger in namespace default
Initial set length: 3
Filtered set length: 1
Pod killed: [pod #0 name=goldpinger-c86c78448-8lfqd namespace=default ... state=Running ...]
Scenario finished

```

`kubectl get pods --watch` confirms the kill and the replacement, exactly as in Chapter 10.

"Matched 3 / Initial 3 / Filtered 1" is the line to read before every run. It tells you the blast radius *before* the action line tells you what happened. In a real cluster, a mis-typed selector shows up here as a frightening number.

11.1.4 Experiment 2b — Network slowness, declaratively

Same plan as Chapter 10's Experiment 2, but using PowerfulSeal's **clone** feature.

How clone works: you point PowerfulSeal at an existing **source deployment**, which it copies at runtime — "to make sure that you don't break the existing running software, and instead add an extra instance, just as you did before." Then you specify **mutations**.

The toxiproxy mutation does almost exactly what you did by hand in Chapter 10:

- Adds a **Toxiproxy container** to the deployment.
- Configures Toxiproxy to **create a proxy configuration for each port specified on the deployment**.
- **Automatically redirects traffic incoming to each port to its corresponding proxy port.**
- Configures any **toxics** requested.

"The only real difference between what you did before and what PowerfulSeal does is the **automatic redirection of ports**, which means that **you don't need to change any port configuration in the deployment**." That is the `PORT=9090 / CLIENT_PORT_OVERRIDE=8080` gymnastics from Chapter 10, eliminated.

```

# Listing 11.3 - experiment2b.yml
config:
  runStrategy:
    runs: 1
scenarios:
- name: Toxiproxy latency
  steps:
  - clone:
      source:

```

```
deployment:
  name: goldpinger
  namespace: default
replicas: 2 # two extra replicas -> 2 of 5 total = 40% affected
mutations:
  - toxiproxy:
      toxics:
        - targetProxy: "8080"
          toxicType: latency
          toxicAttributes:
            - name: latency
              value: 250
  - wait:
      seconds: 120 # time to look around before cleanup
```

```
powerfulseal autonomous --policy-file experiment2b.yml
```

```
Clone deployment created successfully
Sleeping for 120 seconds
Scenario finished
Cleanup started (1 items)
Clone deployment deleted successfully: goldpinger-chaos in default
Cleanup done
```

Experiment Card 11.1 — Cloned, latency-injected replicas (Experiment 2b)

Field	Content
Goal	Reproduce Chapter 10's latency experiment declaratively, at a larger blast radius, with automatic cleanup.
Relevant theory	Match → filter → act; "add degraded capacity instead of degrading existing capacity"; Toxiproxy toxics.
System / setup	3 original Goldpinger pods + 2 cloned pods with Toxiproxy in front → 5 total.
Hypothesis	Same as 10.2: the graph shows all instances healthy; the 250 ms delay is visible in the heatmap.
Steady state	Three healthy Goldpinger nodes, uniform heatmap.
Failure injected	250 ms latency toxic on port 8080 in the cloned deployment.
Blast radius	2 of 5 replicas = 40% of traffic, none of it on the original pods; cleanup is automatic at scenario end.
Observed result	Graph shows five healthy nodes; heatmap shows the two chaos-named pods as slow. Hypothesis confirmed.
The detail worth noticing	"the connections they are making to themselves are unaffected... PowerfulSeal doesn't inject itself into communications on localhost." The diagonal squares of the heatmap stay clean.
Lesson learned	A proxy sitting in front of a service impairs inbound traffic only. A pod talking to itself over localhost bypasses it. Know which direction and which paths your injector actually covers, or you will misread the result.
Production considerations	runs: 1 plus an explicit wait plus automatic cleanup is a safer shape than a manual kubectl apply you must remember to undo.

11.2 Theory — Ongoing testing and service-level objectives

The epistemological argument, and the reason this section exists. "Like everything in science, a single counterexample is enough to prove a hypothesis wrong, but absence of such a counterexample doesn't prove anything."

When you run an experiment once and it passes, "what you've actually proved is that **the system behaved the expected way during the experiment**. But does that guarantee it will work the same way in different conditions (peak traffic, different usage patterns, different data)? Typically, the larger and more complex the system, the harder it is to answer that question."

The fix: "instead of running an experiment once, you can **run it continuously to detect any anomalies, experimenting every time on a system in a different state** and during the kind of failure you expect to see."

The worked business case. You run a PaaS similar to AWS Lambda: a client requests that some code be built, deployed and run. Clients care about deployment speed, so the **SLA** excludes build time and promises **1 minute to deploy**. You measure once, it takes 30 seconds, champagne — "Or does it?"

The engineering response: **run the experiment continuously**, alerting on an **internal SLO more aggressive than the contractual SLA**, "so that you can get alerted when you get close to trouble."

Why pod-start time is genuinely unpredictable

Kubernetes pod phases:

Phase	Meaning
pending	Accepted by Kubernetes but not set up yet
running	Set up, and at least one container is still running
succeeded	All containers terminated in success
failed	All containers terminated, at least one in failure
unknown	State unknown — typically the node stopped reporting to Kubernetes

The happy path is pending → running. Before that happens, several variable-duration things must occur:

- **Image download.** Unless already present, images must be downloaded, possibly from a remote location. Duration varies with image size and how busy the source is, and **like everything on the network it is prone to failure and may need retries**.
- **Preparing dependencies** — potentially large volumes, configuration files, and so on.
- **Actually running the containers** — this varies with how busy the host is.

On a not-so-happy path a pod can go **pending → failed → running**. "The point is that **you can't easily predict how long it's going to take...** So the next best thing you can do is to **continuously test it and alert when it gets too close to the threshold you care about**."

Experiment 3 — Verify pods are ready within n seconds

1. **Observability:** read PowerfulSeal output, and metrics.
2. **Steady state:** N/A.
3. **Hypothesis:** *when you schedule a new pod and a service, it becomes available for HTTP calls within 30 seconds.*
4. **Run — indefinitely.**

The loop (Figure 11.4): **create a pod and a service → wait 30 seconds → call the service to verify it is available, fail if not → remove the pod and service → repeat.**

Three PowerfulSeal features used:

Feature	Behaviour
kubectl step	Behaves exactly like kubectl apply / kubectl delete on the attached YAML. autoDelete: true cleans up at the end of the scenario.
wait	Waits the time you expect to be sufficient
probeHTTP	Makes an HTTP request and detects whether it works. "Fairly flexible; it supports calling services or arbitrary URLs, using proxies and more."

```
# Listing 11.4 — experiment3.yml (abridged)
config:
```

```

runStrategy:
  minSecondsBetweenRuns: 5      # run CONTINUOUSLY (the default), 5-10 s between runs
  maxSecondsBetweenRuns: 10
scenarios:
- name: Verify pod start SLO
  steps:
  - kubectl:
    autoDelete: true
    action: apply
    payload: |
      ---
      apiVersion: v1
      kind: Pod
      metadata:
        name: slo-test
        labels:
          app: slo-test
      spec:
        containers:
        - name: goldpinger
          image: docker.io/bloomberg/goldpinger:v3.0.0
          env:
            - name: HOST
              value: "0.0.0.0"
            - name: PORT
              value: "8080"
          ports:
            - containerPort: 8080
              name: goldpinger
      ---
      apiVersion: v1
      kind: Service
      metadata:
        name: slo-test
      spec:
        type: LoadBalancer
        ports:
          - port: 8080
            name: goldpinger
        selector:
          app: slo-test
  - wait:
    seconds: 30                  # the SLO threshold
  - probeHTTP:
    target:
      service:
        name: slo-test
        namespace: default
        port: 8080
    endpoint: /healthz

```

Minikube caveat: service IPs used by `probeHTTP` must be reachable from your machine, so run `minikube tunnel` in another terminal. It asks for a sudo password, prints a status showing the route it added (`route: 10.96.0.0/12 -> 192.168.99.100`), and undoes the change when stopped.

```

kubectl get pods --watch          # terminal 1
powerfulseal autonomous --policy-file experiment3.yml  # terminal 2, Ctrl-C to stop

```

```

Starting scenario 'Verify pod start SLO' (3 steps)
pod/slo-test created  service/slo-test created
Return code: 0
Sleeping for 30 seconds

```

```
Making a call: http://10.101.237.29:8080/healthz, get, {}, 1000, 200, , , True
Response: {"OK":true,"duration-ns":260,"generated-at":"2020-08-26T08:52:53.572Z"}
Scenario finished
Cleanup started (1 items)
pod "slo-test" deleted service "slo-test" deleted
Sleeping for 8 seconds
```

Experiment Card 11.2 — Continuous SLO verification of pod-start time

Field	Content
Goal	Continuously verify that a newly scheduled pod and service is serving HTTP within 30 seconds — an internal SLO tighter than a 1-minute contractual SLA.
Relevant theory	One passing run proves nothing about other conditions; SLI → SLO → SLA (Ch. 1); unpredictable pod-start latency (image pull, dependencies, host load).
System / setup	A minimal Goldpinger pod + LoadBalancer service, created and destroyed each cycle.
Hypothesis	"When you schedule a new pod and a service, it becomes available for HTTP calls within 30 seconds ."
Steady state	N/A , deliberately. This scenario measures a <i>transition</i> , not a resting state.
Observability	PowerfulSeal's own output and metrics; cross-checked against <code>kubectl get pods --watch</code> .
Failure injected	None . "Normally, you would add some type of failure, and test that the system withstands that. But right now, I just want to illustrate the idea of ongoing experiments."
Blast radius	One throwaway pod and service per cycle, auto-deleted.
Observed result	Pod goes <code>Pending</code> → <code>ContainerCreating</code> → <code>Running</code> in ~1 s; <code>/healthz</code> responds; scenario passes; cleanup verified in the <code>--watch</code> output.
Lesson learned	"With about 50 lines of verbose YAML , you can describe an ongoing experiment and detect when starting a pod takes longer than 30 seconds."
Making it realistic	Use an image that resembles what the platform actually runs ; Goldpinger's image is small. Run multiple scenarios for multiple image types . To always test the worst case, a cold image pull, set <code>imagePullPolicy: Always</code> in the pod template.
Other SLOs to verify the same way	Pod healing : if you kill a pod, how long until it is rescheduled and ready? Scaling : if you scale a deployment, how long until the new pods are available?
Production considerations	This is the shape of an experiment that lives permanently in your cluster, wired to your alerting — the practical realisation of Chapter 1's "work backward from the business goal to an SLO you can continuously test against."

Pop quiz — when does it make sense to run chaos experiments continuously? *All of the above*: to detect when an SLO is not satisfied; when an absence of problems doesn't prove the system works well; when you want an element of randomness; and **when you want to make sure there are no regressions in the new version of the system**.

11.3 Theory — The cloud layer

"In Kubernetes, a lot of the time you can stop thinking about the machines and data centers that your clusters are built on. But that doesn't mean that **they stop existing**. They are very much still there, and you still need to obey the rules of physics governing their behavior. **And with a bigger scale come bigger problems**."

The napkin math you should be able to reproduce

Mean time to failure (MTTF) is the average time hardware runs without failing, established empirically from historical data.

- Servers with **MTTF = 5 years**.
- Chance of any given server failing on a given day $\approx$ **1 in 1826** (5×365 plus a leap day) $\approx$ **0.05%**.
- Treating failures as independent: **20 servers** → **~1% daily chance** that one fails. **200 servers** → **~10%**.

- At **thousands of servers**, failure is a **daily occurrence**.

And "if that failed server is running multiple VMs that you use as Kubernetes nodes, you're going to end up with **a chunk of your cluster down**."

The book flags this as a simplification — a serious calculation needs other factors — but "a good enough estimate for our needs."

Therefore, the four hardware-failure shapes to test:

- Single machines going down and back up
- **Groups** of machines going down and back up
- **Entire regions, datacenters or zones** going down and back up
- **Network partitions** that make it look like other machines are unavailable

Regions and availability zones (Figure 11.5)

Concept	What it is	What it protects against
Region	Different physical locations , often far apart, plugged into separate utility providers (internet, electricity, water, cooling)	Something dramatic in one location — storm, earthquake, flood. Limits the blast radius to a single region.
Availability zone	Groups within a region separated by redundant components (power supply, internet provider, networking hardware) — for example two racks on separate power and separate internet	Failure of those components. Further limits the blast radius within a region.

In the book's diagram: a West Coast region with zones W1 and W2, two machines each, and an East Coast region with E1 and E2. **A region failure wipes out four machines; an availability-zone failure wipes out two.**

Affinity and anti-affinity: marking two machines with the same **affinity group** means they **should** (soft) or **must** (hard) run within the same partition. **Anti-affinity** is the opposite — items in the same group **shouldn't** or **mustn't** share a partition. Spreading across regions makes you immune to a region outage; spreading across availability zones, to a zone outage.

Cloud providers also **express their SLOs in terms of regions and zones** — "for example, promising to keep each region up 95% of the time, but at least one region up 99.99% of the time."

11.3.2 Experiment 4 — Taking VMs down

The Kubernetes connection: "most Kubernetes providers set labels for each node that can be used for anti-affinity. Kubernetes also allows you to set your own criteria of anti-affinity and will try to schedule pods in a way that respects them."

The questions a VM-level experiment answers, which pod-killing does not:

- Will the loss be **detected as quickly** as a pod loss?
- Will the instance be **rescheduled somewhere else**?
- **How long will it take to recover after the VM is brought back up?**

Why use a tool rather than a script: clicking Shutdown in a GUI or scripting the cloud CLI "would absolutely do it. The only problem... is that they are **cloud-provider specific**, and **you might end up reinventing the wheel each time**." PowerfulSeal supports **OpenStack, AWS, Microsoft Azure and GCP**, and "adding a new driver involves implementing a single class with a handful of methods." Drivers are configured the same way as their respective CLIs (`powerfulseal autonomous --help`).

Two shapes of VM experiment:

(a) nodeAction — target nodes directly. It matches on **names, IP addresses, availability zones, groups, and state**.

```
# Listing 11.5 – experiment4a.yml
config:
  runStrategy:
    runs: 1
scenarios:
- name: Test load-balancing on master nodes
  steps:
```

```

- nodeAction:
  matches:
    - property:
        name: "az"
        value: "WEST.*"          # any availability zone starting with WEST
  filters:
    - randomSample:
        size: 1                  # exactly one VM
  actions:
    - stop:
        autoRestart: true       # restarted at the end of the scenario
- probeHTTP:
  target:
    url: "http://load-balancer.example.com" # confirm the system keeps working

```

(b) podAction + stopHost — target the machine *underneath* a pod.

```

# Listing 11.6 — experiment4b.yml
scenarios:
- name: Stop that host!
  steps:
  - podAction:
      matches:
        - namespace: mynamespace
      filters:
        - randomSample:
            size: 1
      actions:
        - stopHost:
            autoRestart: true

```

Both policies work with **any supported cloud provider** unchanged.

Experiment Card 11.3 — Taking a VM down in one availability zone

Field	Content
Goal	Verify the application survives the loss of a machine — the failure mode MTTF math says you will meet routinely at scale.
Relevant theory	MTTF and the arithmetic of scale; regions vs. availability zones; affinity and anti-affinity; "this is what the original Chaos Monkey did."
System / setup	A real cloud cluster. Minikube's single VM cannot demonstrate this.
Hypothesis	Stopping one VM in a WEST-* availability zone leaves the load-balanced service responding.
Observability	probeHTTP against the load balancer's URL; and, for a Goldpinger-style app, the peers' view of detection and recovery time.
Failure injected	stop on a VM, or stopHost on the VM running a selected pod.
Blast radius	randomSample: size: 1 , scoped by availability-zone regex, with autoRestart: true so the machine comes back at the end of the scenario.
Interpretation	A pod kill tests the scheduler. A VM stop tests the scheduler, the node-failure detection path, the rescheduling path, and your anti-affinity configuration at once.
Lesson learned	Test single machines, groups, whole zones and regions, and network partitions — the four shapes above. The cloud API makes all four scriptable.
Production considerations	autoRestart is your teardown; without it a failed scenario leaves capacity down. Scope by availability zone so a mistake cannot cross a region boundary.

Pop quiz — what can PowerfulSeal *not* do for you? Fill in existential discomfort about better versions of you in other universes. It **can** kill pods, take VMs up and down, clone a deployment and inject latency into the copy, and verify services with HTTP requests.

Theory ↔ Practice connections for Chapter 11

- **Chapter 10's manual experiments ↔ 11.1.3 and 11.1.4:** the same two experiments, now ~15 lines of YAML each with automatic cleanup. The book's insistence on doing them by hand first is the same "know the gearbox" argument as Chapter 7.
- **Blast radius (Ch. 2) ↔ match → filter → act:** PowerfulSeal makes the three stages of Chapter 2's **grep**-and-kill explicit, logged and reviewable. **randomSample** with **ratio** or **size** is the dial.
- **SLI / SLO / SLA (Ch. 1 §1.2.1) ↔ §11.2:** Chapter 1 said you can "work backward from the business goals to an engineering-friendly defined SLO that you can continuously test against by using chaos engineering." Experiment 3 is that sentence executed.
- **Steady state (Ch. 1) ↔ "Steady state: N/A" in Experiment 3:** a legitimate variant. When the property under test is a *transition time*, there is no resting state to characterise, and the hypothesis carries the whole load.
- **Toxiproxy by hand (Ch. 10) ↔ the **toxiproxy** mutation (§11.1.4):** identical mechanism, minus the port gymnastics. Knowing the manual version is what lets you interpret the localhost exception.
- **Prometheus and Grafana (Ch. 3) ↔ continuous SLO verification:** an experiment that runs forever needs metrics and alerting, not a UI.
- **Cloud-layer experiments (§11.3) ↔ Chaos Monkey (Ch. 1):** "just like the original Chaos Monkey did" — the book closes the loop on the tool it spent Chapter 1 distancing itself from.
- **Availability zones (§11.3.1) ↔ Chapter 12:** understanding *where* a node sits is what makes control-plane experiments in the next chapter meaningful.

Key Takeaways — Chapter 11

1. High-level tools make sophisticated scenarios cheap, **but learn the underlying technology first**, or you cannot interpret what the tool did.
2. **PowerfulSeal's autonomous mode** reads a YAML **policy file** of **scenarios** made of **steps**. Run it locally for development, or as a cluster deployment for continuous experiments.
3. **podAction = match → filter → act**. **matches** (labels, namespace), **filters** (**randomSample** by **ratio** or **size**), **actions** (**kill**, **stopHost**). The log prints matched, initial and filtered counts — **read them as your blast radius**.
4. **clone plus the **toxiproxy** mutation** reproduces Chapter 10's add-a-degraded-replica technique in a few lines. It adds the proxy container, **redirects ports automatically**, applies toxics, and **deletes the clone at scenario end**.
5. **A proxy in front of a pod does not affect that pod's localhost traffic**. Know which paths your injector actually covers.
6. **One passing run proves only that the system behaved that way, that once**. Absence of a counterexample proves nothing, which is the argument for **continuous** experiments.
7. **Pod-start time is inherently variable**: image download (remote, retryable), dependency preparation, and host load. Pod phases: pending, running, succeeded, failed, unknown.
8. A continuous SLO experiment is **kubectl (with **autoDelete**) → wait → probeHTTP**, looped with **minSecondsBetweenRuns** and **maxSecondsBetweenRuns**. Alert on an **internal SLO tighter than the contractual SLA**.
9. Make it realistic: use representative images, several of them, and **imagePullPolicy: Always** to test the cold-pull worst case. Extend the same pattern to pod-healing and scaling SLOs.
10. **MTTF arithmetic**: a 5-year MTTF gives roughly a 0.05% daily failure chance per server — ~1% at 20 servers, ~10% at 200, daily at thousands. **Hardware failure is a scheduled event, not a surprise**.
11. **Regions** are geographically and utility-independent. **Availability zones** separate redundant components within a region. Use **affinity and anti-affinity** to spread. Providers state SLOs in these terms.
12. Test four failure shapes: single machines, groups, whole zones and regions, and network partitions that merely *look* like unavailability.
13. **nodeAction** targets VMs by name, IP, availability zone, group or state. **podAction plus stopHost** targets the machine under a pod. **autoRestart: true is the teardown**. The same policy runs against OpenStack, AWS, Azure or GCP.

Chapter 8 — Under the Hood of Kubernetes

"To understand its weak points, you need to know how it works." This chapter is a component-by-component anatomy. Each section ends in concrete experiment ideas rather than worked labs.

The book pins its version: "I describe things as they stand for **Kubernetes v1.18.3**. Kubernetes is a fast-moving target... **the only constant is change in Kubernetes Land**." The component responsibilities described here have been stable. Specific timeouts and defaults are what drift.

Why this matters even on managed Kubernetes: the complexity "can be somewhat alleviated by using managed Kubernetes clusters so that most day-to-day management is someone else's problem, **you're never fully insulated from the consequences**."

12.1.1 Theory — The control plane

The brain of the cluster:

Component	Responsibility
etcd	The database storing all information about the cluster
kube-apiserver	The server through which all interactions with the cluster are done, and which stores information in etcd
kube-controller-manager	Implements an infinite loop reading the current state and modifying it to converge on the desired state
kube-scheduler	Detects newly created pods and assigns them to nodes , honouring affinity, resource requirements and policies
kube-cloud-manager (optional)	Controls cloud-specific resources such as VMs and routing

What actually happens on **kubectl apply** (Figure 12.1):

1. Your request reaches **kube-apiserver**, which **validates** it and **stores** the new or modified resource in **etcd** — here, a new *deployment*.
2. **kube-controller-manager** is notified of the new deployment, reads current state, and eventually **creates new pods through another call to kube-apiserver**.
3. kube-apiserver stores those in etcd. **kube-scheduler** is notified about the new pods, **picks the best node**, assigns it, and updates them back through kube-apiserver.

The architectural insight to carry away: "kube-apiserver is at the center of it all, and all the logic is implemented in **asynchronous, eventually consistent loops in loosely connected components**."

Consequences for chaos engineering: (a) **every component's failure mode is "stops converging," not "crashes the cluster"**; (b) **kube-apiserver is the shared dependency of everything**, so degrading it degrades everything; (c) because the loops are eventually consistent, **failures appear as latency and staleness, not as errors** — the hardest kind to notice.

etcd — the memory of the cluster

Origin story: written at CoreOS, which Red Hat bought and IBM later acquired, reportedly as an exercise in implementing the distributed consensus algorithm **Raft**.

Why consensus at all? Four words: availability and fault tolerance.

- **Fault tolerance:** with a single copy of the data, when it's gone, it's gone. Recall Chapter 11's MTTF math: at 20 servers you are playing Russian roulette at 0.05% a day.
- **Availability:** with a single server, when it's down, your system is down.
- You cannot get either without **multiple copies**, and multiple copies must **agree on a version of reality**. That is consensus.

The book's analogy for Raft: consensus is agreeing on a film on Netflix. Alone, no one argues. With a partner, neither of you can gather a majority, so you get power moves and barter. **Add a third person and whoever convinces them gains a majority and wins.**

That is "pretty much exactly how Raft (and by extension, etcd) works": run an **odd number of nodes, typically three or five**; instances use consensus to **elect a leader**, who makes the decisions. **Heartbeats** — regular calls between instances — detect a leader that stops responding; then a **new election** begins, everyone announces candidacy and votes for themselves, and whoever

gets a majority assumes power. "The best thing about Raft is that it's relatively easy to understand. The second best thing is that **it works**." There is an interactive animation at raft.github.io.

The properties that matter operationally:

- etcd holds **pretty much all of the data** about a Kubernetes cluster.
- It is **strongly consistent**: writes replicate to all nodes, and **whichever node you connect to, you get up-to-date data**.
- The price is performance**. Three or five nodes is typical because that gives enough fault tolerance, and **"any extra nodes just slow the cluster with little benefit."**

Why odd numbers — the counter-intuitive bit worth memorising. Quorum is $n/2 + 1$.

- 3 nodes** → majority of 2 → you can lose **1** node.
- 4 nodes** → majority of 3 → you can *still* lose only **1** node, but **there are now more nodes that can fail**.

"Even numbers of members actually decrease fault tolerance." Adding a node made you worse off.

Running etcd reliably "is not easy." It requires understanding your hardware profiles, tweaking parameters accordingly, continuous monitoring, and keeping up with bug fixes — "and building an understanding of what actually happens when failure occurs and whether the cluster heals correctly."

Experiment ideas — etcd

#	Experiment	Questions to answer
1	In a three-node cluster, take down a single etcd instance	Does <code>kubectl</code> still work? Can you schedule, modify and scale pods? Do you see failures connecting to etcd — clients are expected to retry against another instance if the one they reached doesn't respond. When the node comes back, does the cluster recover, and how long does it take? Can you see the new leader election and the small traffic increase in your monitoring?
2	Restrict CPU available to an etcd instance, simulating an unusually loaded host	Does the cluster still work? Does it slow down — by how much?
3	Add networking delay to a single etcd instance	Does one slow instance affect overall performance? Can you see the slowness in monitoring? Will you be alerted? Does your dashboard show how close the values are to the limits that cause time-outs?
4	Take down enough nodes to lose quorum	Does <code>kubectl</code> still work? Do pods already on the cluster keep running? Does healing work — is a killed pod restarted; is a deleted deployment-managed pod recreated?

The consumer's version of this chapter, and the best single piece of advice in it: "if you're using a **managed Kubernetes offering**, you're trusting that the people responsible for running your clusters know the answers to all these questions (**and that they can prove it with experimental data**). **Ask them. If they're taking your money, they should be able to give you reasonable answers!**"

kube-apiserver

It provides the APIs to read and modify cluster state, and **every component interacting with the cluster does so through it**. Run it in multiple copies for availability. Because **all state is in etcd and etcd guarantees consistency, kube-apiserver can be stateless**.

That statelessness means running it is much simpler. You just need **enough instances to handle the request load**, there are **no majorities to worry about**, and instances can be **load-balanced** — "although some internal components are often configured to **skip the load balancer**" (Figure 12.3). kube-apiserver knows about all etcd nodes but **speaks to one at a time**.

The book's verdict: "you will find kube-apiserver **start up quickly and perform pretty well**. Despite the amount of work it does, running it is pretty lightweight."

Experiment ideas — kube-apiserver

#	Experiment	Rationale
1	Create traffic to kube-apiserver	"Since everything (including the internal components responsible for creating, updating and scheduling resources) talks to kube-apiserver, creating enough traffic to keep it busy could affect how the cluster behaves."
2	Add network slowness in front of it	"Adding a networking delay in front of the proxy could lead to a buildup of queuing of new requests and adversely affect the cluster."

These two are the highest-leverage control-plane experiments in the chapter, because kube-apiserver is the one dependency shared by **every** other component. Degrading it is the cheapest way to find out which of your controllers times out first.

kube-controller-manager

It implements **the infinite control loop**, continuously detecting changes in cluster state and reacting to move toward the desired state. "You can think of it as **a collection of loops, each handling a particular type of resource.**"

The full cascade for a deployment (Figure 12.4), more detailed than Figure 12.1:

1. **kubectl** asks **kube-apiserver** to create a **Deployment**.
2. The **deployments controller**, part of kube-controller-manager, is notified and creates a **ReplicaSet**, whose purpose is to ensure the desired number of pods runs.
3. The **replica set controller** is notified about the new ReplicaSet and **creates pods**, with no nodes assigned yet.

Both the notification mechanism — **called a watch in Kubernetes** — and the updates are served by kube-apiserver. The same cascade runs for updates and deletes.

"This loosely coupled setup allows for **separation of responsibilities**; each controller does only one thing. It is also **the heart of the ability of Kubernetes to heal from failure. Kubernetes will attempt to correct any discrepancies from the desired state *ad infinitum*.**"

Leader election. Like kube-apiserver it runs in multiple copies. **Unlike kube-apiserver, only one copy does work at a time.** Instances agree on the leader by **acquiring a lease in etcd** — etcd's API allows acquiring a **lock: a distributed mutex with an expiration date**. Three instances try simultaneously, one succeeds, and the lease must be **renewed before it expires**. If the leader stops working or disappears, **the lease expires and another copy acquires it**. "Once again, etcd comes in handy and allows for **offloading a difficult problem (leader election)** and keeping the component relatively simple."

Experiment ideas — kube-controller-manager

#	Experiment	Questions
1	How does kube-apiserver traffic affect convergence speed?	kube-controller-manager gets all its information from kube-apiserver. "At what point does kube-controller-manager start timing out, rendering the cluster broken?"
2	How does lease expiry affect recovery from losing the leader?	If you run your own cluster you choose the timeouts, including the leadership lease expiry . "A shorter value will increase the speed at which the cluster restarts converging after losing the leader, but it comes at the price of increased load on kube-apiserver and etcd. "

Experiment 2 is a textbook tunable trade-off: **recovery latency against steady-state load on the shared dependency**. Measure both sides before changing it.

kube-scheduler

It detects pods not scheduled on any node and finds them a home — brand-new pods, or replacements for pods whose node went down. It tries to find a **best fit** in two steps:

1. **Filter out** nodes that don't satisfy the pod's requirements.
2. **Rank** the remaining nodes with scores based on a predefined list of priorities.

Filters:

- The **resources (CPU, RAM, disk) requested** by the pod fit in the node.
- Any **ports requested on the host** are available on the node.
- Whether the pod must run on a node with a **particular hostname**.
- The **affinity or anti-affinity** requested by the pod matches, or doesn't match, the node.
- The node is **not under memory or disk pressure**.

Priorities used for ranking:

Priority	Effect
Highest amount of free resources after scheduling — higher is better	Enforces spreading
Balance between CPU and memory utilization — more balanced is better	Avoids lopsided nodes
Anti-affinity — matching nodes are least preferred	Honours your spreading rules
Image locality — nodes that already have the image are preferred	Minimizes the number of image downloads

Like kube-controller-manager, multiple copies run but **only the leader schedules**, so "this component is prone to basically the same issues."

Image locality is the priority that connects this chapter to Chapter 11's SLO experiment. The scheduler *prefers* nodes that already hold the image precisely because pulling it is slow and failure-prone, which is why `imagePullPolicy: Always` is how you force the worst case in an SLO test.

Pop quizzes: Cluster data is stored **in etcd**. The **control plane** is "the set of components implementing the logic of Kubernetes converging toward the desired state." The component that **starts and stops processes on the host is kubelet**. The made-up component in the final quiz is **kube-converge-loop**.

12.1.2 Theory — Kubelet and the pause container

Kubelet is the agent starting and stopping containers on a host to implement the pods you requested. "**Running a Kubelet daemon on a computer turns it into a part of a Kubernetes cluster**. Don't be fooled by the affectionate name; Kubelet is a real workhorse, doing the dirty work ordered by the control plane."

It reads state and takes orders from **kube-apiserver**, and **reports back the factual state**: what is running, whether it is crashing, how much CPU and RAM is actually used. The control plane uses that data for decisions and exposes it to you.

The lifecycle (Figure 12.5):

1. Kubelet is notified about a new pod scheduled for this node.
2. It **downloads the requested image**.
3. It creates **two** containers: the one you requested, **and a special one called pause**.
4. If your container crashes, **Kubelet brings it back up**.
5. On pod deletion or rescheduling, Kubelet removes the containers.

What the pause container is for — "a pretty neat hack." In Kubernetes the unit of software is the **pod**, not the container. Containers in a pod **share some resources and not others**: processes in two containers of one pod **share an IP address and can communicate via localhost**, implemented by **sharing the network namespace** (Chapter 5). Other things, such as **the CPU limit, apply to each container separately**.

"The reason for **pause** to exist is simply to **hold these resources while the other containers might be crashing and coming back up**. The pause container doesn't do much. It **starts and immediately goes to sleep**."

This is why a crash-looping container keeps its IP address, and it is Chapter 5's namespace theory doing load-bearing work inside Kubernetes.

Kubelet is a single point of failure per node. "If it crashes, for whatever reason, **no changes will be made to the containers running on that node, even though Kubernetes will happily accept your changes. They just won't ever get implemented on that node.**"

Experiment ideas — Kubelet

#	Experiment	What you learn
1	After Kubelet dies, how long until pods are rescheduled elsewhere?	When Kubelet stops reporting readiness, after a configurable timeout (default 5 minutes at the time of writing) the node is marked NotReady . Pods are not immediately removed; the control plane waits another configurable timeout before assigning them elsewhere. So if a node disappears — for example the hypervisor crashes — there is a minimum wait before pods run somewhere else.
1b	Kubelet dies or is partitioned while the pod keeps running	"You're going to end up with a node running whatever it was running before the event , and it won't get any updates. One of the possible side effects is to run extra copies of your software with potentially stale configuration. "
2	Does Kubelet restart correctly after crashing?	Kubelet "typically runs directly on the host to minimize the number of dependencies. If it crashes, it should be restarted." "As you saw in chapter 2, sometimes setting things up to get restarted is harder than it initially looks" — test consecutive crashes, time-spaced crashes, and other patterns. "This takes little time and can avoid pretty bad outages."

Tooling note: PowerfulSeal (Ch. 11) can take VMs up and down, kill processes, **and execute commands over SSH** — for example to kill or switch off Kubelet.

Experiment 2 is Chapter 2's systemd `StartLimitIntervalSec` lesson pointed at the most critical daemon on the node. Same defect class, far worse consequences: a Kubelet that gives up restarting leaves a node that silently ignores the control plane.

12.1.3 Theory — Container runtimes and the CRI

Kubelet uses lower-level software — **container runtimes** — to start and stop containers. Kubernetes was originally written to use **Docker** directly, "and you can still see some naming that matches one-to-one to Docker; even the `kubect` CLI feels similar to the Docker CLI."

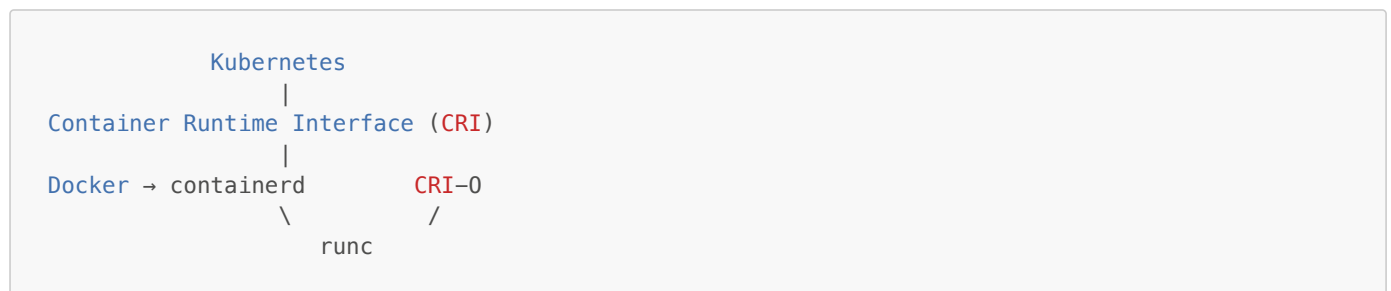
Support for new runtimes was initially baked into Kubernetes internals. To standardise, the **Container Runtime Interface (CRI)** was introduced in **Kubernetes 1.5, in 2016**.

What the CRI enabled:

- **Windows support since Kubernetes 1.14**, using **Windows containers**.
- On Linux, alternatives using basically the same underlying technologies as Docker:

Runtime	Notes
containerd	"The emerging industry standard that seems poised to eventually replace Docker." Confusingly, Docker ≥ 1.11.0 uses containerd under the hood to run containers.
CRI-O	Aims to be a simple, lightweight runtime optimized for use with Kubernetes
rkt	Initially by CoreOS; appears no longer maintained . Pronounced "rocket".

And underneath them all: both containerd — and therefore Docker — and CRI-O share code via **runc**, which manages the lower-level aspects of running a Linux container (Figure 12.6):



The Open Container Initiative (OCI). To avoid competing standards, companies led by Docker formed the OCI, which provides two specifications:

- **Runtime Specification** — how to run a **filesystem bundle**, the new term for what used to be a downloaded and unpacked Docker image.

- **Image Specification** — what an **OCI image** (new term for a Docker image) looks like, and how to build, upload and download one.

"As you might imagine, most people didn't just stop using names like *Docker images*... so things can get a little bit confusing at times. But that's all right. At least there is a standard now!"

The plot twist — runtimes that aren't containers at all (Figure 12.7):

Project	What it actually runs
Kata Containers	" Lightweight VMs " optimized for speed — a container-like experience with stronger isolation from a hypervisor
Firecracker	" micro-VMs ", implemented with KVM . Same idea as Kata, different implementation.
gVisor	Container isolation done differently: a user-space kernel implementing a subset of syscalls , with the program set up to capture the process's syscalls and execute them in that user-space kernel . "That capture and redirection introduces a performance penalty ... the default leverages ptrace (Chapter 6), so it takes a serious performance hit ."

Why this matters for your experiments: "If you're running **Docker**, everything you learned in chapter 5 is **directly applicable** to your Kubernetes cluster. If you're using **containerd or CRI-O**, the experience will be **mostly the same**, because they use the same underlying technologies. **gVisor will differ in many aspects** because of its different approach to isolation. If your cluster uses **Kata Containers or Firecracker**, you're going to be running **VMs rather than containers**."

Before designing a container-level chaos experiment on Kubernetes, **find out which runtime you are actually on**. Note the callback: the book says gVisor's default capture mechanism **leverages ptrace** and therefore "takes a serious performance hit" — the same mechanism whose cost Chapter 6 measured with **strace**. (*Supplementary caution: the book does not claim gVisor's overhead equals strace's measured ~100×, only that both stem from ptrace-based capture.*)

12.1.4 Theory — Kubernetes networking

Three parts to understand: **pod-to-pod**, **service**, and **ingress** networking.

Pod-to-pod networking

Where does a pod's IP come from? "The answer is simple: **it's a made-up IP address that's assigned to the pod by Kubelet when it starts**." A range is configured for the cluster, **subranges are given to every node**, and Kubelet knows its subrange and gives each pod it creates an address from it. From inside the pod, that address looks like the address of its network interface.

"Unfortunately, **by itself, this doesn't implement any pod-to-pod networking**. It merely attributes a fake IP address to every pod and then stores it in kube-apiserver."

Kubernetes expects you to configure networking independently, and specifies only two conditions, not how to meet them:

1. **All pods can communicate with all other pods on the cluster directly.**
2. **Processes running on a node can communicate with all pods on that node.**

This is typically done with an **overlay network**: nodes are configured to route the fake IP addresses among themselves and deliver them to the right containers. The interface is standardised as the **Container Networking Interface (CNI)**, and the official docs listed **29 options** at the time of writing.

Flannel, as the simplest example (Figure 12.8): a daemon (**f^lanneld**) runs on each node; the daemons **agree on subranges** and **store that information in etcd**; each daemon ensures packets for other ranges are **forwarded to the respective node**; the receiving **f^lanneld delivers them to the right container**. Forwarding uses a supported backend, such as **VXLAN**.

Worked example: cluster range **192.168.0.0/16**; node A gets **192.168.1.0/24**, node B gets **192.168.2.0/24**. Pod A1 (**192.168.1.1**) sends to pod B2 (**192.168.2.2**). Flannel's forwarding **matches node B's range, encapsulates, and sends**; on node B, Flannel **decapsulates and delivers**. "From the perspective of a pod, our fake IP addresses are as real as anything else."

"More advanced solutions [do] things like allowing for **dynamic policies that dictate which pods can talk to what other pods**... But the high-level idea is the same: the pod IP addresses get routed, and **a daemon is running on each node that makes sure that**

happens. And that daemon will always be a fragile part of the setup. If it stops working, the networking settings will be stale and potentially wrong."

"Stale and potentially wrong" is the key phrase. Not "down" — *wrong*. Traffic going to the wrong place is far harder to detect than traffic going nowhere.

Service networking

Service IP addresses are **also completely made up**. They are implemented by **kube-proxy**, which runs on every node: it **watches for changes to the pods matching a label** and **reconfigures the host to route these fake IPs to their destinations**. It also provides **load balancing**, since one service IP resolves to many pod IPs.

With the **iptables** backend, kube-proxy creates rules forwarding packets to particular pod IPs, each with a probability, and **the first matching rule wins**. For a service with three pods:

```
1. If IP == SERVICE_IP, forward to pod A with probability 33%
2. If IP == SERVICE_IP, forward to pod B with probability 50%
3. If IP == SERVICE_IP, forward to pod C with probability 100%
```

On average traffic is split roughly equally.

The weakness: "iptables evaluates all the rules one by one until it hits a rule that matches. As you can imagine, **the more services and pods you're running, the more rules there will be**, and therefore the bigger overhead this will create." The alternative backend is **IPVS**, which "scales much better for large deployments."

Experiment ideas — service networking

#	Experiment	What to look for
1	Does the number of services affect networking speed?	"If you're using iptables, you will find that just creating a few thousand services (even if they're empty) will suddenly and significantly slow the networking on all nodes . Do you think your cluster shouldn't be affected? You're one experiment away from checking that. "
2	How good is the load balancing?	"With probability-based load balancing, you might sometimes find interesting results in terms of traffic split . It might be a good idea to verify your assumptions about that."
3	What happens when kube-proxy is down?	"If the networking is not updated, it is quite possible to end up with not only stale routing that doesn't work, but also routing to the wrong service . Can your setup detect when that happens? Would you be alerted if requests start flowing to the wrong destinations? "

Experiment 1 is the most surprising claim in the chapter and the easiest to test: **empty services still cost you**, because the cost is in rule evaluation, not traffic. It is a pure scale-limit experiment with essentially no blast radius.

Ingress networking

An **ingress** is a natively supported Kubernetes resource describing **a set of hosts and the destination service they route to** — for example, requests for **example.com** go to service **example** in namespace **mynamespace** on port 8080.

But creating the resource does nothing by itself. You need an **ingress controller** installed that watches for changes to those resources and implements them. The official docs listed **15 options**.

The NGINX ingress controller as the example: it runs a pod on each host, and inside runs **an instance of NGINX plus an extra process that listens for changes to ingress resources**. On each change it **regenerates the NGINX config and asks NGINX to reload it**. NGINX then knows which hosts to listen on and where to proxy traffic.

"The ingress controller is typically **the single point of entry to the cluster**, and so everything that prevents it from working well will deeply affect the cluster. And **like any proxy, it's easy to mess up its parameters.**"

Experiment ideas — ingress

#	Experiment	Questions
1	Create or modify an ingress so the config is reloaded	Are the existing connections dropped? What about corner cases like WebSockets?
2	Compare timeouts across the proxy boundary	"Does your proxy have the same time-out as the service it proxies to? If you time out quicker, not only can you have outstanding requests being processed long after the proxy dropped the connection, but the consequent retries might accumulate and take down the target service."

Ingress Experiment 2 is Chapter 1's retry storm, reborn at the edge. A proxy timing out faster than its upstream turns every slow request into a retry, and the retries pile onto a service that is already struggling. Check timeout ordering along the whole request path: client → ingress → service → database.

12.2 Summary of key components (Table 12.1)

Component	Key function
kube-apiserver	Provides APIs for interacting with the Kubernetes cluster
etcd	The database used by Kubernetes to store all its data
kube-controller-manager	Implements the infinite loop converging the current state toward the desired one
kube-scheduler	Schedules pods onto nodes, trying to find the best fit
kube-proxy	Implements the networking for Kubernetes services
Container Networking Interface (CNI)	Implements pod-to-pod networking — for example Flannel, Calico
Kubelet	Starts and stops containers on hosts, using a container runtime
Container runtime	Actually runs the processes (containers, VMs) on a host — Docker, containerd, CRI-O, Kata, gVisor

A consolidated experiment map for Chapter 12

Target	Experiment	Primary risk it exposes
etcd	Kill 1 of 3 instances	Client retry behaviour; leader election; recovery time
etcd	CPU-starve one instance	Cluster-wide slowdown from one degraded member
etcd	Add network delay to one instance	Whether one slow member drags the quorum; alerting headroom before timeouts
etcd	Lose quorum	Whether running pods survive; whether healing stops
kube-apiserver	Flood with traffic	Everything depends on it — which controller times out first?
kube-apiserver	Add network delay	Request queue buildup
kube-controller-manager	Traffic on kube-apiserver	Convergence speed; the point at which the cluster is "broken"
kube-controller-manager	Kill the leader	Lease-expiry vs. recovery-time trade-off
kube-scheduler	Same as controller-manager	Leader election; scheduling stalls
Kubelet	Kill it or partition it	NotReady timeout + eviction timeout; stale config still serving traffic
Kubelet	Crash it repeatedly	Restart policy limits (Ch. 2's start-limit-hit)

Target	Experiment	Primary risk it exposes
kube-proxy	Stop it	Stale and wrong routing ; do you alert on traffic reaching the wrong service?
kube-proxy	Create thousands of even empty services	iptables rule-evaluation overhead; IPVS as the fix
CNI daemon (e.g. flannel)	Stop it	Stale, potentially wrong pod networking
Ingress controller	Reload config under load	Dropped connections; WebSockets
Ingress controller	Mismatched timeouts	Retry accumulation taking down the upstream

Theory ↔ Practice connections for Chapter 12

- **Namespaces (Ch. 5) ↔ the pause container (§12.1.2)**: the pod's shared IP *is* a shared network namespace, and **pause** exists to hold it while your container crash-loops. Chapter 5's theory, load-bearing.
- **cgroups (Ch. 5) ↔ "the CPU limit applies to each container separately"**: namespaces are shared within a pod; cgroup limits are not.
- **ptrace overhead (Ch. 6) ↔ gVisor (§12.1.3)**: the same mechanism, the same performance penalty, now as a production isolation choice.
- **systemd restart limits (Ch. 2) ↔ Kubelet Experiment 2 (§12.1.2)**: identical defect class, far higher stakes.
- **Retry amplification (Ch. 1 DNS storm) ↔ ingress Experiment 2 (§12.1.4)**: mismatched timeouts manufacture retries at the front door.
- **MTTF math (Ch. 11 §11.3) ↔ etcd's reason for existing (§12.1.1)**: fault tolerance and availability are the *answers* to the arithmetic Chapter 11 did.
- **The control-plane pod list (Ch. 10 §10.3.2) ↔ this whole chapter**: every name in that first `kubectl get pods -A` now has a failure mode.
- **PowerfulSeal (Ch. 11) ↔ these experiments: nodeAction, stopHost**, and SSH command execution are how most of the table above gets implemented.
- **tc, Pumba, Toxiproxy (Ch. 4, 5, 10) ↔ "add network delay to one etcd instance"**: you already have three ways to do it.

Key Takeaways — Chapter 12

1. Kubernetes is **a set of loosely coupled components using etcd as the storage for all data**, coordinating through **asynchronous, eventually consistent loops**. Most failures therefore show up as **staleness and latency, not errors**.
2. **kube-apiserver sits at the centre**. Everything talks to it, it alone talks to etcd, and it is stateless and therefore easy to scale and load-balance. It is also the shared dependency whose degradation degrades everything.
3. **etcd** gives fault tolerance and availability through **Raft consensus**: odd-sized clusters of 3 or 5, leader election by majority, heartbeats to detect a dead leader, and strong consistency at the cost of performance.
4. **Even-numbered etcd clusters reduce fault tolerance**. Quorum is $n/2 + 1$; four nodes tolerate the same single failure as three while adding a node that can fail.
5. **kube-controller-manager** is a collection of per-resource control loops. Deployment → ReplicaSet → Pods is a cascade of separate controllers, notified by **watch** through kube-apiserver. This separation **is** Kubernetes' self-healing.
6. **Leader election for controller-manager and scheduler is a lease — a distributed mutex with an expiry — held in etcd**. Shorter leases mean faster recovery and more load on etcd and kube-apiserver.
7. **kube-scheduler** filters (resources, host ports, hostname, affinity and anti-affinity, memory and disk pressure) then ranks (most free resources for spreading; CPU/memory balance; anti-affinity; **image locality**).
8. **Kubelet turns a machine into a cluster node**, downloads images, creates containers, reports actual state, and restarts crashed containers. It is **a per-node single point of failure**: when it is gone, your changes are accepted and never applied.
9. **The pause container holds the pod's shared resources — above all its IP, via the network namespace — while the real containers crash and restart**.
10. Node loss is **not** fast: a **NotReady timeout (default 5 min)** plus a **second eviction timeout** before pods move. Plan SLOs accordingly.
11. **CRI decoupled Kubernetes from Docker**. Docker → containerd → runc; CRI-O → runc; plus **Kata Containers** and **Firecracker** (lightweight and micro VMs) and **gVisor** (user-space kernel, **ptrace**-based, with a real performance cost). **Know your runtime before designing container-level experiments**. **OCI** standardised the runtime and image specifications.

12. **Pod IPs are made up**, assigned by Kubelet from a per-node subrange. Kubernetes only requires that **all pods can reach all pods**, and **node processes can reach that node's pods**. The CNI plugin — Flannel, Calico and ~27 others — makes it true, usually via an overlay with encapsulation.
13. **Service IPs are also made up**, implemented by **kube-proxy** via **iptables** (probability rules, evaluated in order, with overhead growing with service and pod count) or **IPVS**, which scales better.
14. **A dead networking daemon leaves routing stale and possibly wrong**. Alert on traffic reaching the wrong destination, not only on traffic failing.
15. **Ingress resources do nothing without an ingress controller**. Reloads can drop connections, and **a proxy timing out faster than its upstream manufactures retry storms**.

Appendices A–D

Appendix A — Installing chaos engineering tools

All tools except Kubernetes are preinstalled in the book's VM, "so the easiest way to benefit from the book is to just start the VM." This appendix covers running them on any host.

A.1 Prerequisites

- A **Linux machine**. Everything was tested on **kernel 5.4.0**, using **Ubuntu 20.04 LTS** — "but none of the tools used in the book are Ubuntu-specific."
- **x86 architecture** assumed; nothing tested on others.
- **At least 8 GB of RAM and multiple cores** recommended. Chapters 10–12 use a small VM for Kubernetes, for which **4 GB of RAM** is recommended.
- An internet connection.

A.2 The Linux packages (Table A.1)

```
sudo apt-get install PACKAGE=VERSION      # e.g. sudo apt-get install git=1:2.25.1-1ubuntu3
```

The author's caveat: "in the fast-moving Wild West of open source packaging, **the versions used here will probably be outdated by the time these words are printed... When in doubt, try to go for the latest packages.**"

Package	Version used	Where it is used
<code>git</code>	1:2.25.1-1ubuntu3	Download the book's code
<code>vim</code>	2:8.1.2269-1ubuntu5	Text editor ("Yes, you can use Emacs.")
<code>curl</code>	7.68.0-1ubuntu2.2	HTTP calls from the terminal, many chapters
<code>nginx</code>	1.18.0-0ubuntu1	HTTP server — Ch. 2 and 4
<code>apache2-utils</code>	2.4.41-4ubuntu3.1	Apache Bench (ab) — used in many chapters
<code>docker.io</code>	19.03.8-0ubuntu1.20.04	Container runtime — Ch. 5
<code>sysstat</code>	12.2.0-2	iostat, mpstat, sar — Ch. 3, used across the book
<code>python3-pip</code>	20.0.2-5ubuntu1	Package manager for Python — Ch. 11
<code>stress</code>	1.0.4-6	Generate CPU/RAM/I-O load — Ch. 3 , used in many chapters
<code>bpfcc-tools</code>	0.12.0-2	BCC tools (eBPF insights) — Ch. 3
<code>cgroup-lite</code> / <code>cgroup-tools</code> / <code>cgroupfs-mount</code>	1.15 / 0.41-10 / 1.4	cgroup utilities — Ch. 5
<code>apache2</code>	2.4.41-4ubuntu3.1	HTTP server — Ch. 4

Package	Version used	Where it is used
php	2:7.4+75	PHP — Ch. 4
wordpress	5.3.2+dfsg1-1ubuntu1	Blogging engine — Ch. 4
manpages	5.05-1	Manual pages, throughout
manpages-dev	5.05-1	Sections 2 (syscalls) and 3 (library calls) — Ch. 6
manpages-posix / manpages-posix-dev	2013a-2	POSIX flavour of the above — Ch. 6
libseccomp-dev	2.4.3-1ubuntu3.20.04.3	Compile code using seccomp — Ch. 6
openjdk-8-jdk	8u265-b01-0ubuntu2~20.04	Run Java — Ch. 7
postgresql	12+214ubuntu0.1	Database — Ch. 9

A.2.1–A.2.5 The non-packaged tools

Pumba (Ch. 5):

```
curl -Lo ./pumba "https://github.com/alexei-
led/pumba/releases/download/0.6.8/pumba_linux_amd64"
chmod +x ./pumba
sudo mv ./pumba /usr/bin/pumba
```

Python 3.7 with DTrace (Ch. 3) — needed for the USDT probes that `pythonstat` and `pythonflow` attach to:

```
# build dependencies (abridged): build-essential checkinstall libreadline-gplv2-dev
# libncursesw5-dev libssl-dev libsqlite3-dev tk-dev libgdbm-dev libc6-dev libbz2-dev
# zlib1g-dev openssl libffi-dev python3-dev python3-setuptools curl wget systemtap-sdt-dev
cd ~
curl -o Python-3.7.0.tgz https://www.python.org/ftp/python/3.7.0/Python-3.7.0.tgz
tar -xzf Python-3.7.0.tgz
cd Python-3.7.0
./configure --with-dtrace # <-- THE point of building from source
make && make test && sudo make install && make clean
./python --version
```

pgweb (Ch. 9):

```
sudo apt-get install -y unzip
curl -s https://api.github.com/repos/sosedoff/pgweb/releases/latest \
| grep linux_amd64.zip | grep download | cut -d '"' -f 4 | wget -qi - \
&& unzip pgweb_linux_amd64.zip && rm pgweb_linux_amd64.zip \
&& sudo mv pgweb_linux_amd64 /usr/local/bin/pgweb
```

Pip dependency (Ch. 3): `pip3 install freegames`

Example data for pgweb (Ch. 9):

```
git clone https://github.com/sosedoff/pgweb.git /tmp/pgweb
cd /tmp/pgweb && git checkout v0.11.6
sudo -u postgres psql -f ./data/booktown.sql
```

A.3 Configuring WordPress (Ch. 4)

1. Apache site config at `/etc/apache2/sites-available/wordpress.conf`:

```
Alias /blog /usr/share/wordpress
<Directory /usr/share/wordpress>
    Options FollowSymLinks
    AllowOverride Limit Options FileInfo
    DirectoryIndex index.php
    Order allow,deny
    Allow from all
</Directory>
<Directory /usr/share/wordpress/wp-content>
    Options FollowSymLinks
    Order allow,deny
    Allow from all
</Directory>
```

1. **Enable it:** `a2ensite wordpress`, then `service apache2 reload || true`
2. **WordPress DB config** at `/etc/wordpress/config-localhost.php`:

```
<?php
define('DB_NAME', 'wordpress');
define('DB_USER', 'wordpress');
define('DB_PASSWORD', 'wordpress');
define('DB_HOST', '127.0.0.1');
define('WP_CONTENT_DIR', '/usr/share/wordpress/wp-content');
define('WP_DEBUG', true);
```

1. Create the database:

```
CREATE DATABASE wordpress;
CREATE USER 'wordpress'@'localhost' IDENTIFIED BY 'wordpress';
GRANT SELECT,INSERT,UPDATE,DELETE,CREATE,DROP,ALTER ON wordpress.* TO wordpress@localhost;
FLUSH PRIVILEGES;
```

Then browse to `localhost/blog`.

A.4 The book's source code

```
git clone https://github.com/seeker89/chaos-engineering-book.git ~/src
```

A.5 Installing Minikube (Ch. 10–12)

Do not run Minikube inside the book's VM. Two reasons: Minikube is officially supported by the Kubernetes team on Windows, Linux and macOS, "so there is no need to reinvent the wheel"; and **Minikube itself starts a VM**, so running it inside the book's VM means "a VM inside of a VM."

Check virtualization support first:

OS	Command	Look for
Linux	<code>`grep -E --color 'vmx`</code>	<code>svm` /proc/cpuinfo`</code>
macOS	<code>`sysctl -a`</code>	<code>grep -E --color 'machdep.cpu.features`</code>
Windows	<code>systeminfo</code>	The Hyper-V Requirements section

Linux install:

```
curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s
https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl
chmod +x ./kubectl && sudo mkdir -p /usr/local/bin/ && sudo mv ./kubectl /usr/local/bin/kubectl
kubectl version --client

curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
chmod +x minikube && sudo install minikube /usr/local/bin/
minikube version
```

macOS is identical with `darwin/amd64` and `minikube-darwin-amd64`. On Windows, download both binaries and add them to `PATH`. Troubleshooting: minikube.sigs.k8s.io/docs/.

Appendix B — Pop quiz answers (consolidated)

A compact self-test. Correct answers in **bold**.

Chapter 2

- False statement about return codes: **"There are 32 possible exit codes."**
- What's OOM? **A mechanism that kills processes when the system runs low on resources.**
- Not part of the chaos experiment template: **Crying in the corner when an experiment fails.**
- What's a blast radius? **The amount of stuff that can be affected by our actions.**

Chapter 3

- What's USE? **A method of debugging a performance issue based around measuring utilization, saturation and errors.**
- Where are kernel logs? **dmesg.**
- Which command does *not* show disk statistics? **top.**
- Which does *not* show networking statistics? **free.**
- Which does *not* show CPU statistics? **free.**

Chapter 4

- What can **tc** *not* do? **Give you permission for landing the aircraft.**
- When should you test in production? **When you've done your homework, tested in other stages, applied common sense, and see the benefits outweighing the potential problems.**
- True statement: **Chaos engineering is a methodology to improve your software beyond the existing testing methodologies.**

Chapter 5

- Example of OS-level virtualization: **Docker container.**
- True statement: **VMs typically offer better security than containers.**
- True statement: **Docker built on top of existing Linux technologies to provide an accessible way of using containers, rendering them much more popular.**
- What does **chroot** do? **Change the root of the filesystem from the perspective of a process.**
- What do namespaces do? **Limit what a process can see and access for a particular type of resource.**
- What do cgroups do? **Limit the resources that a process can consume (CPU, memory, and so forth).**
- What is Pumba? **A handy wrapper around **tc** that facilitates working with Docker containers, and that also lets you kill containers.**

Chapter 6

- What are syscalls? **All of the above** — a way to request actions on physical devices; a way to communicate with the kernel; **and a universal angle of attack for chaos experiments.**
- What can **strace** do? **Show you what syscalls a process is making in real time** — *not* without a performance penalty.
- What's BPF? **Options 2, 3 and much more** — packet filtering *and* executing special code inside the kernel for visibility.
- Worth investing time in BPF? **Yes / Definitely / Absolutely / Positively** (all four).

Chapter 7

- What's javaagent? **A flag used to specify a JAR that contains code to inspect and modify the code loaded into the JVM on the fly.**
- Not built into the JVM: **A mechanism for generating enterprise-ready names.**

Chapter 8

- When to build chaos engineering into the application? **When it's more convenient, easier, safer, or you have access to only the application level.**
- What is *not* important? **Rubbing the ingenuity of your design into everyone else's faces.**

Chapter 9

- What is `XMLHttpRequest`? **One of the two main ways for JavaScript code to make requests, along with the Fetch API.**
- Best way to simulate a frontend loading slowly: **A modern browser, like Firefox or Chrome.**
- True statement: **JavaScript's ubiquitous nature combined with its lack of safeguards makes it very easy to inject code to implement chaos experiments on the fly.**

Chapter 10

- What's Kubernetes? **A container orchestrator that can manage thousands of VMs and will continuously try to converge the current state into the desired state.**
- What's a deployment? **A description of how to deploy some software on your cluster.**
- What happens when a pod dies? **Kubernetes detects it and will restart it as necessary to make sure the expected number of replicas are running.**
- What's Toxiproxy? **A configurable TCP proxy that can simulate various problems, such as dropped packets or network slowness.**

Chapter 11

- What does PowerfulSeal do? **Allows you to write a YAML file to describe how to run and validate chaos experiments.**
- When to run experiments continuously? **All of the above** — SLO violation detection, absence of problems proving nothing, randomness, and regression detection.
- What can PowerfulSeal *not* do? **Fill in the existential discomfort about better versions of you in infinite universes.**

Chapter 12

- Where is cluster data stored? **In etcd.**
- What's the control plane? **The set of components implementing the logic of Kubernetes converging toward the desired state.**
- Which component starts and stops processes on the host? **kubelet.**
- Can you use a runtime other than Docker? **Yes — CRI-O, containerd and others.**
- Which component was made up? **kube-converge-loop.**

Appendix C — Director's cut (what was left out, and why)

This appendix is genuinely useful: **it tells you the boundaries of the book**, which is exactly what you need to plan what to learn next.

Topic left out	The author's reasoning
A "Cloud" chapter	A multi-cloud solution for taking VMs up and down is already covered in Chapter 11 with PowerfulSeal ; different providers have their own tools and APIs and he wanted to focus on things that are as portable as possible ; and "at the end of the day, it's just someone else's computer. "
A big chaos-tool comparison table	Open source projects get different levels of support — "some flourish, some slowly degrade" — so a detailed table "would produce value mainly for the archaeologists who might dig it out a few thousand years later." And " it's better for you to form your own opinions anyway. " The tools he did cover (Pumba, PowerfulSeal, Chaos Toolkit) are ones he's "fairly confident will stay relevant." Up-to-date list: the Awesome Chaos Engineering repo.
Windows	"One of the (valid) criticisms of this book is that it's entirely Linux-based." He doesn't cover it because he "would be out of my depth." Crucially: "The mindset and the methodology are universal, and

Topic left out	The author's reasoning
	will work regardless of which operating system you use. The tooling, on the other hand, will differ."
Runtime-specific observability	Beyond the DTrace-with-Python taste in Ch. 3, "various languages, runtimes, and frameworks often offer metrics out of the box that can be useful from the perspective of observability. This subject could be a book in itself. "
Node.js application-level chaos	A "surprisingly persistent" request. The answer: "If you understand my points from chapter 8 in Python, you can replicate that in JavaScript" , and Chapter 9 already covers JavaScript in the browser.
Architecture problems / how to design reliable systems	"There are shelves and shelves of books on designing good software. This one is about checking how well you've done." The book gives you the mindset, tools and techniques to verify that systems behave the way you expect and detect when they don't , and "leaves the actual fixing part to the users."
A fifth step called "analysis"	A reviewer asked why the model has four steps and not five. "An experiment is useless if you don't analyze your findings at the end." He kept four "primarily for the promotional reasons: fewer steps sound easier and are admittedly catchier. The analysis part is implied. "
Commercial tools	The ecosystem is young and expected to move a lot. "The basics will likely stay the same, but the specifics might look very different in just a short while."
Real-world failure examples	He tried and failed to gather them. "People are pretty excited to talk about their experiences with chaos engineering, [but] it's a completely different story when it's about going on record and telling others why your system was badly designed before you fixed it. A fair amount of stigma surrounds this topic... we all know that software is hard, but we all want to appear to be good at writing it."
The name itself	"'Chaos engineering' is a terrible name! ... The <i>chaos</i> part makes it interesting but goes a long way toward generating initial friction for adoption. "

Two of these are load-bearing for how you use the book.

(1) The missing fifth step. Treat the four-step model as **five**: observability → steady state → hypothesis → run → **analysis**. The author says so himself. Every Experiment Card in this guide has "Interpretation" and "Lesson learned" fields for exactly that reason.

(2) "This one is about checking how well you've done." The book deliberately does not teach you how to fix what you find. When an experiment refutes a hypothesis, the remedy comes from elsewhere — SRE practice, architecture, your own system knowledge.

Appendix D — Chaos-engineering recipes

Two vegetarian recipes, written in the voice of an engineer: the **SRE ('ShRoomEee) burger** — mushroom, onion, garlic and smoked tofu patties bound with flour — and **Chaos pizza**, the secret being **"it all relies on baking the pizza directly on a hot surface** — the heat transfer through direct contact is what leads to this crispy base."

They are a joke, but three details are engineering jokes worth noticing, because they restate the book's themes:

- **"Hidden dependencies"** is a named section of the burger recipe: the frying pan and spatula that the ingredient list omits. *Your dependency list is always incomplete.*
- **"(Parallelism) While the mushrooms are frying, slice the garlic and the onion"** — the Chapter 9 finding about parallel versus sequential work, in a kitchen.
- **"I recommend A/B testing, and scoring the different attempts for observability reasons"**, and for the pizza dough, **"You have a redundant copy in the pipe; no one will know."**

And the recipe's own admission of non-determinism: **"mushrooms give away a variable amount of water, depending on the type of mushroom used. Yep, it's freestyle!"** That is why the instructions are a **loop with an exit condition**: "Add 2 to 3 teaspoons of flour and mix it in well... Try to form a small ball. **If it sticks together, break the loop. If it's too runny and/or sticky, keep going.**"

Key Takeaways — Appendices

1. The book's environment is **Ubuntu 20.04, kernel 5.4.0, x86**, 8 GB RAM, with a prebuilt VM. **Pinned package versions will be stale — take the latest.**
2. The tool inventory maps cleanly onto the chapters: **ab** and **stress** everywhere; **sysstat** and **bpffcc-tools** for Chapter 3; cgroup tools and Docker for Chapter 5; **manpages-dev** and **libseccomp-dev** for Chapter 6; OpenJDK for Chapter 7; PostgreSQL for Chapter 9.
3. **Python must be built --with-dtrace** for the USDT-probe tools. That is the only reason for compiling it from source.
4. **Run Minikube on your host, not inside the book's VM.** Check virtualization support first (**vmx | svm, VMX**, Hyper-V Requirements).
5. **The four-step model is really five.** The author kept "analysis" implicit for catchiness and says so plainly in Appendix C.
6. **The book verifies systems; it does not teach you to design them.** Pair it with the SRE books from the Chapter 13 reading list.
7. **The methodology is portable; the tooling is not.** Windows, other runtimes and other languages need different tools and the same mindset.
8. **Nobody publishes their chaos-engineering failure stories**, because admitting a badly designed system carries stigma. Expect to generate your own evidence rather than find case studies.
9. The name "chaos engineering" creates real adoption friction. Chapter 13's buy-in advice exists partly to work around the word itself.

Master Cheat Sheet

Everything above, compressed for revision and for use at a keyboard. Every entry traces to a chapter.

1. Core definitions

Term	Definition	Ch.
Chaos engineering	"The discipline of experimenting on a system in order to build confidence in the system's capability to withstand turbulent conditions in production" (principlesofchaos.org). Practically: a software testing method focused on finding evidence of problems before users experience them.	1
Chaos experiment	The atomic unit of the practice: observability → steady state → hypothesis → run (→ analysis).	1
Observability	Being able to reliably see whatever metric you are interested in. The keyword is <i>reliably</i> — metrics have bugs and costs of their own.	1, 3
Steady state	The normal range of your chosen metric, so abnormality is detectable.	1
Hypothesis	A testable prediction of system behaviour under a well-defined problem, bounded by a number.	1
Blast radius	The maximum number of things that can be affected by something going wrong — and by your experiment.	2
Emergent property	A behaviour of the whole that no part possesses (cells → a heart; retries → a restart loop that never ends).	1
SLI / SLO / SLA	The number / the agreed target / the contract with a penalty.	1
USE method	For each resource: check Utilization , Saturation , Errors .	3
Utilization	Average time or proportion of a resource used.	3
Saturation	The work a resource can't service at any given moment , often queued. High saturation is not automatically bad.	3
Known / unknown unknowns, "dark debt"	Things you know you don't know / things not on your radar at all. Every complex system has the latter.	3
Black box	A system whose inputs and outputs you see but not its internals. Most chaos engineering targets are black boxes.	2

Term	Definition	Ch.
Container	"A construct designed to limit the resources that a particular program can access." OS-level virtualization: shared kernel.	5
Syscall	The API of an operating system — how a userland program talks to the kernel.	6
Toxic	Toxiproxy's word for an injected failure attached to a proxy configuration.	10
Game day	A team event to brainstorm weaknesses and try chaos engineering together. Form doesn't matter; participation does.	13

2. The methodology

1. **OBSERVABILITY** Can I measure the thing I care about, reliably?
2. **STEADY STATE** What is normal, measured over a long enough window?
3. **HYPOTHESIS** "If <specific failure>, then <metric> stays <within a number>."
4. **RUN** Inject, observe, revert.
5. **ANALYSIS** Why did that happen? What do we change? (Implicit in the book – App. C)

Scoring rule: right → more confidence. **Wrong → you found a problem before your customers did.** Both are wins; the second is the more valuable one.

Craftsmanship rules:

- Simplicity beats cleverness. "You earn no bonus points for elaborate designs."
- One experiment at a time. Two active injections make a result uninterpretable.
- Measure the steady state **with your tooling attached**. strace, proxies and agents all cost something.
- Run a near-zero control injection to bound your injector's own overhead.
- Verify the injection hit only its intended target before believing the result.
- Repeat experiments and vary parameters, especially **timing and repetition**.
- Script the teardown so a crashed experiment cannot leave the fault behind.

What chaos engineering is NOT: a silver bullet; random destruction; a tool; a replacement for unit or integration tests; a source of ready-made answers; production-only; chaos theory.

3. Theory summary, chapter by chapter

Ch.	The one idea
1	The four-step model, and why you experiment at all: quantify risk into SLOs, test the system as a whole, and find emergent properties.
2	Processes die three ways (fault, signal, OOM Killer) and you often can't tell which — so test the system's user-visible behaviour instead. Blast radius is engineering, not an afterthought.
3	You cannot experiment on what you cannot measure. USE per resource, per layer, with per-process attribution (BCC) and continuous history (Prometheus).
4	Guess where fragility lives (state is a good guess), calibrate the injection to a measured baseline, and remember latency multiplies by round trips . Production testing is a risk trade, not a taboo.
5	Containers = chroot + namespaces + cgroups + capabilities + seccomp + union FS + networking. Namespaces = what you can see; cgroups = what you can use . Isolation is thin, and storage is not isolated at all.
6	Syscalls are the universal injection point. Read the man page's ERRORS section to know which failures are real. strace is powerful and 100× expensive; BPF observes cheaply; seccomp blocks cheaply.
7	The JVM lets you rewrite bytecode at load time. grep ") throws" maps the failure surface. Exception handling is Java's weak spot — and an application can fail silently while exiting 0.

Ch. The one idea

8	When you own the code, build the injector in: off by default, gated by an env var, negligible when disabled. Easier and more precise — and riskier, because your bug becomes their outage.
9	The browser is a layer too. Override <code>XMLHttpRequest.prototype.send</code> or <code>window.fetch</code> from the console; refresh to revert. Stale data with no error is worse than an error.
10	Kubernetes converges declaratively. Labels drive discovery, routing and targeting. Prefer adding degraded capacity to degrading running capacity. A green health check can hide severe latency.
11	Automate: match → filter → act, in YAML. One passing run proves nothing, so verify SLOs continuously . At scale, hardware failure is a daily event; test machines, groups, zones and partitions.
12	Kubernetes is loosely coupled loops around kube-apiserver and etcd. Every component's failure mode is "stops converging" — staleness and latency, not errors.
13	Estimate risk instead of feeling it. Failing early beats failing late. Teams are distributed systems : find their SPOFs, bottlenecks, misinformation paths and untested procedures.

4. Practical experiment playbook (every experiment in the book)

#	Ch.	Experiment	Injection	Result
1.1	1	Cut the cache off (FizzBuzzAAS)	<code>iptables -A <ip> -j DROP</code>	Refuted — requests hang; no time-outs existed
2.1	2	Kill both API instances once each	<code>kill</code> via script	Confirmed — 0 failed requests
2.2	2	Kill instance 6x at 1.25 s	<code>kill</code> in a loop	Refuted — systemd <code>start-limit-hit</code> , both instances dead
3.1	3	Busy-neighbour CPU contention	<code>stress --cpu 2 -m 1 -d 1</code>	Refuted — pi iterations slow; fix with cgroups
4.1	4	Slow disks under WordPress	<code>stress --hdd 1</code> (~95% of measured throughput)	Confirmed — 86 → 54 RPS (~38%, under the 50% budget)
4.2	4	2 s latency to MySQL only	<code>tc prio • u32 filter + netem delay 2000ms</code>	Refuted — 11 ms → 54 s (~27 round trips/page)
5.1	5	One container fills the disk	<code>fallocate</code> loop in a second container	Confirmed — No space left on device for both
5.2	5	Kill a host PID from inside a container	<code>kill -9 <host pid></code>	Confirmed — "No such process" (invisible, not forbidden)
5.3	5	Use all the CPU with <code>--cpus=0.5</code>	<code>stress --cpu 1</code>	Confirmed — capped at ~0.5 CPU; <code>cpu.stat</code> shows throttling
5.4	5	Use too much RAM with <code>--memory=128m</code>	<code>stress --vm 1 --vm-bytes 512M</code> ; then a fork bomb	Refuted, instructively — allocation alone doesn't trigger OOM
5.5	5	100 ms latency to MySQL container	<code>pumba netem delay --time 100</code>	Refuted — 26 ms → 490 ms (18x); validated with a 1 ms control
6.1	6	Break <code>close</code>	<code>strace -e inject=close:error=EIO</code>	Refuted — exits 1 on the first call
6.2	6	Break every other <code>write</code>	<code>strace -e inject=write:error=EIO:when=1+2</code>	Confirmed — retries; throughput halved
6.3	6	Block <code>getpid</code> with seccomp	Modified Docker seccomp profile	Works, ~free — 1203 RPS vs. 101 RPS under strace

#	Ch.	Experiment	Injection	Result
7.1	7	Throw <code>IOException</code> in <code>output()</code>	javaagent + ASM <code>invokestatic</code>	Refuted — no output, still exit code 0
8.1	8	Redis latency	<code>ChaosClient</code> wrapper, <code>time.sleep</code>	Confirmed — 5.98 ms → 208 ms (2 × 100 ms)
8.2	8	Redis error every other call	Python decorator raising <code>RedisError</code>	Confirmed — graceful degradation, HTTP 200, logged
9.1	9	1 s delay on every XHR	Override <code>XMLHttpRequest.prototype.send</code> • <code>setTimeout</code>	Confirmed — requests are parallel, not sequential
9.2	9	Fail every other XHR	<code>dispatchEvent(new Event('error'))</code>	Refuted — stale data, no visible error
10.1	10	Kill a random Goldpinger pod	<code>kubectl delete</code>	Confirmed — detected and replaced in ~2 s
10.2	10	250 ms latency via a proxied extra replica	Toxiproxy <code>latency</code> toxic	Confirmed — graph green, heatmap red
11.1	11	Same, declaratively at 40%	PowerfulSeal <code>clone</code> • <code>toxiproxy</code> mutation	Confirmed ; localhost traffic unaffected
11.2	11	Pod ready within 30 s, forever	None — continuous verification	Passes ; the shape of a permanent SLO test
11.3	11	Take a VM down in one AZ	PowerfulSeal <code>nodeAction</code> / <code>stopHost</code>	Tests detection, rescheduling and anti-affinity at once

5. Tool and command reference

Process forensics (Ch. 2)

```

echo $?                # last exit code; 128+n decodes to signal n
kill -L                # list signals with numbers
ps f                   # process tree
ps auxf                # + users, full command lines
ps ao pid,pidns,command # + namespace columns
pkill <name> / pkill -9 <n> # SIGTERM / SIGKILL
dmesg | less           # kernel log; /Kill to search
dmesg --human          # human-readable timestamps
dmesg | grep -i <proc>  # OOM evidence: "oom_reaper", "Out of memory: Kill process"
cat /proc/sys/vm/oom_kill_allocating_task # 0 = scan and score; non-zero = kill the trigger
cat /proc/sys/vm/oom_dump_tasks           # 1 = dump the task table on kill
systemctl status <unit> --no-pager
systemctl daemon-reload

```

Exit codes: $136 = 128+8 = \text{SIGFPE}$ · $137 = 128+9 = \text{SIGKILL}$ · $143 = 128+15 = \text{SIGTERM}$

Observability — USE by resource (Ch. 3)

Resource	Utilization	Saturation	Errors	Per-process
System	<code>uptime</code> , <code>cat /proc/loadavg</code>	load averages (trend, not value)	<code>dmesg</code>	—
Block I/O	<code>df -h</code> (capacity), <code>iostat -x %util</code> , <code>rkB/s</code> , <code>wkB/s</code>	<code>iostat -x aqu-sz</code>	<code>dmesg</code>	<code>sudo biotop-bpfcc</code>
Network	<code>sar -n DEV 1 1%ifutil</code>	<code>sar -n TCP,ETCP 1 1 retrans/s</code>	<code>sar -n EDEV 1 1, isegerr/s, orsts/s</code>	<code>sudo tcptop-bpfcc 1 1</code>

Resource	Utilization	Saturation	Errors	Per-process
RAM	<code>free -h (read available), top %MEM, vmstat</code>	<code>vmstat r/b, swap</code>	<code>dmesg, sudo oomkill-bpfcc</code>	<code>top -o+%MEM</code>
CPU	<code>top %Cpu(s), mpstat -P ALL 1</code>	<code>vmstat r, load avg</code>	<code>%st (steal)</code>	<code>top, mpstat</code>
OS	—	—	—	<code>sudo opensnoop-bpfcc, sudo execsnoop-bpfcc</code>
Application	app metrics	queue depth	error rate	<code>cProfile, pythonstat-bpfcc, pythonflow-bpfcc</code>

top %Cpu(s): `us` user · `sy` kernel · `ni` low-priority · `id` idle · `wa` I/O wait · `hi` hardware IRQ · `si` software IRQ · **st steal (a hypervisor took it)**

top interactive: `e/E` units · `m/t` bar graphs · `0` hide zeros · `f` fields · `L` locate · `V` forest · `1` per-CPU · `x` bold sort column · `W` save config

vmstat: `-s` readable summary (incl. `forks`) · `-f` fork count · `-d/-D` disk stats

Load generation (Ch. 2, 4, 8)

```
ab -t 30 -c 10 -l http://host/path/ # -t seconds, -c concurrency, -l ignore length
ab -c 1 -t 10 -H "Cookie: sessionId=x" \
  -H "Content-type: application/x-www-form-urlencoded" \
  -p query.txt http://127.0.0.1:5000/search # -p body file -> implies POST
stress --cpu 2 -m 1 -d 1 --timeout 30 # CPU/memory/disk workers
stress --hdd 1 --timeout 35 # disk writers
stress --vm 1 --vm-bytes 512M --timeout 30 # memory allocator
dd if=/dev/zero of=/tmp/file1 bs=512M count=15 # raw write throughput
```

Network fault injection (Ch. 4, 5, 10)

```
# tc - whole interface
sudo tc qdisc add dev eth0 root netem delay 500ms
sudo tc qdisc del dev eth0 root

# tc - one destination port only
sudo tc qdisc add dev lo root handle 1: prio
sudo tc filter add dev lo protocol ip parent 1: prio 1 u32 match ip dport 3306 0xffff flowid 1:1
sudo tc filter add dev lo protocol all parent 1: prio 2 u32 match ip dst 0.0.0.0/0 flowid 1:2
sudo tc qdisc add dev lo parent 1:1 handle 10: netem delay 2000ms
sudo tc qdisc add dev lo parent 1:2 handle 20: sfq
telnet 127.0.0.1 3306 # VERIFY the injection before trusting the result

# Pumba - Docker
pumba netem --duration 60s --tc-image gaiadocker/iproute2 \
  delay --time 100 --jitter 0 --correlation 0 "re2:meower_db"

# Toxiproxy - any TCP
toxiproxy-cli -h $URL create chaos -l 0.0.0.0:8080 -u localhost:9090
toxiproxy-cli -h $URL toxic add --type latency --a latency=250 --upstream chaos
toxiproxy-cli -h $URL inspect chaos
```

Toxiproxy toxics: `latency` · `down` · `bandwidth` · `slow` · `close` · `timeout` · `slider`

Docker (Ch. 5)

```

docker run --name x -ti --rm <image:tag> <cmd>
docker images / docker ps / docker ps --all
docker inspect <name>
docker inspect -f '{{ .State.Pid }}' <name>
docker inspect -f '{{ .Id }}' <name>
docker inspect -f '{{ .GraphDriver.Data.MergedDir }}' <name>
docker build -t <tag> .
docker network ls | docker network create --driver bridge --attachable --subnet ... --ip-range ...
<name>
docker run --cpus=0.5 ... / --cpu-shares=N / --memory=128m / --oom-kill-disable
docker run --cap-drop ALL / --cap-add <CAP> / --security-opt seccomp=./profile.json
docker run --network none|host|bridge|<name>
docker swarm init && docker stack deploy -c stack.yml <name> && docker stack ls

```

Namespaces and cgroups:

```

lsns / sudo lsns / lsns --type pid / sudo lsns --task <pid> / lsns --json
ls -l /proc/$$/ns / readlink /proc/$$/ns/pid
sudo nsenter --pid --target <pid> <cmd>
sudo nsenter --net=/proc/<pid>/ns/net
sudo unshare --fork --pid --mount-proc /bin/bash
sudo chroot <dir>
ldd $(which bash) # find the .so files to copy into a chroot
getpcaps <pid>
ls -al /sys/fs/cgroup/
cat /sys/fs/cgroup/cpu/docker/$ID/cpu.stat # nr_periods, nr_throttled, throttled_time
cat /sys/fs/cgroup/memory/docker/$ID/memory.usage_in_bytes
echo 20971520 | sudo tee /sys/fs/cgroup/memory/docker/$ID/memory.limit_in_bytes
sudo cgcreate -g cpu:/name && sudo cgexec -g cpu:/name <cmd>

```

Syscalls (Ch. 6)

```

man man / man 2 syscalls / man 2 <name> / man 3 <name> / man unistd.h / man 7 libc
sudo strace <cmd>
sudo strace -C -S calls <cmd> # summary, sorted by count
sudo strace -p $(pidof <proc>) # attach
sudo strace -e <syscall> -p <pid> # filter
sudo strace -p <pid> -e inject=close:error=EIO
sudo strace -p <pid> -e inject=write:error=EIO:when=1+2
sudo syscount-bpfcc / sudo syscount-bpfcc -p <pid>
cc seccomp.c -lseccomp -o seccomp-example
curl <docker default seccomp json> | grep -v getpid > profile.json

```

-e inject= arguments: fault= · error=<errno> · retval=<value> · signal=<sig> · when=<n> / <n>+ / <n>+<step>

JVM (Ch. 7)

```

javac Example.java # -verbose for detail; -XDignore.symbol.file for jdk.internal
javap -c org.my.Example # PRINT THE BYTECODE
jar vcmf manifest.mf agent.jar org/agent
java -javaagent:./agent.jar -classpath "./lib/*" <MainClass>
java -javaagent:byteman.jar=script:throw.btm ...
java -javaagent:byte-monkey.jar=mode:fault,rate:0.5,filter:<class>/<method> ...
grep -n -r ")" throws" . # map the failure surface

```

The manifest needs `Premain-Class: <class with premain>`. ASM: `ClassReader` → `ClassNode` → filter methods → `InsnList + MethodInsnNode(INVOKESTATIC, ..., "(V)", false) → maxStack += 1 → insertBefore(getFirst())` →

`ClassWriter.toByteArray()`.

Browser (Ch. 9)

```
// latency
const originalSend = window.XMLHttpRequest.prototype.send;
window.XMLHttpRequest.prototype.send = function(){
  let that = this;
  setTimeout(function(){ return originalSend.apply(that); }, 1000);
}
// failure
this.dispatchEvent(new Event('error'));
// fetch
const original = window.fetch;
window.fetch = function(){ return original.apply(this, [...arguments]); }
```

Firefox: **Ctrl-Shift-E** Network · **Ctrl-Shift-K** Console · throttling drop-down (GPRS / Good 2G / DSL) · Greasemonkey or Tampermonkey to persist scripts.

Kubernetes (Ch. 10–12)

```
minikube start --driver=virtualbox / stop / service <svc> [--url] / tunnel
kubectl get pods [-A|-o wide|-o name|--watch|-l app=x]
kubectl apply -f file.yml / kubectl delete -f file.yml
kubectl describe svc <name> # compare Endpoints against pod IPs!
kubectl cp / kubectl exec
kubectl <cmd> --help
# kill a random labelled pod:
kubectl get pods -l app=goldpinger -o name | sort --random-sort | head -n 1 | xargs kubectl delete
```

```
powerfulseal autonomous --policy-file policy.yml
powerfulseal --help
```

Policy building blocks: `config.runStrategy` (`runs: 1`, or `minSecondsBetweenRuns/maxSecondsBetweenRuns`) · `podAction` (`matches` → `filters` → `actions`) · `nodeAction` · `clone` + `mutations` · `kubectl` (+`autoDelete`) · `wait` · `probeHTTP` · `actions` `kill`, `stop`, `stopHost` (+`autoRestart`)

6. Experiment-design checklist

Before

- ☐ Do I know what this system does well enough to guess where it's fragile? (Architecture + logs + `strace/syscount`/Network tab)
- ☐ Is my metric **user-visible** rather than implementation-visible?
- ☐ Can I measure it **reliably**, and does measuring it perturb the system?
- ☐ Have I measured the steady state **over a long enough window** to include periodic behaviour?
- ☐ Is the steady state measured **with the injection tooling attached**?
- ☐ Is my hypothesis **falsifiable and numeric**? ("If X, then Y stays under N.")
- ☐ Is the failure I'm injecting one the system **genuinely admits is possible**? (man page ERRORS, `throws` declarations, documented events)
- ☐ Is the injection **calibrated** against a measured baseline, not an assumed one?
- ☐ What is the blast radius — implementationally (precise targeting) and strategically (environment, subset, timing)?
- ☐ What is the **teardown**, and does it run even if the experiment crashes?
- ☐ Am I running exactly **one** experiment?
- ☐ **Should I do this in production?**

During

- ☐ Verify the injection landed **only** where intended.
- ☐ Watch more than one signal: health *and* latency; UI *and* logs; metric *and* process state.

After

- ☐ Did the result differ from the hypothesis, and **why** — mechanistically?
- ☐ Could the result be an artefact of my tooling? (Run a near-zero control injection.)
- ☐ What confounders does my setup contain, and did I say so?
- ☐ What is the operational lesson, and who needs to hear it?
- ☐ Should this become a **continuous** experiment?
- ☐ Did I clean up?

7. Observability checklist

- ☐ **Per resource:** utilization, saturation and errors — CPU, RAM, block I/O, network, plus software resources (fds, threads, PIDs).
- ☐ **Per layer:** hardware → OS → runtime and libraries → application.
- ☐ **Per process:** can I attribute load to a *process*, not just to the host? (*biotop*, *tcptop*, *top*, *mpstat*)
- ☐ **Distribution, not averages:** p90/p95/p99, not the mean.
- ☐ **Continuous, not sampled:** metrics in a time-series database, not a dashboard I refresh by hand.
- ☐ **Both binary and continuous signals:** a health check that passes tells you nothing about how close to its threshold you are.
- ☐ **Alerting headroom:** does the dashboard show how close values are to the limits that cause time-outs?
- ☐ **Errors are visible even when degradation is graceful** — a swallowed exception, a logged-but-invisible failure.
- ☐ **Exit codes actually reflect outcomes** (Ch. 7's silent success).
- ☐ The measurement doesn't cost more than the thing measured (*strace*'s 100×).

8. Failure-injection matrix

Failure	Host / process	Container	Kubernetes	Application	Notes
Process death	<i>kill</i> , <i>pkill</i> [-9]	<i>docker stop/kill</i> , <i>pumba kill</i>	<i>kubectl delete pod</i> , PowerfulSeal <i>podAction: kill</i>	—	Exit code 128+n
Machine death	power off	—	PowerfulSeal <i>nodeAction: stop, stopHost</i> (<i>autoRestart</i>)	—	Cloud APIs: AWS, Azure, GCP, OpenStack
CPU starvation	<i>stress --cpu</i> , cgroup <i>cpu.cfs_quota_us</i> , <i>nice</i>	<i>--cpus</i> , <i>--cpu-shares</i>	resource limits	—	<i>cpu.stat nr_throttled</i> proves it
Memory exhaustion	<i>stress --vm, perl -e 'while(1) {\$a.="A"x1024;}'</i> , fork bomb	<i>--memory</i> , <i>memory.limit_in_bytes</i>	resource limits	—	Allocation ≠ residency
Disk full	<i>fallocate</i> loop	second container on the same FS	—	—	Not isolated by default under overlay2

Failure	Host / process	Container	Kubernetes	Application	Notes
Disk slowness	<code>stress --hdd</code>	cgroups v2 I/O limits	—	—	Calibrate against <code>iostat/dd</code>
Network latency	<code>tc qdisc ... netem delay</code>	<code>pumba netem delay</code>	Toxiproxy sidecar; PowerfulSeal <code>toxiproxy</code> mutation	<code>time.sleep</code> wrapper; <code>setTimeout</code>	Targetable by port with <code>u32</code> filters
Network drop / partition	<code>iptables -j DROP</code>	<code>pumba netem loss</code>	network policy; Toxiproxy <code>down</code>	—	DROP (silence) ≠ REJECT (fail fast)
Bandwidth throttling	<code>tc</code>	<code>pumba</code>	Toxiproxy <code>bandwidth</code>	browser throttling presets	—
Syscall failure	<code>strace -e inject=...:error=...</code>	seccomp profile	—	—	strace = flexible + 100x cost
Syscall blocking	<code>libseccomp</code>	<code>--security-opt seccomp=</code>	pod security context	—	Near-zero cost, coarse control
Exception in a method	—	—	—	javaagent/ASM, ByteMan, Byte-Monkey, decorator	Choose a declared exception
Dependency error	—	—	—	wrapper class, decorator	Gate behind an env var
Frontend request failure	—	—	—	<code>dispatchEvent(new Event('error'))</code>	Refresh = teardown
Partial failure (rate)	<code>when=n+step</code>	—	<code>randomSample: ratio</code>	<code>rate:0.5, counter % 2</code>	Usually more realistic than total failure

9. Docker mental model

```

DOCKER = convenience (dockerd, CLI, Dockerfile, image format, registry, Hub, protocol)
          on top of
KERNEL FEATURES:
  chroot      → what filesystem root a process sees
  namespaces  → WHAT A PROCESS CAN SEE (mnt, pid, net, ipc, uts, user, cgroup, time)
  cgroups     → WHAT A PROCESS CAN USE (cpu, memory, blkio, pids, ...)
  capabilities → which privileged actions it may take (CAP_KILL, CAP_SYS_CHROOT, ...)
  seccomp     → which syscalls it may make (implemented with BPF)
  filesystems → union FS / overlay2: read-only layers + a writable top layer, copy-on-write
  networking  → none | host | bridge (docker0 + veth per net namespace)

```

Facts to remember

- Docker creates new `mnt`, `uts`, `ipc`, `pid` and `net` namespaces — **but shares `cgroup` and `user` with the host.**
- The **same inode** appears inside and outside a container. Isolation is thin.
- **Storage is not limited by default.** `--storage-opt size=` needs xfs plus pquota under overlay2.
- `--cpus` is a **hard** quota (period/quota); `--cpu-shares` is a **soft** weight, enforced only under contention.
- `--memory` limits **resident** memory; the cgroup OOM Killer enforces it, and reaping a fork bomb costs real CPU.
- Docker grants **14 capabilities by default**, including `cap_sys_chroot`. `--cap-drop ALL` removes them.

- Namespace errors read as **"not found"**; capability errors read as **"operation not permitted."**

10. Kubernetes mental model

CONTROL PLANE (loosely coupled, asynchronous, eventually consistent)

kube-apiserver the only door; stateless; everything talks to it
 etcd all state; **Raft** consensus; odd **sizes** (3 or 5); strongly consistent
 kube-controller-manager per-resource control loops; **Deployment** → **ReplicaSet** → **Pods**; leader
 via etcd lease
 kube-scheduler filter nodes, then rank them; leader via etcd lease
 kube-cloud-manager cloud **resources** (optional)

NODE

kubelet starts/stops containers, reports actual state; per-node **SPOF**
 pause container holds the pod's **shared resources** (above all the IP / net namespace)
 container runtime **Docker** → **containerd** → **runc** | **CRI-O** → **runc** | **gVisor** | **Kata** |
Firecracker

NETWORKING

pod-to-pod route made-up IPs from per-node subranges; CNI plugin (Flannel, Calico, ...) makes them
 service made-up IPs; kube-proxy via iptables (probability rules, $O(n)$ evaluation) or
IPVS
 ingress a resource that does nothing without an ingress controller (e.g. **NGINX**
 regenerating config)

OBJECTS

Pod = co-located containers sharing an IP · Deployment = blueprint + lifecycle · Service =
 stable IP for matched pods
 Labels drive: deployment→pod ownership, service→pod routing, app-level discovery, and your
 targeting
 RBAC = ClusterRole (verbs on resources) + ServiceAccount (workload identity) +
 ClusterRoleBinding

Failure facts

- Every component's failure mode is **"stops converging"** — staleness and latency, not errors.
- Even-numbered etcd clusters **reduce** fault tolerance. Quorum = $n/2 + 1$.
- A dead **kubelet** means your changes are accepted and never applied on that node.
- Node loss takes a **NotReady timeout (default ~5 min)** plus a **second eviction timeout** before pods move.
- A dead **kube-proxy** or CNI daemon leaves routing **stale and possibly wrong** — traffic to the *wrong* service.
- Thousands of **even empty** services slow all nodes under the iptables backend.
- An ingress that times out **faster** than its upstream manufactures retry storms.

11. Injection-layer comparison: syscall vs. JVM vs. application vs. browser

	Syscall (Ch. 6)	JVM bytecode (Ch. 7)	Application code (Ch. 8)	Browser JS (Ch. 9)
Mechanism	ptrace via strace ; seccomp-BPF	java.lang.instrument · ASM	Wrapper class / decorator	Override a prototype method or a global function
Needs source?	No	No	Yes	No
Needs a restart?	No (-p)	Yes for premain , no for agentmain	Yes	No
Granularity	One syscall, nth call	One class + method	One function or client	One request API

	Syscall (Ch. 6)	JVM bytecode (Ch. 7)	Application code (Ch. 8)	Browser JS (Ch. 9)
Partial failure	<code>when=n+step</code>	<code>rate:</code> (Byte-Monkey), <code>IF</code> (Byteman)	<code>counter % 2</code>	<code>counter % 2</code>
Failure types	errno, return value, signal	Any exception; latency; null args; short-circuit	Anything you can code	Latency; any dispatched event
Overhead	~100× (strace) ; ~0 (seccomp)	Small; agent runs at class load	One <code>if</code> when off	Negligible
Teardown	Ctrl-C / restart	Restart without the agent	Unset env var	Refresh the page
Blast radius control	PID + syscall + call number	Class-name filter (the only one)	Env-var gate	The tab
Production-safe?	seccomp yes, strace no	With care (<code>agentmain</code> , class filter)	Yes, if off by default	Per-tab only
Best for	Any binary, any language, no source	Java/JVM services	Your own services, precise dependencies	SPAs, third-party frontends

12. SLI / SLO / SLA reference

Risk → cost per unit of time → SLI (the number) → SLO (the agreed target) → SLA (the contract + penalty)

↑

chaos experiments continuously verify this

Examples from the book

System	SLI	SLO
Consumer photo site	Ratio of success responses to errors	> 99.95% monthly average
Trading API	99th percentile response time	< 25 ms, 99.999% of the time
PaaS deploy time	Time from request to service serving traffic	< 30 s internally (SLA says 60 s)

Number of nines

Availability	Downtime / year	Downtime / day
90%	36.53 days	2.4 hours
99%	3.65 days	14.40 minutes
99.95%	4.38 hours	43.20 seconds
99.999%	5.26 minutes	840 milliseconds

Practice: set the internal SLO **tighter than the contractual SLA** so alerts fire before penalties do. Verify it with a continuously running experiment, not a one-off measurement. Reference cost: Amazon, 2013, **\$66,240 per minute of downtime**.

Scale arithmetic: MTTF 5 years → ~0.05%/day/server → ~1% at 20 servers, ~10% at 200, daily at thousands. MTBF 10 years → ~3 failures/day at 10,000 servers. A 98%-bug-free team on weekly releases ships bugs more than once a year; 25 such teams, every other week.

13. Common failure patterns

Pattern	Shape	Where it appears
Silence vs. rejection	A dropped packet hangs; a refused connection fails fast. Code that handles one may be defenceless against the other.	Ch. 1 (cache), Ch. 4, Ch. 9
Missing time-outs	Any network call on a request path without a time-out is an unbounded wait.	Ch. 1
Latency multiplication	Added latency × number of round trips. 2 s → 54 s; 100 ms → 464 ms.	Ch. 4, 5, 8
Retry amplification / metastable failure	Retries at several layers overwhelm a recovering dependency; it never recovers.	Ch. 1 (DNS), Ch. 6, Ch. 12 (ingress)
Restart rate limits	A supervisor gives up after N restarts in a window. Restart=always is not always.	Ch. 2, Ch. 12 (kubelet)
Noisy neighbour	Slowness with no bug in your code. Fix at the platform layer with cgroups and limits.	Ch. 3, Ch. 5
Unisolated resource	Something you assumed was contained is not — disk under overlay2, kernel across containers.	Ch. 5
Silent failure / lying exit code	Produces nothing, reports success. Nothing monitoring status will ever alert.	Ch. 7
Swallowed exception	The error path was never exercised; the handler itself is broken.	Ch. 7, Ch. 9 (<code>parseJSON(xhr.responseText)</code>)
Stale data with no error	Worse than an outage — the user cannot tell they're looking at the wrong thing.	Ch. 9, Ch. 12, Ch. 13 (Liar, Liar)
Green health check, severe degradation	A binary check hides everything below its threshold.	Ch. 10
Wrong routing	A dead networking daemon leaves routing stale <i>and pointing somewhere else</i> .	Ch. 12
Resilience hiding the defect	Fixing the symptom removes the pressure to fix the cause. Keep the underlying failure on a dashboard.	Ch. 2
Slow-start masking	A workload that behaves for the first N seconds and then degrades. Sample your steady state long enough.	Ch. 3
Dependency ordering on restart	Everything restarts at once; the thundering herd; readiness vs. liveness.	Ch. 5
Human SPOF	One person holds knowledge nobody else has.	Ch. 13

14. Production safety and blast-radius checklist

Strategic (planning)

- ☐ Roll the experiment out on **a small subset of traffic first**, then expand.
- ☐ Run it in **QA before production**.
- ☐ **Automate early**, so findings are reproducible.
- ☐ **Be careful with randomness** — it finds race conditions but makes results hard to reproduce.
- ☐ Choose the **environment** deliberately; the VM exists so you can be reckless somewhere safe.
- ☐ Scope by **availability zone or region** so a mistake cannot cross a boundary.
- ☐ Consider **adding degraded capacity** instead of degrading running capacity.
- ☐ Ask explicitly: **"Should I do that in the production environment?"**

Implementational (execution)

- ☐ Target **precisely** — narrow the selector, use PIDs from a trusted source, use labels and namespaces, match a specific port, filter on a fully-qualified class name.
- ☐ Read the tool's own **match and filter counts** before the action runs.
- ☐ Bound the failure: `--duration, when=n+step, rate:, randomSample, replicas:.`
- ☐ **Script the teardown** and prefer tools that revert automatically (`--duration, autoRestart, autoDelete, --rm`, page refresh).
- ☐ Gate application-level injectors behind an **env var, off by default**.
- ☐ **Verify the injection hit only its target** before believing the result.
- ☐ Have a **stop condition** and someone watching.

Testing in production — the decision

It "boils down to whether you prefer the risk of hurting a portion of production traffic **now**, or potentially running into the bug **later**." Testing outside production is by definition incomplete: **data, scale, user behaviour and configuration all drift**. But production experiments are only defensible **with blast-radius controls in place**, and never as a substitute for the earlier testing stages.

15. Glossary

Affinity / anti-affinity — rules that items should or must (or shouldn't or mustn't) run in the same partition. **aqu-sz** — average queue length of requests issued to a block device; a saturation metric. **Availability zone** — a partition within a region separated by redundant power, network and hardware. **BCC** — BPF Compiler Collection; wrappers and example tools over eBPF. **BPF / eBPF** — kernel execution engine running safe, bounded programs on kernel events. **Capability** — a granular unit of superuser privilege (`CAP_KILL, CAP_SYS_CHROOT`). **cgroup** — kernel feature limiting and accounting resource use by a group of processes. **chroot** — change the filesystem root as seen by a process. **CNI** — Container Networking Interface; implements pod-to-pod networking. **Consensus / Raft** — algorithm by which replicas agree on one version of reality; leader election by majority with heartbeats. **Control plane** — the Kubernetes components implementing convergence to the desired state. **Copy-on-write (COW)** — a file modified on a lower layer is copied wholesale into the writable layer. **CRI** — Container Runtime Interface. **Dark debt** — unknown unknowns in a complex system. **DaemonSet** — one pod per node. **Deployment** — blueprint plus lifecycle management for a set of pods. **Filesystem bundle** — the OCI term for an unpacked image. **Fuzzing** — feeding pseudorandom payloads to find errors written tests miss. **glibc** — the most common C library on Linux; its wrappers are often more than pass-throughs. **Goldpinger** — tool that builds a full node-to-node connectivity graph by pinging its own peers. **Ingress** — a Kubernetes resource mapping hosts to services; inert without a controller. **javaagent** — the JVM flag attaching a JAR that can inspect and rewrite loaded classes. **Kubelet** — the per-node agent that starts and stops containers. **Label** — a key-value pair used to match sets of Kubernetes resources. **libseccomp** — higher-level library for managing seccomp filters. **MTTF / MTBF** — mean time to / between failure. **Namespace** — kernel feature filtering which resources a process can see. **netem** — the `tc` network-emulator qdisc (delay, loss, duplication, corruption, reordering). **OCI** — Open Container Initiative; runtime and image specifications. **Overlay network** — routing made-up pod IPs between nodes, usually by encapsulation. **pause container** — holds a pod's shared resources (notably its IP) while other containers restart. **Pod** — co-located containers sharing an IP and some resources; the schedulable unit. **PowerfulSeal** — YAML-driven chaos tool for Kubernetes and cloud VMs. **ptrace** — the syscall `strace` uses to control other processes. **Pumba** — Docker chaos tool wrapping `tc`, stress-ng and container kill. **qdisc** — queueing discipline, that is, a packet scheduler (nothing to do with disks). **Region** — a geographically and utility-independent group of datacentres. **runc** — the low-level Linux container runtime beneath containerd and CRI-O. **seccomp** — kernel feature filtering which syscalls a process may make, implemented with BPF. **Service** — a Kubernetes resource giving a stable IP that resolves to a set of matched pods. **SPA** — single-page application; only the first page is served, the rest is rendered by JavaScript. **Steal time (st)** — CPU time a hypervisor gave to someone else. **Toxic** — an injected failure attached to a Toxiproxy configuration. **Toxiproxy** — configurable TCP proxy for simulating network problems. **Union filesystem / overlay2** — merging layered directories into one view. **unshare** — command that creates new namespaces and starts a process in them. **USDT probe** — user statically defined tracing point compiled into an application. **USE** — utilization, saturation, errors. **VXLAN** — an encapsulation backend used by overlay networks. **Watch** — the Kubernetes notification mechanism served by kube-apiserver.

16. Rapid exam review — 40 questions

1. Define chaos engineering, and say what it is *not*.
2. Name the four (really five) steps of an experiment.
3. What's the difference between an SLI, an SLO and an SLA?
4. How much downtime per year is 99.95%?
5. Give the book's canonical example of an emergent property.
6. What does exit code 143 mean, and what are the two things that could have caused it?

7. Which command reveals an OOM kill, and what strings do you look for?
8. Why is `Restart=always` not always?
9. Define blast radius and name two strategic and two implementational controls.
10. Spell out USE and give the metric for each letter for block I/O.
11. Why isn't high saturation automatically a problem?
12. Why does `free -h` show almost no free memory on a healthy machine, and which column do you actually read?
13. What is steal time and when does it appear?
14. Which BCC tool answers "which process is doing this?" for disk, for network, and for OOM kills?
15. Why is `nice` a weaker fix than cgroups?
16. What is a qdisc, and what does `netem delay` do?
17. Why did 2 seconds of injected database latency become 54 seconds of page load?
18. State the argument for testing in production in one sentence, and the precondition.
19. Contrast full virtualization with OS-level virtualization on security, overhead and kernel.
20. Namespaces vs. cgroups, in one line each.
21. Which two namespace types does Docker *not* create per container by default?
22. Why can one container fill the disk for all the others, and what would it take to stop it?
23. What does `cpu.stat's nr_throttled` tell you?
24. Why did `stress --vm 1 --vm-bytes 512M` succeed inside a 128 MB container?
25. Explain Pumba's `--tc-image` trick and what it proves about namespaces.
26. Name three ways to discover a black-box binary's behaviour without its source.
27. How much does `strace` cost, and how do you know?
28. Decode `-e inject=write:error=EIO:when=1+2`.
29. Why read `man 2 close's` ERRORS section before injecting into `close`?
30. What are the four steps to building a javaagent?
31. Why inject `invokestatic` rather than the exception itself, and what must you also increment?
32. What was wrong with FizzBuzzEnterpriseEdition's behaviour, and why is it worse than a crash?
33. Give the three rules for building chaos code into your own application.
34. Why is a decorator that returns the original function better than one that checks an env var per call?
35. How do you inject latency into someone else's SPA, and how do you revert it?
36. Why was pgweb showing stale data with no error?
37. What does "Matched 3 / Initial 3 / Filtered 1" tell you, and when do you read it?
38. Why do 250 ms of latency leave a Goldpinger graph fully green?
39. Why are even-numbered etcd clusters a mistake?
40. Name the four human-team games and what each one finds.

Answer key (one line each): 1 — experimenting on a system to build confidence it withstands turbulent conditions; not random destruction, not a tool, not a test replacement, not production-only, not chaos theory. 2 — observability, steady state, hypothesis, run, (analysis). 3 — the number, the agreed target, the contract with a penalty. 4 — 4.38 hours. 5 — DNS restart + layered retries = permanent downtime nobody's component can cause alone. 6 — 128+15 = SIGTERM; an explicit `kill` or the OOM Killer. 7 — `dmesg | grep -i <proc>`; "Out of memory: Kill process", "oom_reaper". 8 — `DefaultStartLimitBurst=5` per `DefaultStartLimitIntervalSec=10s`. 9 — max things your experiment can affect; strategic: subset of traffic, QA first (also: automate, careful with randomness); implementational: narrow selectors, target by trusted PID/label/port. 10 — utilization, saturation, errors; `%util/df -h, aqu-sz, dmesg/device` errors. 11 — batch systems *want* full utilization. 12 — the kernel caches disk in spare RAM and returns it on demand; read `available`. 13 — `%st`, the hypervisor gave your cycles elsewhere; virtualized environments only. 14 — `biotop, tcptop, oomkill`. 15 — `nice` is a relative priority, cgroups set absolute allocations. 16 — a queueing discipline (packet scheduler); the network-emulator qdisc adds delay. 17 — WordPress makes ~27 database round trips per page and each absorbs the full delay. 18 — testing outside production is by definition incomplete (data, scale, behaviour, config drift) — but only with blast-radius controls and never instead of earlier stages. 19 — VM: own kernel, stronger isolation, higher overhead; container: shared kernel, weaker isolation, lower overhead. 20 — namespaces limit what a process can see; cgroups limit what it can use. 21 — `cgroup` and `user`. 22 — container filesystems are layers on one host filesystem; `--storage-opt size=` with xfs + pquota under overlay2. 23 — how many periods the cgroup was throttled — proof a CPU limit is biting. 24 — the limit applies to resident memory and `stress` never touches what it allocates. 25 — it starts a throwaway container holding `tc` inside the target's **net namespace**; namespaces are joinable. 26 — `strace/syscount, opensnoop/execsnoop`, the browser Network tab (also `javacalls, javap -c`). 27 — ~100x; measured with `dd` doing 500k one-byte read/write pairs, filtering a syscall `dd` never makes. 28 — fail the `write` syscall with EIO on the 1st call and every 2nd call thereafter. 29 — to know the injected error is one the system can really produce (EINTR from any signal; NFS ENOSPC surfacing at close). 30 — transformer class, `premain` class, JAR with `Premain-Class, -javaagent:`. 31 — a static no-arg void call is exactly one instruction; `method.maxStack += 1`. 32 — it produced no output and still exited 0 — a silent failure no exit-status monitoring can catch. 33 — keep it simple; optional and off by default; negligible

performance impact. 34 — the env check runs once at decoration time, so the disabled path costs nothing at call time. 35 — override `XMLHttpRequest.prototype.send` (or `window.fetch`) from the console, delegate with `.apply`; refresh the page. 36 — its shared error handler called `parseJSON(xhr.responseText)` on a transport failure, threw, and died. 37 — PowerfulSeal's blast radius for this run; read it before the action line. 38 — the health check is binary against a 300 ms timeout; latency shows only in the heatmap. 39 — quorum is $n/2 + 1$, so four nodes tolerate the same single failure as three while adding another thing that can fail. 40 — Staycation (knowledge SPOFs), Liar, Liar (input validation and trust), Life in the Slow Lane (bottlenecks), Inside Job (remediation procedures).

Completeness Audit

Performed against the full extracted text of all 13 chapters and Appendices A–D before publishing.

A. Coverage by chapter

Ch.	Sections in the book	Covered here	Experiments captured
1	1.1–1.5 + summary	All, including the "number of nines" box, the randomness and fuzzing box, the observability-in-quantum-realm box, Mendel, and the datacentre power-supply illustration	Card 1.1 (cache cut off)
2	2.1–2.6 + 4 pop quizzes	All, including VM setup steps and gotchas, the <code>kill -L</code> signal table, OOM tunables, NGINX <code>max_fails/fail_timeout</code> , Figures 2.1–2.5	Cards 2.1, 2.2
3	3.1–3.6 + 5 pop quizzes	All tools: <code>uptime</code> , <code>/proc/loadavg</code> , <code>dmesg</code> , <code>df</code> , <code>iostat</code> , <code>biotop</code> , <code>sar</code> (DEV/EDEV/TCP/ETCP with every field defined), <code>tcptop</code> , <code>free</code> , <code>top</code> (including the interactive key table and the field dialog), <code>vmstat</code> (all modes), <code>oomkill</code> , <code>/proc/cpuinfo</code> , <code>mpstat</code> , <code>cgcreate/cgexec</code> , <code>opensnoop</code> , <code>execsnoop</code> , <code>cProfile</code> , <code>pythonstat</code> , <code>pythonflow</code> , Node Exporter + Prometheus + Grafana, further reading	Card 3.1 (busy neighbour)
4	4.1–4.3 + 3 pop quizzes	All, including the full <code>tc</code> hierarchy with every command explained, the <code>telnet</code> verification step, the complete self-critique (single host, averages, reads vs. writes, NVMe, bursty traffic), and the internet-bank lifecycle	Cards 4.1, 4.2
5	5.1–5.12 + 5 pop quizzes	All, including the 1979–2013 timeline, VM/container pros and cons, the hybrid projects box, the union FS box, the inode proof, all 8 namespace types, <code>lsns</code> variants, the cgroup v1/v2 note, every CPU and memory cgroup file used, DIY container parts 1–3, all three network modes, the capability list, the seccomp profile, <code>docker stack deploy</code> , Pumba's gotcha and workaround, and all four "other parts of the puzzle"	Cards 5.1–5.5
6	6.1–6.5 + 4 pop quizzes	All, including the <code>man</code> section table, the full <code>sleep 1</code> syscall walk-through, the glibc non-pass-through finding, the <code>dd</code> overhead measurement, the full <code>-e inject</code> argument list, <code>man 2 close</code> 's four error causes, the strace/BPF side-by-side, SystemTap and Ftrace, and both seccomp routes with the four libseccomp functions	Cards 6.1, 6.2, 6.3
7	7.1–7.4 + 2 pop quizzes	All, including the <code>grep ") throws"</code> technique, the <code>javap -c</code> instruction walk-through, both agent packages, the full ASM listing annotated, <code>-XDignore.symbol.file</code> , the Byteman rule skeleton, Byte-Monkey's four modes, Chaos Monkey for Spring Boot's four assaults, all three research papers, and the <code>agentmain</code> note	Card 7.1
8	8.1–8.4 + 2 pop quizzes	All, including the full <code>app.py</code> structure, both listings, the <code>ab</code> POST flags, the complete discussion section (double-edged sword, the "duh" objection, the Redis-set design flaw, the performance argument), and the application-vs-infrastructure trade-off	Cards 8.1, 8.2
9	9.1–9.4 + 3 pop quizzes	All, including the Developer Tools workflow, the five JavaScript facts, both listings, the timestamp-correlation verification, the event model, the <code>parseJSON</code> root cause, the Fetch API, throttling, and Greasemonkey/Tampermonkey	Cards 9.1, 9.2

Ch.	Sections in the book	Covered here	Experiments captured
10	10.1–10.4 + 4 pop quizzes	All, including the ICANT documentation verbatim, the Kubernetes derivation, Minikube setup, pod/deployment/service, RBAC, labels, both YAML listings annotated, the endpoint-verification step, <code>kube-thanos.sh</code> , the four discussion caveats, KubelInvaders and Kube DOOM, the rejected injection options, all six Toxiproxy toxics, the two-container pod design, and the heatmap reading guide	Cards 10.1, 10.2
11	11.1–11.3 + 3 pop quizzes	All, including the minimal policy, installation, all three experiment policies annotated, the clone/toxiproxy mutation's four behaviours, the localhost exception, the epistemological argument, the five pod phases, the three sources of pod-start variability, <code>minikube tunnel</code> , <code>imagePullPolicy: Always</code> , MTTT arithmetic, regions/AZs/affinity, and both VM-experiment shapes	Cards 11.1, 11.2, 11.3
12	12.1–12.2 + 5 pop quizzes	All five control-plane components, Raft and the odd-number rule, all four etcd experiment ideas, both apiserver ideas, both controller-manager ideas, the scheduler's filters and priorities, kubelet with both experiment ideas and the two timeouts, the pause container, CRI/OCI/runc and all six runtimes, pod/service/ingress networking with all three sets of experiment ideas, and Table 12.1	Consolidated experiment map (16 targets)
13	13.1–13.4	All, including the trolley problem, MTBF arithmetic for hardware and for teams, the shark statistic, the four advantages of failing early, all six management arguments and three team arguments, game days, the skill Venn diagram with all six people, all four games with their tuning notes and safety rules, and the full further-reading list	Games 1–4 as structured cards
A–D	All	Package table, all non-packaged tool installs, WordPress config, Minikube per-OS, all pop-quiz answers, all ten Appendix C omissions with reasoning, and Appendix D's engineering jokes	—

B. Audit questions from the brief

Was any important theoretical concept omitted? No. Every named concept in the brief is present with its own treatment: motivations and risk; SLI/SLO/SLA; emergent properties; the four-step model; blast radius; Linux forensics; USE across all six resource groups; BCC/BPF; Prometheus and Grafana; testing in production; all seven Docker kernel features; syscalls and libc; BPF vs. strace; JVM bytecode and instrumentation; application-level injection; browser injection; Kubernetes objects, control plane, kubelet, runtimes and all three networking layers; continuous SLO verification; cloud-layer failure; and the cultural material.

Was any major experiment omitted? No. All 23 hands-on experiments and demonstrations have Experiment Cards, plus the 16 experiment *ideas* from Chapter 12, which the book presents as prompts rather than labs, and the four team games from Chapter 13.

Was any important command, tool or workflow reduced to a vague description? No. Every command appears with its flags decoded and its purpose explained, including the ones that are easy to copy without understanding: the `tc prio/u32/flowid` hierarchy, the cgroups file semantics, `unshare --fork --pid, nsenter` in both forms, `strace -e inject=` in full, `jar vcmf` and `-XDignore.symbol.file`, `ab's -p` and `-H`, PowerfulSeal's policy grammar, and `toxiproxy-cli's -h`-means-host trap.

Are theory and practice balanced? Yes, and deliberately per chapter. Theory-dominant chapters (1, 12, 13) still carry concrete commands, experiment ideas and structured cards. Practice-dominant chapters (2, 5, 10) carry the underlying theory in full — namespaces, cgroups, union filesystems, consensus, declarative convergence — rather than as a preamble.

Are practical results explained, not merely reported? Yes. Every card has a "Why it happened" or "Why" row giving the mechanism: systemd's start-limit window, resident vs. virtual memory, round-trip multiplication, prototype overriding, binary health checks against a 300 ms timeout, a `parseJSON` call on an empty response, and so on.

Does each chapter retain the book's intended lessons? Yes, including the uncomfortable ones the book states about itself: the deliberately weak Chapter 4 experiment design; the admission that the 2-second sleep in Chapter 2 was chosen to make the experiment pass; the Chapter 8 warning that your own chaos code can *reduce* confidence; the Chapter 5 concession that most systems never limit container storage; and Appendix C's admission that the four-step model is missing a step.

C. Known limits of this guide

- **Figures are described, not reproduced.** Screenshots — Goldpinger's UI and heatmap, the Prometheus and Grafana views, KubelInvaders, Kube DOOM, the Raft animation, Appendix D's photographs — are summarised by what they show and which failure paths they reveal.
 - **Long code listings are abridged where the book abridges them,** and annotated line by line where the mechanism matters: `ClassInjector.java`, `chaos.py`, `chaos2.py`, `container-ish-2.sh`, the two Goldpinger YAML files, and the three PowerfulSeal policies.
 - **Versions are the book's,** and several are now historically dated — Kubernetes 1.18.3, Minikube 1.12.3, Docker 19.03, Toxiproxy 2.1.4, Byteman 4.0.11, Python 3.7, OpenJDK 8. The author flags this himself. Component responsibilities have been stable; specific defaults and flags have not.
 - **Everything not from the book is labelled "Supplementary explanation."** Three such labels exist: the industry names for the retry-storm pattern (Ch. 1), the round-trip arithmetic derived from the book's own latency figures (Ch. 4 and Ch. 5), and a caution that the book does not equate gVisor's overhead with strace's measured figure (Ch. 12).
-

Hands-on Labs

The 23 practical labs for this guide are on their own page: [Chaos Engineering \(Pawlikowski\) — Hands-on Labs](#).

Cilium: Up and Running — Comprehensive Book Summary & Technical Reference - ChatGPT